Машинное обучение для физиков
Конспект билетов

Содержание

I Введение и базовая методология **5**

1 Определение и методология машинного обучения **5**
1.1 На каких дисциплинах основано . . . 5
1.2 Разница между классическим программированием и ML . . . 5
1.3 Базовые термины ML . . . 6

2 Приложения ML (подобласти) **6**
2.1 Назвать минимум 3 и привести практические примеры . . . 6

3 Наборы данных **7**
3.1 Способы находить датасеты (минимум 3 источника) . . . 7

4 Разведочный анализ данных (EDA) **7**
4.1 Зачем нужен EDA для моделирования, примеры . . . 7
4.2 Типичные шаги EDA . . . 8

II Вероятностные основы и постановка задач **9**

5 Оценка максимального правдоподобия (MLE) **9**
5.1 Определение правдоподобия . . . 9
5.2 Следствия свойства i.i.d. . . . 9
5.3 Формула отрицательного лог-правдоподобия . . . 9

6 Таксономия задач ML **10**
6.1 Обучение с учителем и без учителя, примеры . . . 10
6.2 Назвать другие категории . . . 10
6.3 Практические примеры (минимум 3 для каждого типа) . . . 10
6.4 Постановка практической задачи, основные сущности . . . 11

7 Наивный байесовский классификатор **11**
7.1 Формула Байеса . . . 11
7.2 Основные предположения . . . 11
7.3 Случай непрерывных признаков . . . 12

8 Библиотека sklearn **12**
8.1 Базовые классы, их функции и использование . . . 12
8.2 Важность пайплайнов, пример . . . 13

III Линейная регрессия, оптимизация и регуляризация **15**

9 Регрессионная модель **15**
9.1 Предположения . . . 15

10 Линейная регрессия **16**
10.1 Мотивация функции потерь MSE . . . 16
10.1.1 Доказательство для гауссовского случая . . . 17
10.2 Аналитическое решение для весов (матричная форма) . . . 18
10.2.1 Доказательство . . . 18
10.3 Теорема Гаусса–Маркова, BLUE . . . 18

11 Градиентный спуск для линейной регрессии **19**
11.1 Почему не используют матричное решение . . . 19
11.2 Стохастический градиентный спуск, преимущества . . . 20

12 Регуляризация в линейных моделях **20**
12.1 Мотивация регуляризации . . . 20
12.2 Гребневая регрессия Ridge (L_2) . . . 21
12.2.1 Формула решения . . . 21
12.3 Lasso-регуляризация (L_1) . . . 22
12.4 Вероятностная интерпретация . . . 22
12.5 Elastic Net . . . 23
12.6 Свойства регуляризаций . . . 23

13 Метрики регрессии MSE, MAE, RMSE, MAPE, SMAPE, R_2 **23**
13.1 Когда их используют . . . 24

14 Параметры и гиперпараметры **25**
14.1 Определения, свойства, использование, примеры . . . 25

IV Валидация и оценка качества **26**

15 Валидация и оценка модели **26**
15.1 Недообучение и переобучение, определения и способы обнаружения. . . . 26
15.2 Способы разбиения датасета . . . 26
15.2.1 Что такое хорошее разбиение, идеальный и практический варианты . . . 26
15.2.2 4 основных типа разбиений . . . 27
15.3 Этапы построения модели . . . 27
15.3.1 Train, validation, test, мотивация каждого . . . 28
15.3.2 Кросс-валидация . . . 29
15.3.3 Как выбрать организацию этапов . . . 29
15.4 Утечки данных . . . 30

16 Линейная классификация **31**
16.1 Постановка задачи классификации . . . 31
16.2 Варианты типов меток . . . 32
16.3 Определение линейного классификатора и геометрическая интерпретация . . . 32
16.4 Маржа, определение и ключевое свойство . . . 32
16.5 Выбор весов . . . 32
16.6 Почему для классификации много разных функций потерь . . . 33

17 Логистическая регрессия **33**
17.1 Базовое предположение и геометрическая интерпретация . . . 34
17.2 Сигмоида как предсказание вероятности, свойства . . . 35
17.3 Формула логистической функции потерь . . . 35
17.3.1 вывод через метод максимального правдоподобия . . . 36
17.4 Свойство калибровки вероятностей . . . 36
17.5 Обобщённая линейная модель . . . 36

18 Стратегии многоклассовой классификации **38**
18.1 One-vs-one, one-vs-all и их свойства . . . 39

19 Метрики классификации **39**
19.1 Определение метрики в ML . . . 40
19.1.1 Свойства, сравнение с функцией потерь . . . 40
19.2 Матрица ошибок (бинарная, многоклассовая) . . . 41
19.2.1 Типы ошибок . . . 42
19.2.2 Правило присвоения меток . . . 42
19.3 Accuracy, balanced accuracy, precision, recall, F1-score . . . 43
19.3.1 Sensitivity, specificity и другие . . . 44
19.3.2 Несбалансированная стоимость ошибок, примеры, как исправлять дисбаланс . . . 44
19.4 ROC AUC . . . 46

19.4.1 Свойства ROC и ROC-AUC . . . 46
19.4.2 PR AUC . . . 47
19.5 Многоклассовые расширения . . . 48
19.5.1 Микро- и макро-усреднение . . . 48

20 Матричные нормы **49**
20.1 p-нормы, самые важные частные случаи . . . 49

21 Метод опорных векторов (SVM) **49**
21.1 Геометрическая интерпретация . . . 49
21.2 Оптимизационная задача для линейно разделимого случая . . . 50
21.2.1 Двойственная задача . . . 50
21.3 Неразделимый случай, slack-переменные . . . 50
21.4 Hinge loss . . . 52
21.5 Ядровой трюк, свойства ядер, примеры ядер . . . 53

V Методы без учителя и методы ближайших соседей **54**

22 Метод главных компонент **54**
22.1 Связь с SVD . . . 54
22.2 Теорема Эккарта–Юнга, доказательство . . . 54
22.2.1 способ через собственное разложение или через подстановку . . . 55
22.3 Две альтернативные итеративные формулировки . . . 56
22.4 Предобработка для PCA . . . 57

23 k ближайших соседей (kNN) для классификации и регрессии **58**
23.1 Параметры и гиперпараметры . . . 59

24 Индексы для kNN **59**
24.1 Зачем нужны индексы . . . 59
24.2 Идеи индексов для kNN от простых к сложным . . . 59
24.3 k-dimensional tree (kd-tree) . . . 59
24.4 Annoy . . . 60
24.5 HNSW . . . 60
24.5.1 Построение графа . . . 61
24.5.2 Использование графа . . . 61
24.6 Примеры векторных баз данных . . . 61

25 Горький урок **61**
25.1 Основные выводы статьи . . . 61

VI Деревья решений, ансамбли, интерпретируемость **62**

26 Деревья решений **62**
26.1 Определение жадных алгоритмов . . . 62
26.2 Процедура построения . . . 62
26.3 Информационные критерии, энтропия, индекс Джини . . . 62
26.4 Ответ в листе . . . 63
26.5 Обработка пропусков . . . 63
26.6 Бинаризация . . . 63
26.7 Обрезка (pruning), варианты C4.5 и CART . . . 63
26.8 Гиперпараметры (минимум 4 основных) . . . 64

27 Разложение bias–variance–noise **64**
27.1 Случайность в ошибке . . . 64
27.1.1 Интуиция каждой компоненты . . . 65
27.2 Доказательство для MSE . . . 65
27.3 Компромисс bias–variance . . . 67
27.4 Анализ основных моделей . . . 67
27.5 За пределами MSE . . . 67

27.6 Double descent . . . 67

28 Bagging **67**
28.1 Мотивация через bias–variance . . . 68
28.2 Bootstrap . . . 68
28.3 Метод случайных подпространств . . . 68
28.4 Случайный лес . . . 69

29 Задача обнаружения аномалий **69**
29.1 Isolation Forest . . . 69
29.2 One-Class SVM . . . 69

30 GB **69**
30.1 Процедура построения для MSE . . . 70
30.2 Бустинг как градиентный спуск . . . 71
30.2.1 вывод для кросс-энтропии . . . 72
30.3 Выбор базовой модели . . . 73
30.4 Параллелизм . . . 73

31 Библиотеки градиентного бустинга **73**
31.1 XGBoost, LightGBM, CatBoost . . . 73
31.2 Плюсы и минусы каждой . . . 74

32 Интерпретируемое ML **74**
32.1 Таксономия по свойствам . . . 74
32.2 Линейные модели, важности в деревьях . . . 74
32.3 LIME . . . 74
32.4 Подход SHAP (Shapley additive explanations) . . . 74
32.4.1 Аксиомы, формула . . . 74
32.4.2 SHAP-значения для ML . . . 74
32.4.3 KernelSHAP, TreeSHAP . . . 74
32.5 GradCAM . . . 74

33 Оптимизация гиперпараметров (HP) **74**
33.1 Классы задач оптимизации . . . 74
33.2 Основные подходы к оптимизации HP . . . 75
33.3 Библиотеки для оптимизации HP . . . 75

VII Нейросети, матричные производные и базовые слои **75**

34 Матричное исчисление и производные матричных операций **75**
34.1 Производная скалярного произведения . . . 76
34.1.1 Для двух векторов . . . 76
34.1.2 Для матрицы и вектора . . . 76

Часть I
Введение и базовая методология

1 Определение и методология машинного обучения

1.1 На каких дисциплинах основано

Машинное обучение – это наука, изучающая алгоритмы, автоматически улучшающиеся благодаря опыту. Для построения таких методов используются средства математической статистики, численных методов, математического анализа, методов оптимизации, теории вероятностей, теории графов, различные техники работы с данными в цифровой форме.

1.2 Разница между классическим программированием и ML

Классическое программирование – это автоматизация хода мыслей человека, а машинное обучение – автоматизация процесса вывода мышления из данных. Более подробно в Таблице 1

Аспект	**Классическое программирование**	**Машинное обучение**
Основная задача	Выполнение четко определенной последовательности инструкций	Обобщение на основе данных и прогнозирование для новых, ранее невиданных случаев
Правила	Явно задаются программистом в виде алгоритмов, условий (if-else), циклов	Выводится автоматически из данных в процессе обучения. Скрыта внутри параметров модели (например, весов нейронной сети)
Входные данные	Часто структурированные (числа, строки, объекты)	Любые данные: таблицы, текст, изображения, звук. Качество и количество данных критически важны.
Объект обучения	Нет. Программа статична, пока код не изменится	Параметры модели настраиваются в процессе обучения.
Критерий успеха	Корректность выполнения алгоритма, отсутствие багов, производительность	Точность (accuracy), ошибка (loss) на новых данных (не тех, на которых учились).
Гибкость	Низкая. Для новых условий или данных часто нужно переписывать код	Высокая. Модель может адаптироваться к изменениям в данных (до определенных пределов)
Область применения	Детерминированные задачи с четкой логикой: веб-серверы, ОС, калькуляторы, бизнес-логика	Задачи с неявными правилами или большой изменчивостью: распознавание образов, прогнозирование, рекомендации, NLP, беспилотные автомобили
Пример решения	Программист изучает кошек, пишет правила	Собираются сотни тысяч фото с кошками и без. Модель (например, CNN) обучается на них. Она сама находит значимые признаки и предсказывает вероятность, что на новом фото – кошка.

Таблица 1: Сравнительная таблица

1.3 Базовые термины ML

Типичная функция в машинке отображает объекты или примеры (samples) в предсказания (targets). Мы назовём функцию, отображающую объекты в предсказания, моделью, а имеющийся у нас набор примеров – обучающей выборкой или датасетом. Обучающая выборка состоит из: объектов и ответов, которые мы также будем иногда называть таргетами

Измеримые качества объекта обычно называются признаками, а совокупность свойств, которые мы выделили у объекта – его признаковым описанием. Вот несколько простых и распространённых разновидностей признаков:

- Численные – например, рост или доход. Иногда отдельно выделяют вещественные и целочисленные признаки
- Категориальные признаки принимают значения из некоторого дискретного множества. Например, профессия человека или день недели
- Бинарные признаки принимают два значения: 0 и 1 или «да» и «нет». С ними можно работать и как с численными, и как с категориальными
- Среди категориальных признаков иногда выделяют ординальные. Они принимают значения из некоторого упорядоченного дискретного множества Класс опасности химического вещества (бывает от 1-го до 4-го) или год обучения студента являются ординальными

feature engineering – процесс извлечения данных из признаков

Матрица объекты-признаки – численная таблица, где строки которой соответствуют объектам, а столбцы – признакам

Предсказательные модели, то есть модели вида $y = f(x)$, которые пытаются уловить зависимость между признаковым описанием x объекта и таргетом y. Иногда мы будем иметь дело и с моделями данных: $''$ признак имеет нормальное распределение $''$. Чаще всего предсказательные модели мы будем брать из некоторого параметрического семейства y= $f_w(x)$, где w — параметры, которые мы будем подбирать по данным

Кроме выбора модели важен также выбор алгоритма обучения. **Алгоритм обучения** – это процедура, которая превращает обучающую выборку в обученную модель. Скажем, в примере выше для константной модели мы в качестве алгоритма обучения использовали поиск нуля градиента. Градиентные методы используются для обучения многих моделей, и это очень богатый класс методов оптимизации, из которого порой не так просто выбрать лучший

Метрика – это способ численно измерить то, насколько модель хорошо справилась с задачей. Метрика не единственная характеристика модели. Ещё выделяют размер, скорость работы, интерпретируемость, отсутствие дискриминации данных

2 Приложения ML (подобласти)

2.1 Назвать минимум 3 и привести практические примеры

Компьютерное зрение (Computer Vision)
Задача: Позволяет компьютерам «видеть» и интерпретировать визуальную информацию

- **Медицинская диагностика** Автоматический анализ рентгеновских снимков, МРТ и КТ для выявления опухолей, переломов или признаков пневмонии (например, алгоритмы на основе свёрточных нейронных сетей)
- **Системы безопасности и наблюдения** Распознавание лиц для разблокировки смартфонов или контроля доступа на объекты Также, отслеживание подозрительного поведения в реальном времени

Обработка естественного языка (Natural Language Processing)
Задача: Понимание, генерация и анализ человеческого языка.

- **Машинный перевод** Сервисы вроде Google Translate или Yandex Translate, которые используют модели на основе трансформеров (например, GPT) для перевода текста между языками
- **Виртуальные ассистенты и чат-боты** Siri, Алиса или ChatGPT, которые понимают устные или текстовые запросы пользователя и выдают осмысленные ответы

Обучение с подкреплением (Reinforcement Learning)
Задача: Обучение агента принимать решения через взаимодействие со средой и получение награды

- **Робототехника и автономные системы** Обучение робота-манипулятора хватать и перемещать предметы, а также обучение беспилотного автомобиля движению в сложной городской среде
- **Оптимизация в играх и симуляциях** Алгоритмы DeepMind (AlphaGo, AlphaZero), которые превзошли чемпионов мира в го, шахматах и StarCraft II, обучаясь через игру сами с собой

3 Наборы данных

3.1 Способы находить датасеты (минимум 3 источника)

Машинное обучение начинается с данных. Важно, чтобы их было достаточно много и чтобы они были достаточно качественными. Чем сложнее задача, тем больше данных нужно, чтобы её решить

Бороться с проблемой нехватки данных можно двумя способами. Первый – использование краудсорсинга, то есть привлечение людей, готовых разметить много данных. Во многих ситуациях (например, при оценке поисковой выдачи) без дополнительной разметки никак не обойтись. Мы расскажем про краудсорсинг подробнее в соответствующем параграфе. Некоторые проекты, в первую очередь научные и социальные, используют также citizen science – разметку данных волонтёрами без какого-либо вознаграждения, просто за чувство причастности к доброму делу исследования.

Второй же способ состоит в использовании неразмеченных данных. Например, в задаче аннотирования изображений у нас есть огромное количество никак не связанных друг с другом изображений и текстов. Однако мы можем использовать их для того, чтобы помочь компьютеру понять, какие слова в принципе могут стоять рядом

Публичные репозитории и платформы данных. Крупные специализированные платформы, где датасеты выложены для свободного или исследовательского использования

- **Kaggle Datasets** содержит огромную коллекция датасетов на самые разные темы (от финансов до компьютерного зрения), часто сопровождаемая обсуждениями и примерами анализа (ноутбуками). Внутри можно, например, найти набор данных `titanic` для прогнозирования выживания пассажиров.
- **UCI Machine Learning Repository** есть один из старейших и самых уважаемых источников классических датасетов, широко используемых в академических исследованиях и для обучения. Например, датасет `Iris` для задач классификации.

API и порталы государственных/международных организаций. Многие государственные и международные структуры публикуют данные в открытом доступе.

- **Порталы открытых данных**, тот же `data.gov.ru`, содержат огромные массивы данных от государственных ведомств: демография, экономика, экология, транспорт. Например, данные о государственных закупках или о качестве атмосферного воздуха в городе
- **Всемирный банк, ВОЗ, Eurostat** предоставляют статистику по странам мира в разных областях. Данные часто доступны для скачивания в структурированном виде (CSV, JSON). Например, статистика ВВП по всем странам за последние 60 лет

Специализированные поисковые системы и агрегаторы Инструменты, созданные специально для поиска по публичным датасетам.

- **Google Dataset Search** – поисковая система, которая индексирует датасеты с различных источников. Работает по аналогии с обычным поиском, но ищет именно метаданные датасетов
- **Awesome Public Datasets (на GitHub)** – это кураторские списки (репозитории), где датасеты сгруппированы по темам (биология, география, социальные науки и т.д.).

4 Разведочный анализ данных (EDA)

4.1 Зачем нужен EDA для моделирования, примеры

Цель EDA – понять структуру данных, выявить закономерности, аномалии и взаимосвязи **до** построения моделей. Это критический этап, который напрямую влияет на качество и интерпретируемость итоговой модели.

- **Понимание данных и генерация гипотез:** EDA помогает сформулировать осмысленные гипотезы о взаимосвязях между переменными. Без этого моделирование может стать «слепым» поиском.
- **Примеры обнаружения и обработки проблем с данными**
  - *Пропущенные значения:* при анализе медицинских данных EDA может показать, что в 40 % записей отсутствует показатель уровня сахара. Это требует решения: удалить строки, импутировать значения или добавить бинарный признак «пропуск».
  - *Выбросы (аномалии):* в данных о транзакциях EDA (например, через box-plot) может выявить платежи на сумму \$10 млн при медиане в \$50. Это может быть ошибка ввода, мошенничество или реальная транзакция, требующая отдельного анализа.
- **Проверка предположений модели:** многие алгоритмы (линейная регрессия, логистическая регрессия) строятся на статистических предположениях Например, построив гистограмму и Q-Q plot целевой переменной «цена дома», можно обнаружить её сильную правостороннюю асимметрию. Применение логарифмического преобразования (*log-transform*) сделает распределение ближе к нормальному и повысит эффективность линейной модели.
- **Выбор признаков и их инжиниринг:** EDA выявляет избыточные или слабые признаки. Наприме, матрица корреляций может показать почти линейную зависимость (*multicollinearity*) между признаками «площадь дома в квадратных метрах» и «площадь в квадратных футах». Один из них следует исключить.
- **Визуальное представление для заказчиков:** графики EDA (гистограммы, scatter plots) — лучший способ донести факты о данных до заказчика или команды, не погружённой в технические детали

4.2 Типичные шаги EDA

Процесс EDA является итеративным и не строго линейным.

1. **Первичный осмотр данных**
   - Загрузка данных (`pandas.read_csv()` и т.д.)
   - `.head()`, `.tail()` — просмотр первых/последних строк
   - `.info()` – информация о типах данных и наличии пропусков.
   - `.describe()` – сводная статистика (среднее, std, минимум и далее) для числовых признаков.
   - `.shape` — размерность данных (число строк и столбцов)
2. **Анализ и обработка пропущенных значений (Missing Values)**
   - Определение доли пропусков в каждом столбце
   - Анализ паттернов пропусков: являются ли они случайными или системными?
   - Принятие решения: удаление строк/столбцов, импутация (средним, медианой, моделью), добавление флага «был пропуск»
3. **Анализ распределений (Distribution Analysis)**
   - Для **категориальных** признаков построение (*bar plot*) для оценки частоты категорий
   - Для **числовых** признаков построение (*histogram*) и (*box plot*) для оценки формы распределения, центральной тенденции, разброса и выбросов.
   - Проверка на нормальность (визуальная или статистические тесты).
4. **Выявление и анализ выбросов (Outlier Analysis)**
   - Использование `box plot`, `scatter plot` или статистических методов (например, правило трёх сигм, интерквартильный размах — IQR).
   - Принятие решения: удаление, трансформация, каппинг или оставление выбросов (если они несут ценную информацию)
5. **Анализ взаимосвязей (Relationship Analysis)**

- **Числовые vs Числовые:** Матрица корреляций (Пирсон, Спирмен), диаграмма рассеяния для пар признаков
- **Категориальные vs Числовые:** *Box plot* или *violin plot* для сравнения распределений числовой переменной across different categories
- **Категориальные vs Категориальные:** Построение тепловой карты (*heatmap*) сводной таблицы или использование критерия хи-квадрат

6. **Инжиниринг и трансформация признаков (на основе EDA)**

- Кодирование категориальных переменных (One-Hot, Label Encoding).
- Масштабирование/нормализация числовых признаков (MinMaxScaler, StandardScaler).
- Создание новых признаков (например, «площадь на комнату» из «общей площади» и «числа комнат»).
- Применение преобразований для нормализации распределения (логарифмирование, корень квадратный).

7. **Формулировка выводов и гипотез для моделирования**

- Краткое резюме: основные закономерности, обнаруженные проблемы и принятые решения по обработке данных.
- Предварительные гипотезы о том, какие признаки могут быть наиболее важными для прогноза target.

Часть II
Вероятностные основы и постановка задач

5 Оценка максимального правдоподобия (MLE)

5.1 Определение правдоподобия

Правдоподобие $L(\theta|X,Y) = P(X,Y|\theta)$, где θ — вектор параметров, X — матрица признаков, а Y — вектор target. Таким образом, правдоподобие – это вероятность получить такую матрицу признаки-цели при данном значении вектора параметров распределения объектов. Метод максимального правдоподобия – метод, при котором правдоподобие стараются максимизировать

5.2 Следствия свойства i.i.d.

i.i.d = независимые и одинаково распределенные (independent and identically distributed). Объект — p-мерный вектор, порожденный из некоторого распределения (то есть случайной величиной), тут подразумевается независимость объектов как независимость таких случайных векторов. Одинаково распределенные: порождены одинаковым процессом (с одинаковым распределением). Это условие мы требуем от объектов

5.3 Формула отрицательного лог-правдоподобия

Мы хотим подобрать такие значения параметров w, для которых модель $p_y(y|x,w)$ была бы наиболее адекватна данным. Суть метода максимального правдоподобия (maximum likelihood estimation) состоит в том, чтобы найти такое w, для которого вероятность (а в данном, непрерывном, случае плотность вероятности) появления выборки $y = y_1, \ldots, y_N$ была бы максимальной, то есть

$$w_{MLE} = argmax_w p(y|X,w)$$

Величина $p(y|X,w)$ называется функцией правдоподобия (likelihood). Если мы считаем, что все объекты независимы, то функция правдоподобия распадается в произведение:

$$p(y|X,w) = p(y_1|x_1,w) \ldots p(y_N|x_i,w)$$

Теперь, поскольку перемножать сложно, а складывать легко (и ещё поскольку мы надеемся, что раз наши объекты всё-таки наблюдаются в природе, их правдоподобие отлично от нуля), мы переходим к логарифму функции правдоподобия:

$$l(y|X,w) = \log p(y1|x1,w) + \cdots + \log p(y_N|x_N,w)$$

Эту функцию мы так или иначе максимизируем по w, находя оценку максимального правдоподобия.

В теории оптимизации принято минимизировать, поэтому задачу переформулируют, используя отрицательное лог-правдоподобие или функцию потерь (то есть стремятся минимизировать потери информации о данных)

6 Таксономия задач ML

6.1 Обучение с учителем и без учителя, примеры

Описанные выше задачи являются примерами задач обучения с учителем (supervised learning), так как правильные ответы для каждого объекта обучающей выборки заранее известны. Задачи обучения с учителем делятся на виды в зависимости от того, каким может быть множество Y всех возможных ответов (таргетов)

Есть и относительно небольшой класс задач, относящихся к обучению без учителя (unsupervised learning), – это задачи, для которых нам известны только данные, а ответы неизвестны или вообще не существуют. Более того, часто поиск "правильных" ответов не является самоцелью. Классическим примером обучения без учителя является кластеризация – задача разделения объектов на группы, обладающие некоторыми неизвестными нам, но, как мы в глубине души надеемся, интерпретируемыми свойствами. Примером может служить кластеризация документов из электронной библиотеки по темам или кластеризация новостей с целью выделения крупных сюжетов. Бывают и другие виды (и даже парадигмы) машинного обучения

6.2 Назвать другие категории

Помимо основных, выделяют также

- **Обучение с подкреплением (Reinforcement Learning)**: агент обучается, взаимодействуя со средой и получая награду за правильные действия (робот, играющий в шахматы, беспилотный автомобиль)
- **Частичное обучение (Semi-supervised Learning)**: используется небольшой набор размеченных и большой набор неразмеченных данных (классификация веб-страниц, распознавание речи)
- **Самообучение (Self-supervised Learning)**: метки генерируются автоматически из самих данных (предобучение больших языковых моделей, обучение представлений в компьютерном зрении)
- **Активное обучение (Active Learning)**: алгоритм может запрашивать разметку для наиболее информативных примеров у эксперта (разметка медицинских изображений, анализ юридических документов)
- **Трансферное обучение (Transfer Learning)**: использование знаний, полученных при решении одной задачи, для быстрого обучения на другой, но родственной задаче (дообучение ImageNet модели для конкретного класса изображений)

6.3 Практические примеры (минимум 3 для каждого типа)

Для обучения с учителем ниже.
Обучение без учителя

1. **Сегментация клиентской базы:** разделить всех клиентов интернет-магазина на группы по истории покупок и демографии для персонализации рекламы (кластеризация, например, методом k-means)
2. **Обнаружение аномалий в работе сервера:** выявить нестандартные паттерны в логах сетевой активности, которые могут указывать на кибератаку (часто используется снижение размерности и анализ плотности)

3. **Рекомендация статей пользователю:** на основе матрицы "пользователь-просмотренная статья" с помощью матричной факторизации (SVD) найти скрытые темы и рекомендовать не прочитанные, но релевантные статьи.

Обучение с подкреплением

1. **Управление роботом-манипулятором:** агент (контроллер робота) учится совершать точные движения, чтобы взять предмет, получая положительную награду за успех и отрицательную за лишние действия или промах
2. **Оптимизация энергопотребления в ЦОД:** система управления учится динамически регулировать охлаждение и нагрузку на серверы, чтобы минимизировать затраты на электроэнергию.
3. **Торговый бот на бирже:** агент учится совершать операции покупки/продажи акций, максимизируя конечный капитал (награда) на исторических или симулированных данных.

6.4 Постановка практической задачи, основные сущности

1. $\mathbb{Y} = \mathbb{R}$ или $\mathbb{Y} = \mathbb{R}^M$ — регрессия. Примерами задач регрессии является предсказание продолжительности поездки на каршеринге, спрос на конкретный товар в конкретный день или погода в вашем городе на завтра (температура, влажность и давление – это несколько вещественных чисел, которые формируют вектор нашего предсказания)
2. $\mathbb{Y} = 0{,}1$ – бинарная классификация. Например, мы можем предсказывать, кликнет ли пользователь по рекламному объявлению, вернёт ли клиент кредит в установленный срок, сдаст ли студент сессию, случится ли определённое заболевание у пациента, есть ли на картинке банан.
3. $\mathbb{Y} = 1, \ldots, K$ – многоклассовая (multiclass) классификация. Например, определение предметной области для научной статьи (математика, биология, психология и так далее)
4. $\mathbb{Y} = 0{,}1^K$ – многоклассовая классификация с пересекающимися классами (multilabel classification). Например, задача автоматического проставления тегов для ресторанов (логично, что ресторан может одновременно иметь несколько тегов)
5. $\mathbb{Y}$ — конечное упорядоченное множество $\rightarrow$ задача ранжирования. Основным примером является задача ранжирования поисковой выдачи, где для любого запроса нужно отсортировать все возможные документы по релевантности этому запросу. При этом оценка релевантности имеет смысл только в контексте сравнения двух документов между собой, её абсолютное значение информации не несёт

7 Наивный байесовский классификатор

7.1 Формула Байеса

Основная идея: мы хотим использовать теорему Байеса. Для этого нам надо чтобы признаки были независимы (а это не всегда так, например, температура в разных единицах).

Формула условной вероятности

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

7.2 Основные предположения

Ставится задача K-классовой классификации (то есть каждый элемент по какому-то признаку может быть одного из k возможных классов)

$$x_i \in \mathbb{R}^p, y_i \in C_1, \ldots, C_k$$

Давайте прикинем условную вероятность метки класса при условии того, что объект описывается какими-то фичами

$$P(y_i = C_k|x_i) = \frac{P(x_i|y_i = C_k)P(y_i = C_k)}{P(x_i)}$$

То есть мы оцениваем, какова вероятность того, что метка класса это C_k при условии, что мы наблюдаем x_i. Уже имея данную формулу мы можем оценивать вероятность лейблов. $P(y_i = C_k)$ оценим как частоту (встречаемости) каждого класса в нашей выборке (просто руками посчитаем, насколько вероятно попадание в тот или иной класс). Но нам надо оценить $P(x_i|y_i = C_k)$. Это сделать непонятно, как,

потому что x_i – это вектор. Здесь и понадобится наше наивное предположение – все признаки (p штук) независимы и тогда хорошо факторизуются.

$$P(x_i|y_i = C_k) = \prod_{n=1}^{p} P(x_i^n|y_i = C_k)$$

В реальной жизни признаки часто зависимы, причем довольно сложным образом, так что просто так взять не удастся. Но есть хорошая новость – признаки бывают частично независимы (например, среди них бывают независимые подмножества), так что какой-то смысл в этом есть. Теперь вектор x_i представляется как вектор p независимых случайных величин. Мы хотим найти метку класса, которая наиболее вероятна

$$C^* =_k P(y_i = C_k|x_i) =_k \frac{P(x_i|y_i = C_k)P(y_i = C_k)}{P(x_i)}$$

Это и будет нашим предсказанием.

Теперь заметим, что знаменатель не зависит от k, поэтому на нахождение максимума он не влияет. То есть если его убрать, у нас, конечно, выражение перестанет быть вероятностью, но метка класса, при котором достигается этот самый максимум, не изменится. Поэтому на него можно забить, и даже сэкономить на этом какое-то время (хотя и не очень большое).

Правда, на самом деле, мы в любом случае его и так практически посчитаем по пути, так как $P(x_i) = \sum_k P(x_i|y_i = C_k)$

Для того, чтобы посчитать $P(x_i^l|y_i = C_k)$, требуется ввести какое-то распределение на наши признаки. Строго понять это вряд ли получится, так что или нам это было известно, или мы должны просто предположить. Тут есть два пути. Первый: если они тоже все категориальные, можем просто посчитать частоту каждого из них. Второй: если они какие-то числовые, то тут чуть сложнее, нам придется ввести какое-то априорное распределение на x. Самый простой случай, когда x пришел из нормального распределения. В этом случае мы считаем дисперсию и среднее по выборке, предполагая что распределение нормальное, и тогда можем оценить вероятность. Такой классификатор будет хорошо работать на классификации текстов (например, определять, грубый ли текст).

7.3 Случай непрерывных признаков

Для работы с ними нам нужно ввести параметрическую вероятность объектов, которые мы анализируем. Затем мы подбираем параметры для обучающего набора и используем их для прогнозирования плотности для одной точки. Распределение может быть гауссовыми, полиномиальным, бернуллиевским. Также суммы превращаются в интегралы

8 Библиотека sklearn

8.1 Базовые классы, их функции и использование

Scikit-learn (sklearn) построен на основе единого, консистентного программного интерфейса приложений, основными абстракциями которого являются **оценщики** (estimators), **преобразователи** (transformers) и **предсказатели** (predictors).

Базовый класс `sklearn.base.BaseEstimator`. Является родительским классом для всех оценщиков. Определяет общие методы:

- `__init__()`: Конструктор для установки гиперпараметров
- `get_params()`, `set_params()`: Методы для получения и установки параметров модели (важно для настройки гиперпараметров и кросс-валидации)

Классы `sklearn.base.ClassifierMixin / RegressorMixin / TransformerMixin` Миксины (примеси), которые добавляют специфичное поведение:

- `ClassifierMixin`: добавляет метод `score(X, y)`, возвращающий **accuracy** (долю правильных ответов) для классификаторов.
- `RegressorMixin`: добавляет метод `score(X, y)`, возвращающий **коэффициент детерминации** R^2 для регрессоров.
- `TransformerMixin`: добавляет метод `fit_transform(X[, y])`, который выполняет `fit()` и `transform()` за один шаг. Ускоряет написание кода.

Ключевые интерфейсы и методы

Любой "обученный" объект в sklearn (модель, преобразователь) следует одному из двух основных шаблонов.

Оценщик (Модель) для обучения с учителем (`Classifier`, `Regressor`)

- `.fit(X_train, y_train)`. Основной метод для обучения модели на тренировочных данных. Изменяет внутреннее состояние объекта (становится "обученным").
- `.predict(X_new)` использует обученную модель для предсказания целевой переменной на новых данных. Возвращает массив `y_pred`.
- `.predict_proba(X_new)`: работает только для классификаторов и возвращает вероятности принадлежности к каждому классу.
- `.score(X, y)`: Возвращает метрику качества предсказаний на предоставленных данных

Пример использования:

```
# from sklearn.ensemble import RandomForestClassifier
# from sklearn.datasets import load_iris
# from sklearn.model_selection import train_test_split
#
# #
# X, y = load_iris(return_X_y=True)
# X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
#
# #
# clf = RandomForestClassifier(n_estimators=100, random_state=42)
# clf.fit(X_train, y_train) #                    (fit)
#
# #
# y_pred = clf.predict(X_test) #                          (predict)
# accuracy = clf.score(X_test, y_test) #              (score)
# print(f"Accuracy: {accuracy:.2f}")
```

Важное правило: параметры преобразования (например, `scaler.mean_`) всегда вычисляются **только на тренировочной выборке** (`fit`), а затем применяются и к тренировочной, и к тестовой (`transform`), чтобы избежать неучёта данных.

8.2 Важность пайплайнов, пример

Проблемы, которые решает Pipeline

Без использования `Pipeline` процесс предобработки и обучения часто выглядит громоздким и ошибочным

1. **Утечка данных (Data Leakage):** при ручном выполнении `fit_transform` на всем датасете до разделения или при вычислении статистик (например, среднего для заполнения пропусков) на тесте.
2. **Сложность воспроизведения:** много промежуточных шагов и переменных.
3. **Неудобство кросс-валидации и поиска гиперпараметров:** невозможно корректно настроить гиперпараметры для этапа предобработки (например, степень полинома в `PolynomialFeatures`) совместно с параметрами модели в рамках GridSearchCV.
4. **Сложность развёртывания:** пайплайн инкапсулирует всю цепочку преобразований и модель в один объект, который легко сохранить (`pickle` или `joblib`) и использовать в production. Без этого предстоит всё делать самому

Пример создания и использования Pipeline

Листинг 1: Полный пример пайплайна с настройкой

```
# from sklearn.pipeline import Pipeline
# from sklearn.compose import ColumnTransformer
# from sklearn.preprocessing import StandardScaler, OneHotEncoder
# from sklearn.impute import SimpleImputer
# from sklearn.ensemble import RandomForestClassifier
```

```
# from sklearn.model_selection import train_test_split, GridSearchCV
# import pandas as pd
# import numpy as np
#
# # 1.
# np.random.seed(42)
# n_samples = 100
# data = pd.DataFrame({
#     'age': np.random.randint(18, 70, n_samples),
#     'salary': np.random.normal(50000, 15000, n_samples),
#     'city': np.random.choice(['Moscow', 'SPb', 'Kazan'], n_samples),
#     'experience': np.random.normal(10, 4, n_samples),
#     'target': np.random.choice([0, 1], n_samples)
# })
# #
# data.loc[10:15, 'salary'] = np.nan
# data.loc[20, 'age'] = 150
#
# X = data.drop('target', axis=1)
# y = data['target']
# X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_stat
#
# # 2.
# numeric_features = ['age', 'salary', 'experience']
# numeric_transformer = Pipeline(steps=[
#     ('imputer', SimpleImputer(strategy='median')), #
#     ('scaler', StandardScaler()) #
# ])
#
# categorical_features = ['city']
# categorical_transformer = Pipeline(steps=[
#     ('imputer', SimpleImputer(strategy='constant', fill_value='missing')), #
#     ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False)) # One-Ho
# ])
#
# # 3.                                                    ColumnTransformer
# preprocessor = ColumnTransformer(
#     transformers=[
#         ('num', numeric_transformer, numeric_features),
#         ('cat', categorical_transformer, categorical_features)
#     ])
#
# # 4.                                                    :                          ->
# full_pipeline = Pipeline(steps=[
#     ('preprocessor', preprocessor),
#     ('classifier', RandomForestClassifier(random_state=42))
# ])
#
# # 5.
# #                                                                 :
# param_grid = {
#     'preprocessor__num__imputer__strategy': ['mean', 'median'],
#     'classifier__n_estimators': [50, 100, 200],
#     'classifier__max_depth': [5, 10, None]
# }
#
# # 6.                                    GridSearchCV
# grid_search = GridSearchCV(full_pipeline, param_grid, cv=5, scoring='accuracy', n_jo
# grid_search.fit(X_train, y_train) #
#
# print(f"                                : {grid_search.best_score_:.3f}")
```

```
# print(f"                                    : {grid_search.best_params_}")
#
## 7.
# best_model = grid_search.best_estimator_
# test_score = best_model.score(X_test, y_test) #
# print(f"                                      : {test_score:.3f}")
#
## 8.
# new_data = pd.DataFrame({
#     'age': [25, 45],
#     'salary': [60000, np.nan],
#     'city': ['Moscow', 'Kazan'],
#     'experience': [2, 15]
# })
# predictions = best_model.predict(new_data)
# print(f"                              : {predictions}")
```

Преимущества данного подхода

- **Защита от утечки:** `SimpleImputer` и `StandardScaler` вычисляют свои параметры **только** на тренировочном фолде во время кросс-валидации.
- **Чистота кода:** весь процесс упакован в один объект `best_model`.
- **Удобная настройка:** можно подбирать гиперпараметры как для предобработки, так и для модели.
- **Простое развертывание:** объект `best_model` можно сразу сохранить (`joblib.dump`) и загрузить для предсказаний на новых сырых данных без необходимости помнить о порядке преобразований.

Часть III
Линейная регрессия, оптимизация и регуляризация

9 Регрессионная модель

9.1 Предположения

Пусть есть данные геологоразведки, по которым вы хотели бы оценить перспективы разных месторождений. В данном случае по набору геологических данных ваша модель будет, к примеру, оценивать потенциальную годовую доходность шахты. Это пример задачи регрессии. Числа, которым мы хотим сопоставить объекты из нашего множества иногда называют таргетами (от английского target). Таким образом, задачи классификации и регрессии можно сформулировать как поиск отображения из множества объектов $\mathbb{X}$ в множество возможных таргетов (может быть многомерным). Математически задачу регрессии можно описать так: $\mathbb{X} \to \mathbb{R}$.

Очевидно, что просто сопоставить какие-то объекты каким-то числам – дело довольно бессмысленное. Мы же хотим верно принимать решение, где строить шахту. Есть простой вопрос: среди каких отображений мы будем искать самое лучшее? Возможных отображений может быть много, но мы можем упростить себе задачу и договориться, что хотим искать решение только в каком-то заранее заданном параметризированном семействе функций

Итак, регрессия – задача обучения с учителем, в которой есть выборка объектов, с известным значением вещественной целевой функции и выборка объектов, для которых это целевое значение нужно предсказать. Предполагаем, что целевая функция – функция признаков объекта и некоторого небольшого шума (желательно белого). В нашем курсе рассматривалась в основном линейная регрессия – регрессия, в которой предполагается, что эта зависимость линейная

Целевая функция, может быть также многомерной. Например, если мы можем предсказать 10 чисел, то у нас таргет может лежать именно в одномерном множестве действительных чисел? Что такое число – это точка на числовой прямой. Что такое точка в 10-мерном пространстве - это радиус-вектор, описывающийся 10 числами. Поэтому мы можем предсказывать хоть 10 чисел одновременно и тогда предсказываемый таргет будет из $\mathbb{R}^{10}$. Например, цвет; чтобы его точно предсказать, надо будет предсказать сразу три

канала R, G, B, а не один, так как если по отдельности предсказывать каждый канал, то у вас нужный цвет не получится никак

Для того, чтобы понять что такое линейная регрессия, прежде всего надо понять что такое регрессия. В задаче регрессии $\mathbb{E}(Y|X) = f(X)$, где вектор Y и матрица X – случайные величины. Считаем матожидание Y для каких-то известных X детерминированной функцией от X, то есть от известных фичей. Эквивалентно можно переписать в менее пугающем виде $Y = f(X) + \varepsilon$. В левой части равенства вектор таргетов Y, который определен детерминистически компонентой $f(X)$ и случайным шумом ε, у которого нулевое матожидание, потому что мы всё матожидание вынесли в детерминированную часть, а в ε есть только какие-то ошибки. $f(X)$ называют функцией регрессии. То есть мы поделили целевую функцию на две части: непосредственно регрессию с нулевой дисперсией (которое мы хотим предсказать) и шум с нулевым средним

Допущения регрессионной модели

1. Выборка является репрезентативной для генеральной совокупности в целом
2. Независимые переменные измеряются без ошибок
3. Ожидаемое значение отклонений от модели (шума) равно нулю при условии соблюдения ковариат: $E(e_i|X_i) = 0$
4. Дисперсия невязок постоянна во всех наблюдениях (гомоскедастичность)
5. Невязки не коррелируют друг с другом. Математически: дисперсионно–ковариационная матрица ошибок является диагональной

10 Линейная регрессия

10.1 Мотивация функции потерь MSE

Мы начнём с использования линейных моделей для решения задачи регрессии. Простейшим примером постановки задачи линейной регрессии является метод наименьших квадратов.

Пусть у нас задан датасет $(X,\, y)$, где $y = (y_i)_{i=1}^N \in \mathbb{R}^N$ – вектор значений целевой переменной, а $X = (x_i)_{i=1}^N \in \mathbb{R}^{N\times D}$, $x_i \in \mathbb{R}^D$ – матрица объекты-признаки, в которой i-я строка – это вектор признаков i-го объекта выборки. Мы хотим моделировать зависимость y_i от x_i как линейную функцию со свободным членом. Общий вид такой функции из $\mathbb{R}^D$ в $\mathbb{R}$ выглядит следующим образом:

$$f_w(x_i) = (w,\, x_i) + w_0$$

Свободный член w_0 часто опускают, потому что такого же результата можно добиться, добавив ко всем x_i признак, тождественно равный единице; тогда роль свободного члена будет играть соответствующий ему вес:

$$(x_{i_1} \dots x_{i_D})\dot{(}w_1 \dots w_D)^T + w_0 = (1\ x_{i_1} \dots x_{i_D})\dot{(}w_0\ w_1 \dots w_D)^T$$

Поскольку это сильно упрощает запись, в дальнейшем мы будем считать, что это уже сделано и зависимость имеет вид просто $f_w(x_i) = (w,\, x_i)$. Мы хотим, чтобы на нашем датасете (то есть на парах $(x_i,\, y_i)$ из обучающей выборки) функция f_w как можно лучше приближала нашу зависимость

Для того чтобы чётко сформулировать задачу, нам осталось только одно: на математическом языке выразить желание «приблизить $f_w(x)$ к y». Говоря простым языком, мы должны научиться измерять качество модели и минимизировать её ошибку, меняя обучаемые параметры. В нашем примере обучаемые параметры – это веса w. Функция, оценивающая то, как часто модель ошибается, традиционно называется функцией потерь, функционалом качества или просто loss (loss function). Важно, чтобы её было легко оптимизировать: гладкая функция потерь – это хорошо, а кусочно постоянная – ужасно.

Функции потерь бывают разными. От их выбора зависит то, насколько задачу в дальнейшем легко решать, и то, в каком смысле у нас получится приблизить предсказание модели к целевым значениям. Интуитивно понятно, что для нашей текущей задачи нам нужно взять вектор y и вектор предсказаний модели и как-то сравнить, насколько они похожи. Так как эти векторы «живут» в одном векторном пространстве, расстояние между ними вполне может быть функцией потерь. Более того, положительная непрерывная функция от этого расстояния тоже подойдёт в качестве функции потерь. При этом способов задать расстояние между векторами тоже довольно много. От всего этого разнообразия глаза разбегаются, но мы обязательно поговорим про это позже. Сейчас давайте в качестве loss возьмём квадрат L_2-нормы вектора разницы предсказаний модели и y. Во-первых, как мы увидим дальше, так задачу будет нетрудно решить, а во-вторых, у этого loss есть ещё несколько дополнительных свойств

- L_2-норма разницы – это евклидово расстояние $|y - f_w(x)|^2$ между вектором таргетов и вектором ответов модели, то есть мы их приближаем в смысле самого простого и понятного «расстояния».
- Как мы увидим в разделе про вероятностные модели, с точки зрения статистики это соответствует гипотезе о том, что наши данные состоят из линейного «сигнала» и нормально распределенного «шума».

Так вот, наша функция потерь выглядит так:

$$L(f,X,y) = |y - f_w(x)|_2^2 = |y - Xw|_2^2 = \sum_{i=1}^{N}(y_i - (x_i,w))^2$$

Такой функционал ошибки не очень хорош для сравнения поведения моделей на выборках разного размера. Представьте, что вы хотите понять, насколько качество модели на тестовой выборке из 2500 объектов хуже, чем на обучающей из 5000 объектов. Вы измерили L_2-норму ошибки и получили в одном случае 300, а в другом 500. Эти числа не очень интерпретируемы. Гораздо лучше посмотреть на среднеквадратичное отклонение

$$L(f,X,y) = \frac{1}{N}\sum_{i=1}^{N}(y_i - (x_i,w))^2$$

По этой метрике на тестовой выборке получаем 0,12, а на обучающей 0,10.

Функция потерь $\frac{1}{N}\sum_{i=1}^{N}(y_i-(x_i,w))^2$ называется Mean Squared Error, MSE или среднеквадратическим отклонением. Разница с L_2-нормой чисто косметическая, на алгоритм решения задачи она не влияет:

$$MSE(f,X,y) = \frac{1}{N}|y - Xw|_2^2$$

В самом широком смысле, функции работают с объектами множеств: берут какой-то входящий объект из одного множества и выдают на выходе соответствующий ему объект из другого. Если мы имеем дело с отображением, которое на вход принимает функции, а на выходе выдаёт число, то такое отображение называют функционалом. Если вы посмотрите на нашу функцию потерь, то увидите, что это именно функционал: для каждой конкретной линейной функции, которую задают веса w_i, мы получаем число, оценивающее, насколько точно эта функция приближает наши значения y. Чем меньше это число, тем точнее наше решение, значит для того, чтобы найти лучшую модель, этот функционал нам надо минимизировать по w:

$$|y - Xw|_2^2 \to \min_w$$

10.1.1 Доказательство для гауссовского случая

Гауссово предположение Предположим, что ошибки ε_i:

1. Независимы и одинаково распределены (i.i.d.).
2. Следуют нормальному распределению: $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$.
3. Имеют нулевое математическое ожидание: $\mathbb{E}[\varepsilon_i] = 0$.
4. Имеют постоянную дисперсию: $\mathrm{Var}(\varepsilon_i) = \sigma^2 < \infty$.

Тогда условное распределение y_i при заданном $\boldsymbol{x}_i$ также нормальное:

$$y_i \mid \boldsymbol{x}_i \sim \mathcal{N}(\boldsymbol{x}_i^\top \boldsymbol{\beta}, \sigma^2)$$

Функция правдоподобия. Функция правдоподобия для всех наблюдений при условии независимости:

$$\begin{aligned} L(\boldsymbol{\beta}, \sigma^2) &= p(\boldsymbol{y} \mid \boldsymbol{X}; \boldsymbol{\beta}, \sigma^2) \\ &= \prod_{i=1}^{n} p(y_i \mid \boldsymbol{x}_i; \boldsymbol{\beta}, \sigma^2) \\ &= \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \boldsymbol{x}_i^\top \boldsymbol{\beta})^2}{2\sigma^2}\right) \\ &= (2\pi\sigma^2)^{-n/2} \exp\left(-\frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \boldsymbol{x}_i^\top \boldsymbol{\beta})^2\right) \end{aligned}$$

Логарифмическое правдоподобие. Удобнее работать с логарифмической функцией правдоподобия:

$$\ell(\boldsymbol{\beta}, \sigma^2) = \log L(\boldsymbol{\beta}, \sigma^2) = -\frac{n}{2}\log\bigl(2\pi\sigma^2\bigr) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \boldsymbol{x}_i^\top \boldsymbol{\beta})^2$$

Максимизируем по $\boldsymbol{\beta}$ в силу того, что $\boldsymbol{\beta}$ входит только во второе слагаемое. Поскольку $\sigma^2 > 0$, максимизация $\ell(\boldsymbol{\beta}, \sigma^2)$ по $\boldsymbol{\beta}$ эквивалентна **минимизации суммы квадратов ошибок**:

$$\boldsymbol{\beta}_{\text{MLE}} = \arg\max_{\boldsymbol{\beta}} \ell(\boldsymbol{\beta}, \sigma^2) = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n}(y_i - \boldsymbol{x}_i^\top \boldsymbol{\beta})^2$$

Итог

- При гауссовом предположении о нормальности ошибок **метод максимального правдоподобия** приводит к необходимости минимизировать сумму квадратов ошибок.
- Таким образом, минимизация MSE является **естественным** и **оптимальным** критерием в рамках этой вероятностной модели.

10.2 Аналитическое решение для весов (матричная форма)

В случае минимизации среднеквадратичной ошибки существует аналитическое решение:

10.2.1 Доказательство

$$|y - Xw|_2^2 = (y - Xw)(y - Xw) = Y^TY - (Xw)^TY - Y^T(Xw) + (Xw)^T(Xw) = Y^TY - w^TX^TY - Y^TXw + w^TX^TXw$$
$$\nabla_w |y - Xw|_2^2 = 0 - X^TY - X^TY - (X^TX + X^TX)w$$

Приравнивая $\nabla_w |y - Xw|_2^2$ к нулю, получаем $w = (X^TX)^{-1}X^TY$

Аналитическое решение существует только для 2-нормы. Для остальных норм решение ищется только градиентными методами

10.3 Теорема Гаусса–Маркова, BLUE

Теорема Гаусса-Маркова. Пусть целевые значения выражаются в следующем виде: $Y = Xw + \varepsilon$, где $\varepsilon = (\varepsilon_1, \ldots, \varepsilon_N)$ – случайный вектор, при этом:

1. $\mathbb{E}\varepsilon_i = 0 \forall i$
2. $\mathbb{D}\varepsilon_i = \sigma^2 < \inf \forall i$
3. $cov(\varepsilon_i, \varepsilon_j) = 0 \forall i \neq j$

Тогда решение задачи наименьших квадратов является оптимальным среди несмещённых. Другими словами, является несмещённой.

[BLUE — Best Linear Unbiased Estimator] Оценка $\tilde{\boldsymbol{\beta}}$ называется **наилучшей линейной несмещенной оценкой** (BLUE) вектора параметров $\boldsymbol{\beta}$, если:

1. **Линейна:** $\tilde{\boldsymbol{\beta}} = \boldsymbol{C}\boldsymbol{y}$ для некоторой матрицы $\boldsymbol{C} \in \mathbb{R}^{p \times n}$ (не зависящей от $\boldsymbol{y}$)
2. **Несмещенна:** $\mathbb{E}[\tilde{\boldsymbol{\beta}}] = \boldsymbol{\beta}$ для всех $\boldsymbol{\beta} \in \mathbb{R}^p$
3. **Наилучша:** среди всех линейных несмещенных оценок имеет наименьшую дисперсию (в смысле положительно определенных матриц ковариаций)

Формально, если $\tilde{\boldsymbol{\beta}}_1$ и $\tilde{\boldsymbol{\beta}}_2$ — две линейные несмещенные оценки, то $\tilde{\boldsymbol{\beta}}_1$ лучше, если матрица $\mathrm{Cov}(\tilde{\boldsymbol{\beta}}_2) - \mathrm{Cov}(\tilde{\boldsymbol{\beta}}_1)$ является положительно полуопределенной.

11 Градиентный спуск для линейной регрессии

11.1 Почему не используют матричное решение

Заметим, что для получения ответа нам нужно обратить матрицу X^TX. Это создаёт множество проблем:

1. Основная проблема в обращении матрицы – это то, что вычислительно обращать большие матрицы дело сложное, а мы бы хотели работать с датасетами, в которых у нас могут быть миллионы точек,
2. Матрица X^TX, хотя почти всегда обратима в разумных задачах машинного обучения, часто плохо обусловлена. Особенно если признаков много, между ними может появляться приближённая линейная зависимость, которую мы можем упустить на этапе формулировки задачи. Тогда погрешность нахождения w будет зависеть от квадрата числа обусловленности матрицы X, что очень плохо. Это делает полученное таким образом решение численно неустойчивым: малые возмущения y могут приводить к катастрофическим изменениям w.

Полностью вылечить проблемы мы не сможем, но никто и не обязывает нас останавливаться на «точном» решении (которое всё равно никогда не будет вполне точным). Поэтому используют другой метод

Минимизируемый функционал является гладким и выпуклым, а это значит, что можно эффективно искать точку его минимума с помощью итеративных градиентных методов. Далее об одном самом базовом подходе.

Как известно, градиент функции в точке направлен в сторону её наискорейшего роста, а антиградиент (противоположный градиенту вектор) в сторону наискорейшего убывания. То есть имея какое-то приближение оптимального значения параметра w, мы можем его улучшить, посчитав градиент функции потерь в точке и немного сдвинув вектор весов в направлении антиградиента:

$$w_j \to w_j - \alpha \frac{d}{dw_j} L(f_w, X, y)$$

, где α – это параметр алгоритма («темп обучения»), который контролирует величину шага в направлении антиградиента. Описанный алгоритм называется градиентным спуском.

Посмотрим, как будет выглядеть градиентный спуск для функции потерь $L(f_w, X, y) = \frac{1}{N}|Xw - y|_2^2$. Градиент квадрата евклидовой нормы мы уже считали; соответственно,

$$\nabla_w L = \frac{2}{N} X^T (Xw - y)$$

Следовательно, стартовав из какого-то начального приближения, мы можем итеративно уменьшать значение функции, пока не сойдёмся (по крайней мере в теории) к минимуму (вообще говоря, локальному, но в данном случае глобальному)

Несколько общих замечаний:

- Поскольку задача выпуклая, выбор начальной точки влияет на скорость сходимости, но не настолько сильно, чтобы на практике нельзя было стартовать всегда из нуля или из любой другой приятной вам точки;
- Число обусловленности матрицы X существенно влияет на скорость сходимости градиентного спуска: чем более вытянуты эллипсоиды уровня функции потерь, тем хуже;
- Темп обучения α тоже сильно влияет на поведение градиентного спуска; вообще говоря, он является гиперпараметром алгоритма, и его, возможно, придётся подбирать отдельно. Другими гиперпараметрами являются максимальное число итераций S и (или) порог tolerance.

Вычислительная сложность градиентного спуска – $O(NDS)$, где, как и выше, N – длина выборки, D – число признаков у одного объекта. Сравните с оценкой $O(D^2N + D^3)$ для «наивного» вычисления аналитического решения.

Сложность по памяти – $O(ND)$ на хранение выборки. В памяти мы держим и выборку, и градиент, но в большинстве реалистичных сценариев доминирует выборка.

11.2 Стохастический градиентный спуск, преимущества

На каждом шаге градиентного спуска нам требуется выполнить потенциально дорогую операцию вычисления градиента по всей выборке (сложность $O(ND)$). Возникает идея заменить градиент его оценкой на подвыборке (в английской литературе такую подвыборку обычно именуют batch или mini-batch; в русской разговорной терминологии тоже часто встречается слово батч или мини-батч).

А именно, если функция потерь имеет вид суммы по отдельным парам объект-таргет

$$L(w,X,y) = \frac{1}{N}\sum_{i=1}^{N} L(w,x_i,y_i)$$

а градиент, соответственно, записывается в виде

$$\nabla_w L(w,X,y) = \frac{1}{N}\sum_{i=1}^{N} \nabla_w L(w,x_i,y_i)$$

то предлагается брать оценку

$$\nabla_w L(w,X,y) = \frac{1}{B}\sum_{t=1}^{B} \nabla_w L(w,x_{i_t},y_{i_t})$$

для некоторого подмножества этих пар $(x_{i_t},y_{i_t})_{t=1}^{B}$.

Как делить выборку на батчи? Ясно, что можно было бы случайным образом сэмплировать их из полного датасета, но даже если использовать быстрый алгоритм вроде резервуарного сэмплирования, сложность этой операции не самая оптимальная. Поэтому используют линейный проход по выборке (которую перед этим лучше всё-таки случайным образом перемешать). Давайте введём ещё один параметр нашего алгоритма: размер батча, который мы обозначим B. Теперь на B очередных примерах вычислим градиент и обновим веса модели. При этом, вместо количества шагов алгоритма обычно задают количество эпох E. Это ещё один гиперпараметр. Одна эпоха – это один полный проход нашего сэмплера по выборке. Заметим, что если выборка очень большая, а модель компактная, то даже первый проход бывает можно не заканчивать

Сложность по времени – $O(NDE)$. На первый взгляд, она такая же, как и у обычного градиентного спуска, но заметим, что мы сделали в $\frac{N}{B}$ раз больше шагов, то есть веса модели претерпели намного больше обновлений.

Сложность по памяти можно довести до $O(BD)$: ведь теперь всю выборку не надо держать в памяти, а достаточно загружать лишь текущий батч (а остальная выборка может лежать на диске, что удобно, так как в реальности задачи, в которых выборка целиком не влезает в оперативную память, встречаются сплошь и рядом). Заметим, впрочем, что при этом лучше бы B взять побольше: ведь чтение с диска – намного более затратная по времени операция, чем чтение из оперативной памяти.

Шаги стохастического градиентного спуска заметно более шумные, но считать их получается значительно быстрее. Они тоже сходятся к оптимальному значению потому, что матожидание оценки градиента на батче равно самому градиенту. По крайней мере, сходимость можно получить при хорошо подобранных коэффициентах темпа обучения в случае выпуклого функционала качества.

Существует определённая терминологическая путаница, иногда стохастическим градиентным спуском называют версию алгоритма, в которой размер батча равен единице (то есть максимально шумная и быстрая версия алгоритма), а версии с гораздо большим размером батча называют batch gradient descent. В книгах, которые, возможно, старше вас, такая процедура иногда ещё называется incremental gradient descent. Это не очень принципиально, но вы будьте готовы, если что

12 Регуляризация в линейных моделях

12.1 Мотивация регуляризации

Всегда ли решение задачи регрессии единственно? Вообще говоря, нет. Так, если в выборке два признака будут линейно зависимы (и следовательно, ранг матрицы будет меньше D), то гарантировано найдётся такой вектор весов v что $(v,x_i) = 0\ \forall x_i$. В этом случае, если какой-то w является решением оптимизационной задачи, то и $w + \alpha\nu$ тоже является решением для любого α. То есть решение не только не обязано быть уникальным, так ещё может быть сколь угодно большим по модулю. Это создаёт вычислительные трудности. Малые погрешности признаков сильно возрастают при предсказании ответа, а в градиентном спуске накапливается погрешность из-за операций со слишком большими числами.

Конечно, в жизни редко бывает так, что признаки строго линейно зависимы, а вот быть приближённо линейно зависимыми они вполне могут быть. Такая ситуация называется мультиколлинеарностью. В этом случае у нас, всё равно, возникают проблемы, близкие к описанным выше. Дело в том, что $X\nu \sim 0$ для вектора v, состоящего из коэффициентов приближённой линейной зависимости, и, поэтому $X^TX\nu$, то есть матрица X^TX снова будет близка к вырожденной. Как и любая симметричная матрица, она диагонализируется в некотором ортонормированном базисе, и некоторые из собственных значений λ_i близки к нулю. Если вектор X^Ty в выражении $(X^TX)^{-1}X^Ty$ будет близким к соответствующему собственному вектору, то он будет умножаться на $\frac{1}{\lambda_i}$, что опять же приведёт к появлению у w очень больших по модулю компонент (при этом w ещё и будет вычислен с большой погрешностью из-за деления на маленькое число). И, конечно же, все ошибки и весь шум из матрицы X при вычислении $y \sim Xw$ будут умножаться на эти большие и неточные числа и возрастать во много-много раз, что приведёт к проблемам, от которых нас не спасёт никакое сингулярное разложение.

Отметим, что в случае, когда несколько признаков линейно зависимы, веса w_i при них теряют физический смысл. Может оказаться, что вес признака, с ростом которого таргет, казалось бы, должен увеличиваться, станет отрицательным. Это делает модель не только неточной, но и принципиально не интерпретируемой. Вообще, неадекватность знаков или величины весов – хорошее указание на мультиколлинеарность.

Для того чтобы справиться с этой проблемой, задачу обычно регуляризуют, то есть добавляют к ней дополнительное ограничение на вектор весов. Это ограничение можно, как и исходный loss, задавать по-разному, но, как правило, ничего сложнее, чем L_1- и L_2-нормы, не требуется.

Вместо исходной задачи теперь предлагается решить такую:

$$\min_w L(f,X,y) = \min_w(|Xw - y|_2^2 + \lambda|w|_k^k)$$

λ – это очередной параметр, а $|w|_k^k$ – это либо $|w|_2^2 = w_1^2 + \cdots + w_D^2$, либо $|w|_1^1 = |w_1| + \cdots + |w_D|$

Добавка $\lambda|w|_k^k$ называется регуляризационным членом или регуляризатором, а число λ – коэффициентом регуляризации. Коэффициент λ является гиперпараметром модели и достаточно сильно влияет на качество итогового решения. Его подбирают по логарифмической шкале (скажем, от 1e-2 до 1e+2), используя для сравнения моделей с разными значениями λ дополнительную валидационную выборку. При этом качество модели с подобранным коэффициентом регуляризации уже проверяют на тестовой выборке, чтобы исключить переобучение.

Отдельно надо договориться о том, что вес w_0 соответствующий отступу от начала координат (то есть признаку из всех единичек), мы регуляризовать не будем, потому что это не имеет смысла: если даже все значения y равномерно велики, это не должно портить качество обучения. Обычно это не отображают в формулах, но если придираться к деталям, то стоило бы написать сумму по всем весам, кроме w_0: $|w|_2^2 = \sum_{j=1}^D w_j^2$ и $|w|_1 = \sum_{j=1}^D |w_j|$

12.2 Гребневая регрессия Ridge (L_2)

В случае L_2-регуляризации решение задачи изменяется не очень сильно.

12.2.1 Формула решения

Продифференцировав новый loss по w, легко получить, что «точное» решение имеет вид:

$$w = (X^TX + \lambda I)^{-1}X^Ty$$

Отметим, что за этой формулой стоит и понятная численная интуиция: раз матрица X^TX близка к вырожденной, то обращать её сродни самоубийству. Мы лучше слегка исказим её добавкой λI, которая увеличит все собственные значения на λI, отодвинув их от нуля. Да, аналитическое решение перестаёт быть «точным», но за счёт снижения численных проблем мы получим более качественное решение, чем при использовании «точной» формулы.

В свою очередь, градиент функции потерь

$$\min_w L(f_w,X,y) = (|Xw - y|^2 + \lambda|w|^2)$$

по весам теперь выглядит так:

$$\nabla_w L(f_w,X,y) = 2X^T(Xw - y) + 2\lambda w$$

Подставив этот градиент в алгоритм стохастического градиентного спуска, мы получаем обновлённую версию приближенного алгоритма, отличающуюся от старой только наличием дополнительного слагаемого.

12.3 Lasso-регуляризация (L_1)

Отметим, что L_1- и L_2-регуляризацию можно определять для любой функции потерь $L(w,X,y)$ (и не только в задаче регрессии, а и, например, в задаче классификации тоже). Новая функция потерь будет соответственно равна $L^{(}w,X,y) = L(w,X,y) + \lambda|w|_1$ или $L^{(}w,X,y) = L(w,X,y) + \lambda|w|_2^2$

L_2-регуляризация работает прекрасно и используется в большинстве случаев, но есть одна полезная особенность L_1-регуляризации: её применение приводит к тому, что у признаков, которые не оказывают большого влияния на ответ, вес в результате оптимизации получается равным 0. Это позволяет удобным образом удалять признаки, слабо влияющие на таргет. Кроме того, это даёт возможность автоматически избавляться от признаков, участвующих в соотношениях приближённой линейной зависимости, соответственно, спасает от проблем, связанных с мультиколлинеарностью, о которых мы писали выше.

12.4 Вероятностная интерпретация

Байесовский подход к регуляризации. Регуляризация имеет естественную интерпретацию в рамках байесовской статистики как априорное распределение на параметры.

Ridge-регрессия как апостериорное среднее
[Байесовская интерпретация Ridge] Если:

1. Правдоподобие: $p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\beta}, \sigma^2) = \mathcal{N}(\boldsymbol{X}\boldsymbol{\beta}, \sigma^2 \boldsymbol{I}_n)$
2. Априорное распределение: $p(\boldsymbol{\beta} \mid \tau^2) = \mathcal{N}(\boldsymbol{0}, \tau^2 \boldsymbol{I}_p)$

Тогда апостериорное распределение:

$$p(\boldsymbol{\beta} \mid \boldsymbol{y}, \boldsymbol{X}, \sigma^2, \tau^2) = \mathcal{N}(\hat{\boldsymbol{\beta}}_{\text{Ridge}}, \boldsymbol{\Sigma}_{\text{post}})$$

где:

$$\hat{\boldsymbol{\beta}}_{\text{Ridge}} = \left(\frac{1}{\sigma^2}\boldsymbol{X}^\top \boldsymbol{X} + \frac{1}{\tau^2}\boldsymbol{I}_p\right)^{-1} \frac{1}{\sigma^2}\boldsymbol{X}^\top \boldsymbol{y}$$
$$\boldsymbol{\Sigma}_{\text{post}} = \left(\frac{1}{\sigma^2}\boldsymbol{X}^\top \boldsymbol{X} + \frac{1}{\tau^2}\boldsymbol{I}_p\right)^{-1}$$

Доказательство. По теореме Байеса:

$$p(\boldsymbol{\beta} \mid \boldsymbol{y}, \boldsymbol{X}, \sigma^2, \tau^2) \propto p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\beta}, \sigma^2) \cdot p(\boldsymbol{\beta} \mid \tau^2)$$

Логарифм апостериорного распределения:

$$\begin{aligned}\log p(\boldsymbol{\beta} \mid \cdot) &\propto -\frac{1}{2\sigma^2}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|^2 - \frac{1}{2\tau^2}\|\boldsymbol{\beta}\|^2 \\ &\propto -\frac{1}{2}\left[\frac{1}{\sigma^2}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|^2 + \frac{1}{\tau^2}\|\boldsymbol{\beta}\|^2\right]\end{aligned}$$

Максимизация логарифма апостериорного распределения (MAP-оценка) эквивалентна минимизации:

$$\frac{1}{\sigma^2}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|^2 + \frac{1}{\tau^2}\|\boldsymbol{\beta}\|^2$$

, что совпадает с задачей Ridge-регрессии при $\lambda = \frac{\sigma^2}{\tau^2}$. □

Lasso как MAP-оценка
[Байесовская интерпретация Lasso] Если:

1. Правдоподобие: $p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\beta}, \sigma^2) = \mathcal{N}(\boldsymbol{X}\boldsymbol{\beta}, \sigma^2 \boldsymbol{I}_n)$
2. Априорное распределение: $p(\boldsymbol{\beta} \mid \lambda) = \prod_{j=1}^{p} \frac{\lambda}{2} e^{-\lambda|\beta_j|}$ (независимые распределения Лапласа)

Тогда MAP-оценка совпадает с Lasso-решением:

$$\hat{\boldsymbol{\beta}}_{\text{MAP}} = \arg\max_{\boldsymbol{\beta}} p(\boldsymbol{\beta} \mid \boldsymbol{y}, \boldsymbol{X}) = \arg\min_{\boldsymbol{\beta}} \left\{\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|^2 + 2\sigma^2\lambda\|\boldsymbol{\beta}\|_1\right\}$$

Доказательство. Априорное распределение Лапласа:

$$p(\boldsymbol{\beta} \mid \lambda) = \left(\frac{\lambda}{2}\right)^p e^{-\lambda \sum_{j=1}^p |\beta_j|} = \left(\frac{\lambda}{2}\right)^p e^{-\lambda \|\boldsymbol{\beta}\|_1}$$

Логарифм апостериорного распределения:

$$\begin{aligned}\log p(\boldsymbol{\beta} \mid \boldsymbol{y}, \boldsymbol{X}) &\propto \log p(\boldsymbol{y} \mid \boldsymbol{X}, \boldsymbol{\beta}) + \log p(\boldsymbol{\beta} \mid \lambda) \\ &\propto -\frac{1}{2\sigma^2}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|^2 - \lambda\|\boldsymbol{\beta}\|_1\end{aligned}$$

Максимизация эквивалентна минимизации:

$$\frac{1}{2\sigma^2}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|^2 + \lambda\|\boldsymbol{\beta}\|_1$$

, что соответствует Lasso с $\alpha = \lambda\sigma^2$. □

Выводы из вероятностной интерпретации

1. Регуляризация соответствует введению априорных знаний о параметрах
2. Гиперпараметр регуляризации λ связан с параметрами априорного распределения
3. Использование L_2-регуляризации предполагает, что все коэффициенты малы и имеют нулевое среднее.
4. Использование L_1-регуляризации предполагает, что многие коэффициенты равны нулю (разреженность)
5. Байесовский подход позволяет естественным образом оценивать неопределенность предсказаний

12.5 Elastic Net

Можно пойти и далее, и соединить в себе оба регуляризационных члена:

$$L_{EN} = |Y - Xw|_2^2 + \lambda_1^2|w|_1 + \lambda_2^2|w|_2^2$$

Что лучше работает? Лучше работает кросс-валидация, но если выбирать из этих вариантов, то обычно L_2.

12.6 Свойства регуляризаций

Метод	**Априорное распределение**	**Свойства**
МНК	Неинформативное (равномерное)	Нет регуляризации
Ridge	Гауссово $\mathcal{N}(0, \tau^2 I)$	Неразреженное решение
Lasso	Лапласовское Laplace$(0, 1/\lambda)$	Разреженное решение
Elastic Net	Комбинация Гаусса и Лапласа	Компромисс

Таблица 2: Байесовская интерпретация различных методов регуляризации

13 Метрики регрессии MSE, MAE, RMSE, MAPE, SMAPE, R_2

MSE.

$$MSE(f,X,y) = \frac{1}{N}(y - \tilde{y})^2$$

Стохастический градиентный спуск можно очевидным образом обобщить для решения задачи линейной регрессии с любой другой функцией потерь, не только квадратичной: ведь всё, что нам нужно от неё, – это чтобы у функции потерь был градиент. На практике это делают редко, но тем не менее рассмотрим ещё пару вариантов.

13.1 Когда их используют

MAE. Mean absolute error, абсолютная ошибка, появляется при замене L_2 нормы в MSE на L_1:

$$MAE(y,\tilde{y}) = \frac{1}{N}\sum_{i=1}^{N} = |y_i - \tilde{y_i}|$$

Можно заметить, что в MAE по сравнению с MSE существенно меньший вклад в ошибку будут вносить примеры, сильно удалённые от ответов модели. Дело тут в том, что в MAE мы считаем модуль расстояния, а не квадрат, соответственно, вклад больших ошибок в MSE получается существенно больше. Такая функция потерь уместна в случаях, когда вы пытаетесь обучить регрессию на данных с большим количеством выбросов в таргете.

На эту разницу можно посмотреть так: MSE приближает матожидание условного распределения $y|x$, а MAE – медиану.

RMSE.

$$RMSE(f,X,y) = \sqrt{MSE}$$

Метрика штрафует за большие ошибки. Поскольку ошибки возводятся в квадрат, RMSE очень чувствительна к выбросам. Это полезно, когда крупные ошибки критически нежелательны (например, прогноз паводка). Также применяют, когда

- ошибки распределены нормально.
- Когда большие ошибки должны иметь большее влияние на оценку модели (это одновременно и сила, и слабость).
- Когда нужно получить интерпретируемую в "физических" единицах метрику

MAPE. Mean absolute percentage error, относительная ошибка.

$$MAPE(y,\tilde{y}) = \frac{1}{N}\sum_{i=1}^{N}\left|\frac{y_i - \tilde{y_i}}{y_i}\right|$$

Часто используется в задачах прогнозирования (например, погоды, загруженности дорог, кассовых сборов фильмов, цен), когда ответы могут быть различными по порядку величины, и при этом мы бы хотели верно угадать порядок, то есть мы не хотим штрафовать модель за предсказание 2000 вместо 1000 в разы сильней, чем за предсказание 2 вместо 1.

Член вычитаемый из 1 можно интерпретировать как оценённую дисперсию, отнесённую к реальной дисперсии. Ещё можно сказать, что мы модель сравниваем с моделью, которая предсказывает просто константу. Смотрим насколько далеко ушла наша модель от тупого предсказания среднего. Может иметь отрицательные значения, так как числитель может неограниченно расти.

SMAPE Когда есть большие выбросы в MAPE, следует использовать отнормированную и на прогнозируемое значение метрику.

$$SMAPE(y,\tilde{y}) = \frac{1}{N}\sum_{i=1}^{N}\frac{2|y_i - \tilde{y_i}|}{|y_i| + |\tilde{y_i}|}$$

R^2

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(Y_i - \tilde{Y_i})^2}{\sum_{i=1}^{N}(Y_i - \overline{Y_i})^2}$$

Её используют, когда:

1. Важно объяснить, а не просто предсказать. («Какие факторы влияют на продажи и насколько сильно?»)
2. Нужно сравнить качество моделей, обученных на данных с разным масштабом целевой переменной.
3. Требуется интуитивно понятная для заказчика метрика в процентах («Наша модель объясняет 90% колебаний»).

14 Параметры и гиперпараметры

14.1 Определения, свойства, использование, примеры

Отличие параметров модели от гиперпараметров:

- параметры настраиваются в процессе обучения модели на данных. Например, веса в линейной регрессии, нейросетях, структура решающего дерева;
- гиперпараметры – это характеристики модели, которые фиксируются до начала обучения: глубина решающего дерева, значение силы регуляризации в линейной модели, learning rate для градиентного спуска.

Рассмотрим, например, модель линейной регрессии $f(X) = Xw$, где $w = (w_0,w_1,\ldots,w_n)$ – веса модели, $X = (x_{ij})$ – матрица, в которой каждая строка содержит признаки одного объекта выборки (для удобства записи считаем, что первый столбец в этой матрице константный).

Эта модель может обучаться посредством минимизации функционала $L = |y - Xw|^2 + C|w|^2$, где y – целевая переменная,C – коэффициент регуляризации. В процессе минимизации L веса w настраиваются по обучающей выборке, то есть являются параметрами. В то же время величина коэффициента регуляризации задаётся до начала обучения, то есть она – гиперпараметр.

Свойства параметров

- Изучаются (Learned): настраиваются алгоритмом оптимизации (например, градиентным спуском).
- Внутренние: определяют, как модель преобразует входные данные в прогноз.
- Зависимость от данных: их значения полностью зависят от обучающего набора
- Вы не выбираете их вручную.
- Вы контролируете процесс их обучения через:
  1. Выбор функции потерь (MSE, Cross-Entropy).
  2. Выбор оптимизатора (SGD, Adam).
  3. Настройку гиперпараметров
- После обучения они фиксируются и используются для предсказаний.

Свойства гиперпараметров

- Задаются (Set): выбираются экспериментально, по правилам или автоматически.
- Внешние: управляют «поведением» алгоритма обучения.
- Независимость от данных: выбираются до обучения и не зависят от конкретных значений в обучающей выборке (хотя выбор зависит от её характера).

Гиперпараметры критически важны, так как от них зависит, сможет ли модель вообще чему-то научиться и как хорошо. Вот как их использовать

1. Определить ключевые гиперпараметры для настройки (max depth, estimators number, min samples split).
2. Выбрать метод настройки (например, Random Search с кросс-валидацией на Train).
3. Оценить производительность лучшей конфигурации на Validation-сете.
4. Зафиксировать лучшие гиперпараметры.
5. Ещё раз обучить модель на объединённых Train+Validation данных с этими гиперпараметрами.
6. Финальная оценка на слепом test-сете (качество на нём — ваш итоговый результат).
7. Prod: Развернуть модель с найденными гиперпараметрами и обученными параметрами.

Настройка гиперпараметров – это часть обучения модели. Поэтому ни в коем случае нельзя использовать test-сет для этой настройки. Иначе вы «подсмотрите» в тестовые данные, и ваша оценка качества будет чрезмерно оптимистичной (data leakage). Для настройки используйте validation-сет или кросс-валидацию

Часть IV
Валидация и оценка качества

15 Валидация и оценка модели

15.1 Недообучение и переобучение, определения и способы обнаружения.

Недообучение – ситуация, когда модель уловила не все общие закономерности и не способна достаточно точно воспроизвести распределение, из которого создаются объекты. Переобучение – ситуация, когда модель не только успешно смоделировала распределение, но и включила в него шумовые факторы (то есть переобучилась под выбросы). Как детектировать это все? Вспомним про train и test, будем измерять loss на них в зависимости от сложности модели (например, число эпох или норма вектора весов в линейной регрессии). Если и на train, и на test loss падает, то мы уловили не все основополагающие зависимости, то есть недообучились. Если на train loss падает, а на test loss растёт, то мы уловили шумовые зависимости, чуждые распределению, то есть переобучились

15.2 Способы разбиения датасета

15.2.1 Что такое хорошее разбиение, идеальный и практический варианты

Хорошее разбиение должно удовлетворять следующим критериям:

1. **Репрезентативность:** каждое подмножество (train/validation/test) должно отражать распределение данных генеральной совокупности
2. **Независимость:** отсутствие пересечений между подмножествами
3. **Адекватный размер:**
   - train: достаточно для обучения модели (обычно 60-80 %)
   - validation: достаточно для надёжной оценки качества при настройке гиперпараметров
   - test: достаточно для финальной оценки обобщающей способности
4. **Сохранение структуры:** для зависимых данных (временные ряды, панельные данные) необходимо сохранять временной порядок
5. **Стратификация:** сохранение распределения целевой переменной в каждом подмножестве

Идеальный и практический варианты

Идеальный вариант	**Практический вариант**
Огромный объём данных ($n > 10^6$)	Ограниченный объем данных
Train: 80%, Val: 10%, Test: 10%	Train: 60%, Val: 20%, Test: 20%
Стационарное распределение	Возможны дрейфы распределения
i.i.d. наблюдения	Возможны зависимости
Простое случайное разбиение	Стратифицированное/временное разбиение
Много ресурсов для перекрестной проверки	Ограниченные вычислительные ресурсы

Таблица 3: Сравнение идеального и практического сценариев

15.2.2 4 основных типа разбиений

Простое случайное разбиение (Hold-out)
Преимущества

- Простота реализации
- Быстрота выполнения.
- Подходит для очень больших датасетов

Недостатки

- Неэффективное использование данных
- Высокая дисперсия оценки качества
- Риск несбалансированного разбиения

K-Fold Cross-Validation (K-блочная перекрестная проверка)
Преимущества

- Эффективное использование данных
- Более надёжная оценка качества
- Меньшая дисперсия оценки

Недостатки

- k раз больше вычислений
- Не подходит для зависимых данных.
- При больших k увеличивается корреляция между folds

Стратифицированная K-Fold Cross-Validation
Преимущества

- Сохраняет распределение классов
- Особенно важно для несбалансированных данных
- Более стабильная оценка

Временное разбиение (Time-based split)
Преимущества

- Реалистичная оценка для временных данных.
- Учитывает временные зависимости.
- Избегает утечки информации из будущего

15.3 Этапы построения модели

Полный конвейер построения модели состоит из 8 этапов
1. Сбор и понимание данных

- Определение бизнес-цели
- Сбор релевантных данных
- Исследовательский анализ данных (EDA)
- Проверка гипотез о данных

2. Предобработка и очистка данных

- Обработка пропущенных значений
- Обработка выбросов

- Кодирование категориальных признаков
- Масштабирование числовых признаков

3. Разбиение данных

- Выбор стратегии разбиения
- Создание train/validation/test наборов
- Проверка репрезентативности разбиения

4. Инженерия признаков

- Создание новых признаков
- Отбор признаков
- Уменьшение размерности

5. Выбор и обучение модели

- Выбор семейства моделей
- Обучение на train данных
- Валидация на validation данных

6. Настройка гиперпараметров

- Grid Search / Random Search
- Bayesian Optimization
- Выбор лучшей конфигурации

7. Оценка модели

- Финальная оценка на test данных
- Анализ ошибок
- Интерпретация результатов

8. Deploy и monitoring

- Развертывание модели
- Создание API
- Мониторинг качества
- Периодическое переобучение

15.3.1 Train, validation, test, мотивация каждого

Тренировочная выборка – часть полной выборки, на котором мы обучаем модель.

Валидационная выборка – часть полной выборки, происходит настройка гиперпараметров. Это мы и назовём валидированием или валидацией модели. Изначально человек из каких-то знаний, опыта и других соображений выбирает некоторый пул гиперпараметров для текущей задачи. И далее по ним с каким-то шагом запускаем модель и проверяем получившиеся предсказания с помощью метрик модели. Выбирает и фиксирует значения гиперпараметров.

Тестовая выборка – часть полной выборки, который мы заранее отложили и проверяем значение функции потерь, а также метрики качества модели. НЕ нАДо ВАЛИДИроВАться нА тЕстоВоЙ ВыБорКЕ. Считайте, что проверка на тестовой выборке – это как финальная проверка работы. Уже с оптимизированными параметрами и гиперапараметрами запускаем модель, чтобы проверить её качество на этой части данных. Иногда для конечного теста, в тестовую выборку докидывают и валидационный датасет, но такой шаг является спорным.

15.3.2 Кросс-валидация

Кросс-валидация – это процедура для оценки качества работы модели, которая широко применяется в машинном обучении. Она помогает сравнить между собой различные модели и выбрать наилучшую для конкретной задачи

Для работы модели нужно определить ее гиперпараметры. Минус подхода с разбиениями в том, что модель будет зависеть от случайного выбора куска train (а вдруг объекты в train упорядочены по алфавиту и не все объекты попадут в обучающую выборку при определении гиперпараметров. Модель с этими гиперпараметрами может показать плохой результат на всей обучающей выборке). Вторая идея (кросс-валидация): часть объектов из train идёт на валидацию, часть на train, затем данный процесс повторяется несколько раз (часть с валидацией меняется). Мы можем измерить дисперсию ошибки для каждого набора гиперпараметров и сам loss, чтобы выбрать оптимальный набор. Далее речь идёт о K-fold кросс-валидации, где K – число запусков кросс-валидации и одновременно то, на сколько примерно равных кусков мы бьём train. Тогда в рамках каждого запуска один из K блоков является валидацией, а остальные — train. При увеличении числа блоков дисперсия ответа уменьшается (закономерно, ибо объём данных увеличивается). Аналогичная ситуация с bias – матожидание разности между истинным ответом и выданным алгоритмом – он уменьшается Чем меньше количество блоков, тем быстрее это работает, но тем меньше шанс, что модель подстроится под какой-то кусок данных

Кросс-валидацию следует делать, когда данных мало, потому что она работает лучше. Но, когда выборки огромные, делать кросс-валидацию очень долго. Используем Train, validation and test stage поскольку количество данных и так позволяет хорошо обучить модель.

15.3.3 Как выбрать организацию этапов

Тип задачи	**Рекомендуемая стратегия**	**Обоснование**
Маленький датасет ($n < 1000$)	Вложенная перекрестная проверка	Максимальное использование данных
Несбалансированные данные	Стратифицированная CV + oversampling/undersampling	Сохранение распределения классов
Временные ряды	Временное разбиение + expanding window	Сохранение временной структуры
Большой датасет ($n > 10^5$)	Hold-out + простые модели	Скорость вычислений важнее точности
Конкурс Kaggle	Public/private split + ensemble	Избегание overfitting на public leaderboard

Таблица 4: Выбор стратегии в зависимости от задачи

Принципы выбора

1. **Размер данных**
   - Маленькие данные: перекрестная проверка (5-10 folds)
   - Большие данные: hold-out (80/20 или 70/30)
2. **Распределение данных**
   - Сбалансированные: простая K-fold
   - Несбалансированные: стратифицированная K-fold
3. **Зависимости в данных**
   - Независимые: случайное разбиение
   - Временные: временное разбиение
   - Групповые: групповые K-fold
4. **Вычислительные ресурсы**
   - Ограниченные: hold-out или small K
   - Большие: K-fold с большим K или повторная CV
5. **Цель исследования**

- Быстрый прототип: hold-out
- Научная статья: K-fold CV
- Продукт: nested CV + финальный test set

Вложенная перекрестная проверка (Nested Cross-Validation)
Для надёжной оценки модели с настройкой гиперпараметров

15.4 Утечки данных

Data leak – утечка данных, при которой обучение происходит в том числе и на тестовых данных. К основным типам утечек относят

Train-Test Contamination (Загрязнение обучающего и тестового наборов)

- **Причина:** предобработка данных выполняется на всем датасете до разбиения
- **Пример:** масштабирование с вычислением mean/std по всему датасету
- **Решение:** все преобразования должны вычисляться только на train данных и применяться к test

3. Time Leakage (Временная утечка)

- **Причина:** использование информации из будущего для предсказания прошлого
- **Пример:** для предсказания курса акций используются данные, которые стали известны позже
- **Решение:** строгое временное разбиение, использование только исторических данных

4. Data Duplication (Дублирование данных)

- **Причина:** одинаковые или очень похожие объекты попадают и в train, и в test
- **Пример:** разные фотографии одного человека в разных наборах
- **Решение:** проверка на дубликаты перед разбиением, групповые разбиения

Практические рекомендации

1. **Всегда разделяйте данные первым шагом.**
2. **Используйте Pipeline.**
3. **Проверяйте временные метки:** убедитесь, что в train нет данных с более поздними временными метками, чем в test.
4. **Анализируйте признаки:** признаки с нереалистично высокой важностью могут указывать на утечку.
5. **Сравнивайте распределения:** признаки в train и test должны иметь схожие распределения.
6. **Используйте здравый смысл:** если модель предсказывает лучше, чем кажется возможным, проверьте на утечку

Примеры утечек в реальных задачах

Метрики для обнаружения утечек

- **Разница в качестве:** если test accuracy $\gg$ validation accuracy.
- **Анализ признаков:** признаки, которые не должны влиять, имеют высокую важность.
- **Аномальная корреляция:** слишком высокая корреляция признаков с целевой переменной
- **Проверка на шум:** если модель хорошо предсказывает на полностью случайных данных

Задача	Типичная утечка	Решение
Медицинская диагностика	Пациент присутствует в train и test	Группировка по пациенту
Прогнозирование продаж	Использование итоговых сумм месяца	Использовать только данные на момент прогноза
Рекомендательные системы	Использование будущих взаимодействий	Строгое временное разделение
Обработка текстов	Обучение TF-IDF на всем корпусе	Обучение векторизатора только на train
Анализ временных рядов	Использование скользящего среднего с окном, включающим будущее	Только исторические данные

Таблица 5: Примеры утечек в различных задачах

Заключение

Правильное разбиение данных и избегание утечек — фундаментальные аспекты построения надёжных ML-моделей. Ключевые принципы:

1. Разделяй данные как можно раньше.
2. Используй Pipeline для инкапсуляции преобразований.
3. Выбирай стратегию разбиения в зависимости от природы данных
4. Всегда сохраняй отдельный тестовый набор для финальной оценки.
5. Будь подозрительным к слишком хорошим результатам

16 Линейная классификация

Теперь давайте поговорим про задачу классификации. Для начала будем говорить про бинарную классификацию на два класса. Обобщить эту задачу до задачи классификации на K классов не составит большого труда.

16.1 Постановка задачи классификации

Мы хотим обучить линейную модель так, чтобы плоскость, которую она задаёт, как можно лучше отделяла объекты одного класса от другого.

В идеальной ситуации найдётся плоскость, которая разделит классы: положительный окажется с одной стороны от неё, а отрицательный с другой. Выборка, для которой это возможно, называется линейно разделимой. Увы, в реальной жизни такое встречается крайне редко.

Как обучить линейную модель классификации, нам ещё предстоит понять, но уже ясно, что итоговое предсказание можно будет вычислить по формуле $y = sgn(w,x_i)$

Сконструируем теперь функционал ошибки так, чтобы он вышеперечисленными проблемами (регрессии) не обладал. Мы хотим минимизировать число ошибок классификатора, то есть

$$\mathbb{I}(y_i \neq sgn(w,x_i)) \to \min_w$$

Домножим обе части на y_i и немного упростим

$$\mathbb{I}(y_i < sgn(w,x_i)) \to \min_w$$

Величина $M = y_i(w, x_i)$ называется отступом (margin) классификатора. Такая функция потерь называется misclassification loss. Свойство.

- Отступ положителен, когда $sgny_i = sgn(w,x_i)$, то есть класс угадан верно; при этом чем больше отступ, тем больше расстояние от x_i до разделяющей гиперплоскости, то есть «уверенность классификатора»;
- отступ отрицателен, когда $sgny_i \neq sgn(w,x_i)$, то есть класс угадан неверно; при этом чем больше по модулю отступ, тем более сокрушительно ошибается классификатор.

От каждого из отступов мы вычисляем функцию $F(M) = \mathbb{I}(M < 0)$. Она кусочно-постоянная, и из-за этого всю сумму невозможно оптимизировать градиентными методами: ведь её производная равна нулю во всех точках, где она существует. Но мы можем мажорировать её какой-нибудь более гладкой функцией, и тогда задачу можно будет решить.

Мы остаёмся в домене supervised learning, то есть уже заданные ответы, на заданных объектах. Множество ответов имеет меньше чем континуальную мощность. То есть множество классов, которое мы предсказываем C – конечное множество.

$$X \in \mathbb{R}^{n \times p}, Y \in C^n, |C| < \inf$$

Например, бинарное множество классов, которое мы будем рассматривать большую часть лекции: $C = -1, 1$. То есть будем заниматься бинарной классификацией, а потом уже перейдем к мультиклассовой. Соответственно, наша цель – построить такой классификатор, зависящий от данных, который бы приближал вектор ответов достаточно хорошо: $c(X) = \tilde{Y} \approx Y$. Что значит достаточно хорошо – это мы и обсудим

16.2 Варианты типов меток

Пусть теперь наши таргеты y кодируют принадлежность к положительному или отрицательному классу, то есть принадлежность множеству -1,1 (в этом параграфе договоримся именно так обозначать классы, хотя в жизни вам будут нередко встречаться и метки 0,1), а x – по-прежнему векторы из $\mathbb{R}^D$.

0,1 используются даже чаще. Разнознаковые обычно в контексте SVM (Support Vector Machines) по историческим и математическим причинам

16.3 Определение линейного классификатора и геометрическая интерпретация

Допустим у нас есть матрица признаков и вектор весов, как из этого всего, что уже было в линейной регрессии, составить линейную классификацию? Надо делить на классы предсказанный ответ, обрубая по какой-то границе, если у нас два класса. $c(x) = \mathbb{I}(x \geq 0)$ Здесь граница поставлена ровно 0, почему это не сужает множество классификаторов, которые мы получим и не нарушает общности? Ещё на прошлой лекции мы ввели свободный член (bias term) и не пишем его в формулах, чтобы их сократить, но он тренируется при обучении модели. Запишем чуть более математично, выражение эквивалентно следующему: $c(x) = sgn(f(x)) = sgn(x^T w)$

Существует геометрическая интерпретация данного выражения: вектор w является нормалью к гиперплоскости в p-мерном пространстве фичей (при равенстве скалярного произведения нулю $x^T w = 0$), с одной стороны гиперплоскости будут одного класса объекты, с другой – другого класса объекта.

16.4 Маржа, определение и ключевое свойство

Введём понятие margin (отступ): $M_i = y_i f(x_i) = y_i x_i^T w$ Это некоторый аналог ориентированного расстояния, причём если $M_i \leq 0$, значит объект классифицирован неверно. Отступ обычно используют в теоретических доказательствах (величина проекции на верный класс).

Тогда логично предложить следующую функцию потерь: $Loss = \sum_i M_i \leq 0$ Но у такой loss function будут проблемы, так как это не гладкая функция. Это главная причина почему мы не будем использовать такой loss, по крайней мере в наших текущих задачах. Есть ещё один менее очевидный минус: такая метрика ничего нам не говорит об уверенности классификатора. Представьте объекты как точки в пространстве признаков, есть гиперплоскость классификатора, и было бы разумно полагать, что объекты на границе, где перемешиваются два класса, будут с меньшей уверенностью разделены классификатором, чем лежащие далеко "в толще"класса, которые с большей точностью принадлежат своему классу.

16.5 Выбор весов

Общая задача оптимизации

Веса линейного классификатора выбираются путём минимизации эмпирического риска:

$$\hat{\boldsymbol{w}}, \hat{b} = \arg\min_{\boldsymbol{w}, b} \left[\frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(y_i, \boldsymbol{w}^\top \boldsymbol{x}_i + b) + \lambda R(\boldsymbol{w}) \right]$$

где:

- $\mathcal{L}(y, z)$ — функция потерь (loss function)
- $R(\boldsymbol{w})$ — регуляризатор

- $\lambda \geq 0$ — параметр регуляризации

Основные подходы к выбору весов

1. **Перцептрон Розенблатта:** Итеративный алгоритм, сходится для линейно разделимых данных

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} + \eta \cdot y_i \boldsymbol{x}_i \cdot \mathbb{I}[y_i(\boldsymbol{w}^{(t)\top} \boldsymbol{x}_i) \leq 0]$$

2. **Логистическая регрессия:** максимизация правдоподобия с логистической функцией потерь
3. **Метод опорных векторов (SVM):** максимизация запаса с hinge-функцией потерь
4. **Метод наименьших квадратов:** минимизация MSE (редко используется для классификации)

16.6 Почему для классификации много разных функций потерь

Вариантов приблизить ступеньку много и в зависимости от задачи и свойств берут разное приближение

17 Логистическая регрессия

В этом параграфе мы будем обозначать классы нулём и единицей.

Ещё один интересный метод появляется из желания посмотреть на классификацию как на задачу предсказания вероятностей. Хороший пример – предсказание кликов в интернете (например, в рекламе и поиске). Наличие клика в обучающем логе не означает, что, если повторить полностью условия эксперимента, пользователь обязательно кликнет по объекту опять. Скорее у объектов есть какая-то "кликабельность", то есть истинная вероятность клика по данному объекту. Клик на каждом обучающем примере является реализацией этой случайной величины, и мы считаем, что в пределе в каждой точке отношение положительных и отрицательных примеров должно сходиться к этой вероятности.

Проблема состоит в том, что вероятность, по определению, величина от 0 до 1, а простого способа обучить линейную модель так, чтобы это ограничение соблюдалось, нет. Из этой ситуации можно выйти так: научить линейную модель правильно предсказывать какой-то объект, связанный с вероятностью, но с диапазоном значений $(-\inf, \inf)$, и преобразовать ответы модели в вероятность. Таким объектом является logit или log odds – логарифм отношения вероятности положительного события к отрицательному $\log \frac{p}{1-p}$.

Если ответом нашей модели является $\log \frac{p}{1-p}$, то искомую вероятность посчитать не трудно:

$$(w, x_i) = \log \frac{p}{1-p}$$
$$e^{(w,x_i)} = \frac{p}{1-p}$$
$$p = \frac{1}{1+e^{-(w,x_i)}}$$

Таким образом, $p = \sigma((w,x_i))$

Как теперь научиться оптимизировать w так, чтобы модель как можно лучше предсказывала logit? Нужно применить метод максимума правдоподобия для распределения Бернулли. Это самое простое распределение, возникающее, например, при бросках монетки, которая орлом выпадает с вероятностью p. У нас только событием будет не орёл, а то, что пользователь кликнул на объект с такой вероятностью.

Правдоподобие позволяет понять, насколько вероятно получить данные значения таргета y при данных X и весах w. Оно имеет вид $p(y|X,w) = \prod_i p(y_i|x_i,w)$ и для распределения Бернулли его можно выписать так:

$$p(y|X,w) = \prod_i p_i^{y_i}(1-p_i)^{1-y_i}$$

Здесь p_i – это вероятность, посчитанная из ответов модели. Оптимизировать произведение неудобно, хочется иметь дело с суммой, так что мы перейдём к логарифмическому правдоподобию и подставим формулу для вероятности, которую мы получили выше:

$$l(w,X,y) = \sum_i y_i \log(p_i) + (1-y_i)\log(1-p_i) = \sum_i (y_i \log(\sigma((w,x_i))) + (1-y_i)\log(1-\sigma((w,x_i))))$$

17.1 Базовое предположение и геометрическая интерпретация

Вероятностная модель и предположения

[Логистическая регрессия] Логистическая регрессия моделирует условную вероятность принадлежности к классу 1 (бинарный случай) как:

$$P(y = 1 \mid \boldsymbol{x}; \boldsymbol{w}, b) = \sigma(\boldsymbol{w}^\top \boldsymbol{x} + b) = \frac{1}{1 + e^{-(\boldsymbol{w}^\top \boldsymbol{x} + b)}}$$

Здесь $\sigma(z) = \frac{1}{1+e^{-z}}$ — **сигмоидная (логистическая) функция**.

Ключевые предположения

1. **Вероятностная модель:** предполагается, что $y_i \mid \boldsymbol{x}_i$ следуют распределению Бернулли:

$$y_i \mid \boldsymbol{x}_i \sim \text{Bernoulli}(p_i), \quad p_i = \sigma(\boldsymbol{w}^\top \boldsymbol{x}_i + b)$$

2. **Линейность по logit (log-odds):** основное предположение — линейная зависимость между признаками и **logit** вероятности:

$$\text{logit}(p_i) = \ln\left(\frac{p_i}{1 - p_i}\right) = \boldsymbol{w}^\top \boldsymbol{x}_i + b$$

3. **Условная независимость:** наблюдения $(y_i, \boldsymbol{x}_i)$ предполагаются i.i.d.

4. **Отсутствие мультиколлинеарности:** Столбцы матрицы признаков $\boldsymbol{X}$ должны быть линейно независимы (для идентифицируемости модели)

Геометрическая интерпретация

Преобразование линейной комбинации в вероятность

Логистическая регрессия преобразует линейную комбинацию признаков в вероятность с помощью сигмоидной функции как на Рисунке 1

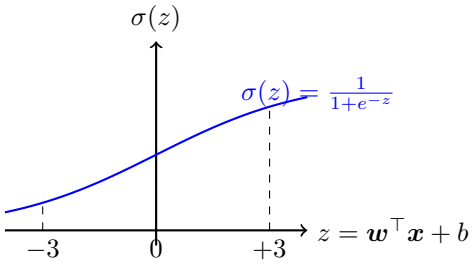


Рис. 1: Логистическая регрессия: преобразование линейной комбинации в вероятность (слева) и разделение в пространстве признаков (справа)

Изолинии вероятностей

В пространстве признаков $\mathbb{R}^d$ изолинии постоянной вероятности представляют собой **параллельные гиперплоскости**:

$$P(y = 1 \mid \boldsymbol{x}) = c \quad \Leftrightarrow \quad \boldsymbol{w}^\top \boldsymbol{x} + b = \text{logit}(c) = \ln\left(\frac{c}{1 - c}\right)$$

Таким образом, для фиксированной вероятности c получаем гиперплоскость:

$$\boldsymbol{w}^\top \boldsymbol{x} + b = \text{const}$$

- При $c = 0{,}5$: $\boldsymbol{w}^\top \boldsymbol{x} + b = 0$ — решающая граница
- При $c = 0{,}9$: $\boldsymbol{w}^\top \boldsymbol{x} + b = \ln(9) \approx 2.197$
- При $c = 0{,}1$: $\boldsymbol{w}^\top \boldsymbol{x} + b = \ln(1/9) \approx -2.197$

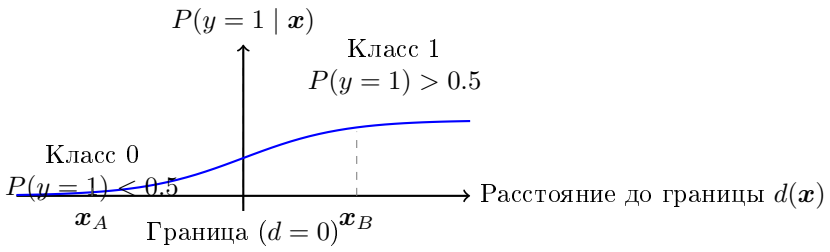


Рис. 2: Зависимость вероятности от расстояния до решающей границы

Геометрия расстояния до решающей границы
Расстояние от точки $\boldsymbol{x}$ до решающей границы $\boldsymbol{w}^\top \boldsymbol{x} + b = 0$:

$$d(\boldsymbol{x}) = \frac{|\boldsymbol{w}^\top \boldsymbol{x} + b|}{\|\boldsymbol{w}\|}$$

Вероятность зависит от **знака и величины** этого расстояния как на Рисунке 2
Линейность в пространстве logit
В пространстве **logit** (log-odds) зависимость становится линейной:

$$\text{logit}(P(y=1 \mid \boldsymbol{x})) = \ln\left(\frac{P(y=1 \mid \boldsymbol{x})}{1 - P(y=1 \mid \boldsymbol{x})}\right) = \boldsymbol{w}^\top \boldsymbol{x} + b$$

Это основное геометрическое свойство: логистическая регрессия предполагает, что **logit вероятностей лежат в линейном подпространстве**, натянутом на признаки.

17.2 Сигмоида как предсказание вероятности, свойства

Функция в правой части называется сигмоидой и обозначается

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Если заметить, что $\sigma(-z) = \frac{1}{1+e^z} = \frac{e^-z}{1+e^{-z}} = 1 - \sigma(z)$, то выражение можно переписать проще:

$$l(w, X, y) = \sum_i y_i \log \sigma((w, x_i)) + (1 - y_i) \log \sigma(-(w, x_i))$$

Нас интересует w, для которого правдоподобие максимально. Чтобы получить функцию потерь, которую мы будем *минимизировать*, умножим его на минус один:

$$L(w, X, y) = -\sum_i y_i \log \sigma((w, x_i)) + (1 - y_i) \log \sigma(-(w, x_i))$$

В отличие от линейной регрессии, для логистической нет явной формулы решения. Деваться некуда, будем использовать градиентный спуск. К счастью, градиент устроен очень просто:

$$\nabla_w L(y, X, w) = \sum_i x_i (y_i - \sigma((w, x_i)))$$

17.3 Формула логистической функции потерь

Нам окажется полезным ещё одно свойство сигмоиды:

$$\frac{d \log \sigma(z)}{dz} = \left(\log\left(\frac{1}{1+e^{-z}}\right)\right)' = \frac{e^{-z}}{1+e^{-z}} = \sigma(-z)$$

$$\frac{d \log \sigma(-z)}{dz} = -\sigma(z)$$

17.3.1 вывод через метод максимального правдоподобия

Отсюда:

$$\nabla_w \log \sigma(\langle w, x_i\rangle) = \sigma(-\langle w, x_i\rangle)x_i$$
$$\nabla_w \log \sigma(-\langle w, x_i\rangle) = -\sigma(\langle w, x_i\rangle)x_i$$

и градиент оказывается равным

$$\begin{aligned}\nabla_w L(y, X, w) &= -\sum_i \big(y_i x_i \sigma(-\langle w, x_i\rangle) - (1-y_i)x_i\sigma(\langle w, x_i\rangle)\big)\\ &= -\sum_i \big(y_i x_i (1-\sigma(\langle w, x_i\rangle)) - (1-y_i)x_i\sigma(\langle w, x_i\rangle)\big)\\ &= -\sum_i \big(y_i x_i - y_i x_i\sigma(\langle w, x_i\rangle) - x_i\sigma(\langle w, x_i\rangle) + y_i x_i\sigma(\langle w, x_i\rangle)\big)\\ &= -\sum_i \big(y_i x_i - x_i\sigma(\langle w, x_i\rangle)\big)\end{aligned}$$

17.4 Свойство калибровки вероятностей

Предсказание модели будет вычисляться, как мы договаривались, следующим образом: $p = \sigma((w,x_i))$. Это вероятность положительного класса, а как от неё перейти к предсказанию самого класса? В других методах нам достаточно было посчитать знак предсказания, но теперь все наши предсказания положительные и находятся в диапазоне от 0 до 1. Что же делать? Интуитивным и не совсем (и даже совсем не) правильным является ответ «взять порог 0,5». Более корректным будет подобрать этот порог отдельно, для уже построенной регрессии минимизируя нужную вам метрику на отложенной тестовой выборке. Например, сделать так, чтобы доля положительных и отрицательных классов примерно совпадала с реальной.

Отдельно заметим, что метод называется логистической *регрессией*, а не логистической *классификацией* именно потому, что предсказываем мы не классы, а вещественные числа – logit.

Определение калибровки

[Калибровка вероятностей] Модель называется **калиброванной**, если предсказанные вероятности отражают истинные частоты событий. Формально, для любого уровня вероятности p:

$$P\left(y = 1 \mid \hat{p} = p\right) = p$$

Здесь $\hat{p}$ — предсказанная моделью вероятность.

Калибровочная кривая (Reliability Diagram)

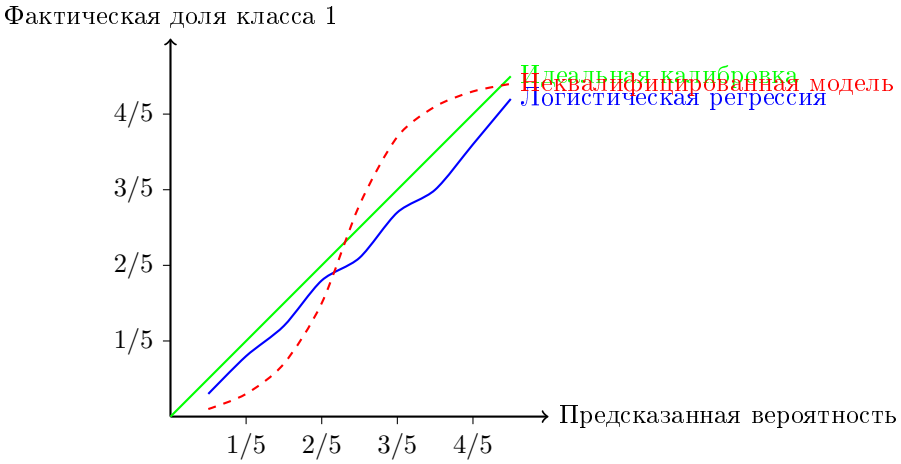
Калибровочная кривая изображена на Рисунке 3

17.5 Обобщённая линейная модель

Определение и структура GLM

[Обобщённая линейная модель] GLM состоит из трёх компонентов

1. **Случайная компонента:** y_i независимы и имеют распределение из **экспоненциального семейства**:
$$f(y_i \mid \theta_i, \phi) = \exp\left\{\frac{y_i\theta_i - b(\theta_i)}{a(\phi)} + c(y_i, \phi)\right\}$$
Здесь θ_i — natural parameter, ϕ — dispersion parameter.
2. **Систематическая компонента:** линейный предиктор
$$\eta_i = \boldsymbol{w}^\top \boldsymbol{x}_i$$
3. **Функция связи (link function):** Связывает математическое ожидание $\mu_i = \mathbb{E}[y_i]$ с линейным предиктором:
$$g(\mu_i) = \eta_i = \boldsymbol{w}^\top \boldsymbol{x}_i$$



Калибровочная кривая показывает соответствие между предсказанными и фактическими вероятностями

Рис. 3: Калибровочные кривые для разных моделей

Логистическая регрессия как частный случай GLM

Логистическая регрессия в терминах GLM. 1. **Распределение:** Бернулли $y_i \sim \text{Bernoulli}(p_i)$

$$f(y_i \mid p_i) = p_i^{y_i}(1-p_i)^{1-y_i} = \exp\left\{ y_i \ln\left(\frac{p_i}{1-p_i}\right) +\right.$$

1. $\ln(1-p_i)$

   Сравнивая с канонической формой экспоненциального семейства

   - Natural parameter: $\theta_i = \ln\left(\frac{p_i}{1-p_i}\right)$
   - $b(\theta_i) = \ln\left(1+e^{\theta_i}\right) = -\ln(1-p_i)$
   - $a(\phi) = 1,\ \phi = 1$
   - $c(y_i, \phi) = 0$

2. **Функция связи:** каноническая функция связи для Бернулли:

$$g(\mu_i) = \ln\left(\frac{\mu_i}{1-\mu_i}\right) = \text{logit}(\mu_i)$$

   где $\mu_i = \mathbb{E}[y_i] = p_i$

3. **Линейный предиктор**

$$\eta_i = \boldsymbol{w}^\top \boldsymbol{x}_i$$

   Таким образом, $\text{logit}(p_i) = \boldsymbol{w}^\top \boldsymbol{x}_i$.

□

Другие примеры GLM отображены в Таблице 6

Модель регрессии	**Распределение**	**Каноническая связь**	**Область применения**
Линейная	Нормальное	Identity: $g(\mu) = \mu$	Непрерывная отклик
Логистическая	Бернулли	Logit: $g(\mu) = \ln\frac{\mu}{1-\mu}$	Бинарная классификация
Пуассоновская	Пуассон	Log: $g(\mu) = \ln\mu$	Подсчёты (count data)
Gamma	Gamma	Inverse: $g(\mu) = 1/\mu$	Положительные непрерывные

Таблица 6: Примеры обобщённых линейных моделей

Оценка параметров в GLM
Метод максимального правдоподобия
Функция правдоподобия для GLM:

$$L(\boldsymbol{w}) = \prod_{i=1}^{n} f(y_i \mid \theta_i(\boldsymbol{w}), \phi)$$

Логарифмическое правдоподобие:

$$\ell(\boldsymbol{w}) = \sum_{i=1}^{n} \left[\frac{y_i \theta_i(\boldsymbol{w}) - b(\theta_i(\boldsymbol{w}))}{a(\phi)} + c(y_i, \phi) \right]$$

Уравнения правдоподобия
Оценки MLE находятся из уравнений:

$$\sum_{i=1}^{n} \frac{(y_i - \mu_i) x_{ij}}{\mathrm{Var}(y_i)} \cdot \frac{\partial \mu_i}{\partial \eta_i} = 0, \quad j = 1, \ldots, d$$

Алгоритм IRLS (Iteratively Reweighted Least Squares)
Для GLM параметры оцениваются итеративно:
Преимущества и ограничения GLM
Преимущества.

1. **Единая теоретическая основа:** все модели из экспоненциального семейства.
2. **Интерпретируемость:** коэффициенты имеют чёткую интерпретацию
3. **Статистические свойства:** известны асимптотические распределения оценок
4. **Гибкость:** можно моделировать различные типы данных (бинарные, счётные, непрерывные)

Ограничения

1. **Линейность по параметрам:** предиктор должен быть линейной комбинацией признаков
2. **Предположение о распределении:** данные должны соответствовать экспоненциальному семейству
3. **Отсутствие взаимодействий:** сложно моделировать сложные взаимодействия без явного задания
4. **Чувствительность к выбросам:** как и другие методы MLE

Расширения GLM

1. **GLM с регуляризацией:** Lasso, Ridge, Elastic Net для GLM
2. **Generalized Additive Models (GAM):** замена линейного предиктора на аддитивный:

$$g(\mu_i) = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip})$$

3. **GLMM (Generalized Linear Mixed Models):** добавление случайных эффектов
4. **Полиномиальная логистическая регрессия:** обобщение на многоклассовый случай

18 Стратегии многоклассовой классификации

В этом разделе мы будем следовать изложению Евгению Соколова.

Пусть каждый объект нашей выборки относится к одному из K классов: $Y = 1, \ldots, K$. Чтобы предсказывать эти классы с помощью линейных моделей, нам придётся свести задачу многоклассовой классификации к набору бинарных, которые мы уже хорошо умеем решать. Мы разберём два самых популярных способа это сделать – one-vs-all и all-vs-all, а проиллюстрировать их нам поможет вот такой игрушечный датасет

18.1 One-vs-one, one-vs-all и их свойства

Один против всех (one-versus-all)

Обучим K линейных классификаторов $b_1(x), \ldots, b_K(x)$, выдающих оценки принадлежности классам $1, \ldots, K$ соответственно. В случае с линейными моделями эти классификаторы будут иметь вид $b_k(x) = sgn((w_k, x) + w_{0k})$

Классификатор с номером k будем обучать по выборке $(x + i, 2\mathbb{I}[y_i = k] - 1)_{i=1}^N$. Иными словами, мы учим классификатор отличать k-й класс от всех остальных.

Логично, чтобы итоговый классификатор выдавал класс, соответствующий самому уверенному из бинарных алгоритмов. Уверенность можно в каком-то смысле измерить с помощью значений линейных функций: $a(x) = argmax_k((w_k, x) + w_{0k})$

Давайте посмотрим, что даст этот подход применительно к нашему датасету. Обучим три линейных модели, отличающих один класс от остальных. Затем сравним значения линейных функций и для каждой точки выберем тот класс, которому соответствует большее значение, то есть самый «уверенный» классификатор:

Хочется сказать, что самый маленький класс «обидели».

Проблема данного подхода заключается в том, что каждый из классификаторов $b_1(x), \ldots, b_K(x)$ обучается на своей выборке, и значения линейных функций $(w_k, x) + w_{0k}$ или, проще говоря, "выходы" классификаторов могут иметь разные масштабы. Из-за этого сравнивать их будет неправильно. Нормировать вектора весов, чтобы они выдавали ответы в одной и той же шкале, не всегда может быть разумным решением: так, в случае с SVM веса перестанут являться решением задачи, поскольку нормировка изменит норму весов.

Все против всех (all-versus-all)

Обучим $C_K^2 a_{ij}(x); i,j = 1, \ldots, K, i$. Например, в случае с линейными моделями эти модели будут иметь вид $b_{ij}(x) = sgn((w_{ij}, x) + w_{0ij})$

Классификатор $a_{ij}(x)$ будем настраивать по $X_{ij} \subset X$, содержащей только объекты классов i и j. Соответственно, классификатор $a_{ij}(x)$ будет выдавать для любого объекта либо класс i, либо класс j.

Чтобы классифицировать новый объект, подадим его на вход каждого из построенных бинарных классификаторов. Каждый из них проголосует за свой класс; в качестве ответа выберем тот класс, за который наберется больше всего голосов $a(x) = argmax_k \sum_{i=1}^K \sum_{j \neq i} \mathbb{I}[a_{ij}(x) = k]$

Для нашего датасета получается следующая картинка. Обратите внимание на серый треугольник на стыке областей. Это точки, для которых голоса разделились (в данном случае каждый классификатор выдал какой-то свой класс, то есть у каждого класса было по одному голосу). Для этих точек нет явного способа выдать обоснованное предсказание.

19 Метрики классификации

Задачи машинного обучения с учителем, как правило, состоят в восстановлении зависимости между парами (признаковое описание, целевая переменная) по данным, доступным нам для анализа. Алгоритмы машинного обучения (learning algorithm), со многими из которых вы уже успели познакомиться, позволяют построить модель, аппроксимирующую эту зависимость. Но как понять, насколько качественной получилась аппроксимация?

Почти наверняка наша модель будет ошибаться на некоторых объектах: будь она даже идеальной, шум или выбросы в тестовых данных всё испортят. При этом разные модели будут ошибаться на разных объектах и в разной степени. Задача специалиста по машинному обучению – подобрать подходящий критерий, который позволит сравнивать различные модели.

Важно: качество модели нельзя оценивать на обучающей выборке. Как минимум, это стоит делать на отложенной (тестовой) выборке, но если вам это позволяют время и вычислительные ресурсы, стоит прибегнуть и к более надёжным способам проверки — например, *кросс-валидации*

Выбор метрик в реальных задачах Возможно, вы уже участвовали в соревнованиях по анализу данных. На таких соревнованиях метрику (критерий качества модели) организатор выбирает за вас, и она, как правило, довольно понятным образом связана с результатами предсказаний. Но на практике всё бывает намного сложнее. Варианты наших действий

- Решить, сколько коробок с бананами нужно завтра привезти в конкретный магазин – чтобы предложение соответствовало спросу, и не пришлось выбрасывать излишки;
- увеличить счастье пользователя от работы с сервисом, чтобы он стал лояльным и приносил стабильный прогнозируемый доход;
- решить, нужно ли направить человека на дополнительное обследование.

В каждом конкретном случае может возникать целая иерархия метрик. Представим, например, что речь идёт о стриминговом музыкальном сервисе, пользователей которого мы решили порадовать сгенерированными самодельной нейросетью треками – не защищёнными авторским правом, а потому совершенно бесплатными.

Иерархия метрик

1. Самый верхний уровень: будущий доход сервиса – невозможно измерить в моменте, сложным образом зависит от совокупности всех наших усилий.
2. Медианная длина сессии, возможно, служащая оценкой радости пользователей, которая, как мы надеемся, повлияет на их желание продолжать платить за подписку – её нам придётся измерять в продакшене, ведь нас интересует реакция настоящих пользователей на новшество.
3. Доля удовлетворённых качеством сгенерированной музыки асессоров, на которых мы потестируем её до того, как выставить на суд пользователей.
4. Функция потерь, на которую мы будем обучать генеративную сеть.

На этом примере мы можем заметить сразу несколько общих закономерностей. Во-первых, метрики бывают offline и online. Online-метрики вычисляются по данным, собираемым с работающей системы (например, медианная длина сессии). Offline-метрики могут быть измерены до введения модели в эксплуатацию, например, по историческим данным или с привлечением специальных людей, асессоров.

Последнее часто применяется, когда метрика – это реакция живого человека: скажем, так поступают поисковые компании, которые предлагают людям оценить качество ранжирования экспериментальной системы ещё до того, как рядовые пользователи увидят эти результаты в обычном порядке. На самом же нижнем этаже иерархии лежат оптимизируемые в ходе обучения функции потерь.

В данном разделе нас будут интересовать offline-метрики, которые могут быть измерены без привлечения людей

19.1 Определение метрики в ML

Метрика в машинном обучении – это показатель, который количественно характеризует качество и свойство компьютерного алгоритма

19.1.1 Свойства, сравнение с функцией потерь

Свойства метрик могут включать простоту интерпретации, универсальность, учёт дисбаланса классов и другие. Важно помнить, что ни одна метрика не идеальна и профессиональный подход требует рассмотрения комбинации метрик, а также их соотнесения с бизнес-требованиями

Функция потерь не есть метрика качества Как мы узнали ранее, методы обучения реализуют разные подходы

- Обучение на основе прироста информации (как в деревьях решений);
- обучение на основе сходства (как в методах ближайших соседей);
- обучение на основе вероятностной модели данных (например, максимизацией правдоподобия);
- обучение на основе ошибок (минимизация эмпирического риска).

И в рамках обучения на основе минимизации ошибок мы уже отвечали на вопрос: как можно штрафовать модель за предсказание на обучающем объекте.

Во время сведения задачи о построении решающего правила к задаче численной оптимизации, мы вводили понятие функции потерь и, обычно, объявляли целевой функцией сумму потерь от предсказаний на всех объектах обучающей выборки.

Важно понимать разницу между функцией потерь и метрикой качества. Её можно сформулировать следующим образом:

- Функция потерь возникает в тот момент, когда мы сводим задачу построения модели к задаче оптимизации. Обычно требуется, чтобы она обладала хорошими свойствами (например, дифференцируемостью).
- Метрика — внешний, объективный критерий качества, обычно зависящий не от параметров модели, а только от предсказанных меток.

В некоторых случаях метрика может совпадать с функцией потерь. Например, в задаче регрессии MSE играют роль как функции потерь, так и метрики. Но, скажем, в задаче бинарной классификации они почти всегда различаются: в качестве функции потерь может выступать кросс-энтропия, а в качестве метрики – число верно угаданных меток (accuracy). Отметим, что в последнем примере у них различные аргументы: на вход кросс-энтропии нужно подавать logit, а на вход accuracy — предсказанные метки (то есть по сути argmax logit).

Начнём с самой простой разновидности классификации – бинарной, а затем постепенно будем наращивать сложность.

Напомним постановку задачи бинарной классификации: нам нужно по обучающей выборке $(x_i,y_i)_{i=1}^N$, где $y_i \in 0,1$ построить модель, которая по объекту x предсказывает метку класса $f(x) \in 0,1$.

Первый критерий качества, который приходит в голову, – accuracy, то есть доля объектов с правильным предсказанием класса:

$$Accuracy(y,y_{pred}) = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}[y_i = f(x_i)]$$

Или же сопряжённая ей метрика – доля ошибочных классификаций (error rate): Error rate $= 1$ Accuracy

Познакомившись чуть внимательнее с этой метрикой, можно заметить, что у неё есть несколько недостатков

- Она не учитывает дисбаланс классов. Например, в задаче диагностики редких заболеваний классификатор, предсказывающий всем пациентам отсутствие болезни будет иметь достаточно высокую accuracy просто потому, что больных людей в выборке намного меньше;
- она также не учитывает цену ошибки на объектах разных классов. Для примера снова можно привести задачу медицинской диагностики: если ошибочный положительный диагноз для здорового больного обернётся лишь ещё одним обследованием, то ошибочно отрицательный вердикт может повлечь роковые последствия.

19.2 Матрица ошибок (бинарная, многоклассовая)

Исторически задача бинарной классификации – это задача об обнаружении чего-то редкого в большом потоке объектов, например, поиск человека, больного туберкулёзом, по флюорографии. Или задача признания пятна на экране приёмника радиолокационной станции бомбардировщиком, представляющем угрозу охраняемому объекту (в противовес стае гусей).

Для задачи классификации с C классами, **матрица ошибок** (confusion matrix) $\mathbf{M}$ размера $C \times C$ определяется как:

$$\mathbf{M} = [m_{ij}]_{i,j=1}^{C}$$

где:

- m_{ij} — количество объектов класса i, отнесённых алгоритмом к классу j
- i — индекс истинного класса (строка)
- j — индекс предсказанного класса (столбец)

Пример для 3 классов

Истина \ Предсказание	**Предсказанный класс**			**Всего**
	Класс 1	Класс 2	Класс 3	
Класс 1	m_{11} (TP_1)	m_{12}	m_{13}	N_1
Класс 2	m_{21}	m_{22} (TP_2)	m_{23}	N_2
Класс 3	m_{31}	m_{32}	m_{33} (TP_3)	N_3
Всего	P_1	P_2	P_3	N

Таблица 7: Матрица ошибок для трёх классов

Вычисление основных показателей для класса c Для каждого класса c можно вычислить:

$$\mathrm{TP}_c = m_{cc} \quad \text{(True Positives для класса } c\text{)}.$$
$$\mathrm{FP}_c = \sum_{i \neq c} m_{ic} \quad \text{(False Positives для класса } c\text{)}.$$
$$\mathrm{FN}_c = \sum_{j \neq c} m_{cj} \quad \text{(False Negatives для класса } c\text{)}.$$
$$\mathrm{TN}_c = \sum_{i \neq c} \sum_{j \neq c} m_{ij} \quad \text{(True Negatives для класса } c\text{)}$$

19.2.1 Типы ошибок

Класс, который представляет для нас интерес, называется «положительным», а оставшийся — «отрицательным». Заметим, что для каждого объекта в выборке возможно 4 ситуации

- Мы предсказали *положительную* метку и *угадали*. Будет относить такие объекты к true positive (TP) группе. True — потому что предсказали мы правильно, а positive — потому что предсказали положительную метку;
- мы предсказали *положительную* метку, но *ошиблись* в своём предсказании – false positive (FP). False, потому что предсказание было неправильным;
- мы предсказали *отрицательную* метку и *угадали* — true negative (TN);
- и наконец, мы предсказали *отрицательную* метку, но *ошиблись* – false negative (FN). Для удобства все эти 4 числа изображают в виде таблицы, которую называют confusion matrix (матрицей ошибок):

В многоклассовой задаче каждый класс рассматривается как "положительный", а все остальные вместе как "отрицательный":

- **True Positive для класса** c: Объект класса c правильно классифицирован как c
- **False Positive для класса** c: Объект не из класса c ошибочно классифицирован как c
- **False Negative для класса** c: Объект класса c ошибочно классифицирован как другой класс
- **True Negative для класса** c: Объект не из класса c правильно классифицирован как не c

19.2.2 Правило присвоения меток

1. **Пороговое правило для вероятностей:** Для бинарной классификации с порогом τ:
$$\hat{y}_i = \begin{cases} 1, & \text{если } \hat{p}_i \geq \tau \\ 0, & \text{если } \hat{p}_i < \tau \end{cases}$$
2. **Argmax для многоклассовой:**
$$\hat{y}_i = \arg \max_{c \in \{1,\ldots,C\}} \hat{p}_{ic}$$
где $\hat{p}_{ic}$ — предсказанная вероятность принадлежности к классу c
3. **Однопроходное правило:**
$$\hat{y}_i = \begin{cases} 1, & \text{если } f(\boldsymbol{x}_i) \geq 0 \\ 0, & \text{если } f(\boldsymbol{x}_i) < 0 \end{cases}$$
для классификатора $f : \mathbb{R}^d \to \mathbb{R}$

Стратегии для несбалансированных данных

- **Настройка порога:** смещение порога в сторону меньшего класса
$$\tau_{\text{adjusted}} = \frac{N_{\text{negative}}}{N_{\text{positive}} + N_{\text{negative}}}$$

- **Разные пороги для разных классов:**

$$\hat{y}_i = c \quad \text{если} \quad \frac{\hat{p}_{ic}}{\tau_c} > \frac{\hat{p}_{ij}}{\tau_j} \ \forall j \neq c$$

- **Правило с учётом стоимости ошибок:**

$$\hat{y}_i = \arg\min_c \sum_{j=1}^{C} L(j,c) \cdot \hat{p}_{ij}$$

где $L(j,c)$ — стоимость ошибки классификации класса j как класса c

19.3 Accuracy, balanced accuracy, precision, recall, F1-score

Accuracy был выше.
[Balanced Accuracy] Для бинарной классификации:

$$\text{Balanced Accuracy} = \frac{1}{2}\left(\frac{TP}{TP+FN} + \frac{TN}{TN+FP}\right) = \frac{\text{Sensitivity} + \text{Specificity}}{2}$$

Для многоклассовой классификации:

$$\text{Balanced Accuracy} = \frac{1}{C}\sum_{c=1}^{C}\frac{TP_c}{TP_c + FN_c}$$

Преимущества

- Устойчива к дисбалансу классов.
- Даёт равный вес каждому классу
- Лучше отражает качество на несбалансированных данных

В случае асимметрии классов, можно использовать метрики, которые не учитывают TN и ориентируются на TP. Если мы рассмотрим долю правильно предсказанных положительных объектов среди всех объектов, предсказанных положительным классом, то мы получим метрику, которая называется точностью (precision)

$$Precision = \frac{TP}{TP+FP}$$

Интуитивно метрика показывает долю релевантных документов среди всех найденных классификатором. Чем меньше ложноположительных срабатываний будет допускать модель, тем больше будет её Precision.

Если же мы рассмотрим долю правильно найденных положительных объектов среди всех объектов положительного класса, то мы получим метрику, которая называется полнотой (recall)

$$Recall = \frac{TP}{TP+FN}$$

Интуитивно метрика показывает долю найденных документов из всех релевантных. Чем меньше ложно отрицательных срабатываний, тем выше recall модели.

Например, в задаче предсказания злокачественности опухоли точность показывает, сколько из определённых нами как злокачественные, действительно таковые, а полнота – какую долю злокачественных опухолей нам удалось выявить.

F_1-мера

Как мы уже отмечали ранее, модели очень удобно сравнивать, когда их качество выражено одним числом. В случае пары Precision-Recall существует популярный способ скомпоновать их в одну метрику – взять их среднее гармоническое. Данный показатель эффективности исторически носит название F_1-меры (F_1-measure).

$$F_1 = \frac{2}{\frac{1}{Recall} + \frac{1}{Precision}}$$

Стоит иметь в виду, что F_1-мера предполагает одинаковую важность Precision и Recall, если одна из этих метрик для вас приоритетнее, то можно воспользоваться F_β мерой:

$$F_1 = (\beta^2+1)\frac{Recall\dot{P}recision}{Recall + \beta^2 Precision}$$

19.3.1 Sensitivity, specificity и другие

Sensitivity = recall. В медицинских приложениях:

- **Sensitivity (Чувствительность):** Вероятность того, что тест будет положительным при наличии заболевания
$$\text{Sensitivity} = P(\text{Тест}+ \mid \text{Болезнь}+) = \frac{TP}{TP + FN}$$
- **Specificity (Специфичность):** Вероятность того, что тест будет отрицательным при отсутствии заболевания или доля найденных отрицательных в общём объёме отрицательных
$$\text{Specificity} = P(\text{Тест}- \mid \text{Болезнь}-) = \frac{TN}{TN + FP}$$

Расширенные метрики

1. **Matthews Correlation Coefficient (MCC):**
$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$
Корреляция между истинными и предсказанными метками
2. **Cohen's Kappa** Согласованность классификаций с учётом случайного угадывания
$$\kappa = \frac{p_o - p_e}{1 - p_e}$$
где p_o — наблюдаемая точность, p_e — ожидаемая точность при случайном угадывании

19.3.2 Несбалансированная стоимость ошибок, примеры, как исправлять дисбаланс

Матрица стоимости ошибок
Для задачи с C классами задаётся матрица стоимости $\mathbf{L} \in \mathbb{R}^{C \times C}$:

$$\mathbf{L} = [L_{ij}]_{i,j=1}^{C}$$

Здесь L_{ij} — стоимость классификации объекта класса i как класса j.
Свойства:

- $L_{ii} = 0$ (правильная классификация бесплатна)
- $L_{ij} > 0$ для $i \neq j$ (ошибки имеют положительную стоимость)
- Часто $L_{ij} \neq L_{ji}$ (стоимость ошибок несимметрична)

Примеры из реальных задач

Задача	Дорогая ошибка	Менее дорогая ошибка
Медицинская диагностика	Пропустить болезнь (FN)	Ложная тревога (FP)
Спам-фильтрация	Пропустить спам (FN)	Пометить не-спам как спам (FP)
Кредитный scoring	Выдать кредит неплательщику (FP)	Отказать плательщику (FN)
Автономное вождение	Не заметить пешехода (FN)	Ложное срабатывание тормозов (FP)

Таблица 8: Примеры несбалансированной стоимости ошибок

Способы учёта различной стоимости ошибок
1. Взвешенная функция потерь
Модификация функции потерь с весами:

$$\mathcal{L}_{\text{weighted}}(\boldsymbol{w}) = \sum_{i=1}^{n} \alpha_{y_i} \cdot \ell(y_i, f(\boldsymbol{x}_i; \boldsymbol{w}))$$

Здесь α_c — вес для класса c, обычно $\alpha_c \propto \frac{1}{N_c}$.

2. Изменение порога принятия решения

Для бинарной классификации с разными стоимостями C_{FP} и C_{FN} оптимальный порог:

$$\tau^* = \frac{C_{FP}}{C_{FP} + C_{FN}}$$

При условии калиброванных вероятностей.

3. Cost-sensitive learning

Модификация алгоритмов для минимизации общей стоимости:

$$\hat{\boldsymbol{w}} = \arg\min_{\boldsymbol{w}} \sum_{i=1}^{n} \sum_{j=1}^{C} L(y_i, j) \cdot \mathbb{I}[\hat{y}_i = j]$$

4. Методы выборки (sampling)

- **Oversampling:** дублирование примеров миноритарного класса
- **Undersampling:** удаление примеров мажоритарного класса
- **SMOTE (Synthetic Minority Oversampling):** генерация синтетических примеров миноритарного класса

Как исправлять дисбаланс классов

На уровне данных

1. **Стратифицированная выборка:** сохранение пропорций классов при разбиении.
2. **Взвешенная случайная выборка:** выборка с вероятностями, обратно пропорциональными размеру класса.
3. **Генеративные методы:**
   - SMOTE: создание синтетических примеров миноритарного класса
   - ADASYN: адаптивный вариант SMOTE
   - генеративно-состязательные сети (GAN)

На уровне алгоритма

1. **Взвешенная функция потерь:** присвоение больших весов ошибкам на миноритарном классе
2. **Cost-sensitive learning:** явное задание матрицы стоимости ошибок
3. **Ансамблирование:** использование методов вроде EasyEnsemble или BalanceCascade

На уровне оценки

1. Использование метрик, устойчивых к дисбалансу:
   - Balanced Accuracy
   - F1-score, Fβ-score
   - MCC, Cohen's Kappa
   - AUC-ROC, AUC-PR
2. Стратифицированная кросс-валидация

Практические рекомендации

Соотношение классов	Рекомендуемый подход	Метрики
Небольшой дисбаланс (1:3)	Взвешенная функция потерь	F1-score, Balanced Accuracy
Средний дисбаланс (1:10)	SMOTE + взвешенная loss	AUC-PR, MCC
Сильный дисбаланс (1:1000+)	One-class classification	Precision, Specificity

Таблица 9: Рекомендации по работе с различными уровнями дисбаланса

19.4 ROC AUC

Пусть мы хотим учитывать ошибки на объектах обоих классов. При уменьшении порога отсечения мы будем находить (правильно предсказывать) всё большее число положительных объектов, но также и неправильно предсказывать положительную метку на всё большем числе отрицательных объектов. Естественным, кажется, ввести две метрики TPR и FPR:

TPR (true positive rate) – это полнота, доля положительных объектов, правильно предсказанных положительными:

$$TPR = \frac{TP}{P} = \frac{TP}{TP + FN}$$

FPR (false positive rate) – это доля отрицательных объектов, неправильно предсказанных положительными:

$$FPR = \frac{FP}{N} = \frac{FP}{FP + TN}$$

Обе эти величины растут при уменьшении порога. Кривая в осях TPR/FPR, которая получается при варьировании порога, исторически называется ROC-кривой (receiver operating characteristics curve, сокращённо ROC curve).

Жёлтая и синяя кривые показывают распределение предсказаний классификатора на объектах положительного и отрицательного классов соответственно. То есть значения на оси X (на графике с двумя гауссианами) мы получаем из классификатора.

- Если классификатор идеальный, – две кривые разделимы по оси X, – то на правом графике мы получаем ROC-кривую (0,0)$->$(0,1)$->$(1,1), площадь под которой равна 1.
- Если классификатор случайный (предсказывает одинаковые метки положительным и отрицательным объектам), то мы получаем ROC-кривую (0,0)$->$(1,1), площадь под которой равна 0.5.

19.4.1 Свойства ROC и ROC-AUC

Чем лучше классификатор разделяет два класса, тем больше площадь (*area under curve*) под ROC-кривой – и мы можем использовать её в качестве метрики. Эта метрика называется AUC и она работает благодаря следующему свойству ROC-кривой:

AUC равен доле пар объектов вида (объект класса 1, объект класса 0), которые алгоритм верно упорядочил, то есть предсказание классификатора на первом объекте больше:

$$AUC = \frac{\sum_{i=1}^{N}\sum_{j=1}^{N} I[y_i < y_j] \cdot I'[f(x_i) < f(x_j)]}{\sum_{i=1}^{N}\sum_{j=1}^{N} I[y_i < y_j]}$$

$$I'[f(x_i) < f(x_j)] = \begin{cases} 0, & f(x_i) > f(x_j) \\ 0.5, & f(x_i) = f(x_j) \\ 1, & f(x_i) < f(x_j) \end{cases}$$

$$I[y_i < y_j] = \begin{cases} 0, & y_i \geq y_j \\ 1, & y_i < y_j \end{cases}$$

В каких случаях лучше отдать предпочтение этой метрике? Рассмотрим следующую задачу: некоторый сотовый оператор хочет научиться предсказывать, будет ли клиент пользоваться его услугами через месяц. На первый взгляд кажется, что задача сводится к бинарной классификации с метками 1, если клиент останется с компанией и 0 если иначе.

Однако если копнуть глубже в процессы компании, то окажется, что такие метки практически бесполезны. Компании скорее интересно упорядочить клиентов по вероятности прекращения обслуживания и

в зависимости от этого применять разные варианты удержания: кому-то прислать скидочный купон от партнёра, кому-то предложить скидку на следующий месяц, а кому-то и новый тариф на особых условиях.

Таким образом, в любой задаче, где нам важна не метка сама по себе, а правильный порядок на объектах, имеет смысл применять AUC. Утверждение выше может вызывать у вас желание использовать AUC в качестве метрики в задачах ранжирования, но мы призываем вас быть аккуратными.

19.4.2 PR AUC

[PR кривая] Кривая Precision-Recall строится путём варьирования порога классификации τ и вычисления:

- Precision (точность) на оси Y
- Recall (полнота) на оси X

Для каждого порога τ:

$$\text{Precision}(\tau) = \frac{TP(\tau)}{TP(\tau) + FP(\tau)}, \quad \text{Recall}(\tau) = \frac{TP(\tau)}{TP(\tau) + FN(\tau)}$$

PR AUC vs ROC AUC

Характеристика.	**PR AUC**	**ROC AUC**
Чувствительность к дисбалансу	Высокая	Низкая
Интерпретация при дисбалансе	Более информативна	Может быть обманчивой
Оси	Precision (Y) vs Recall (X)	TPR (Y) vs FPR (X)
Базовая линия	Положительная доля	Диагональ y = x
Подходит для	Сильно несбалансированных данных	Умеренного дисбаланса

Таблица 10: Сравнение PR AUC и ROC AUC

Вычисление PR AUC

1. Точное вычисление (по всем порогам)

[1] Получить предсказанные вероятности $\hat{p}_i$ для всех объектов. Отсортировать объекты по убыванию $\hat{p}_i$ Для каждого объекта в порядке убывания вероятности: Рассмотреть его вероятность как порог τ. Вычислить Precision и Recall при этом пороге. Вычислить площадь под ломаной кривой (правило трапеций)

2. Формула через ранги

Для бинарной классификации PR AUC можно вычислить как:

$$\text{PR AUC} = \frac{\sum_{i=1}^{n_+} \frac{i}{r_i}}{n_+}$$

где:

- n_+ — количество положительных объектов
- r_i — ранг i-го положительного объекта после сортировки по убыванию вероятности

Интерпретация PR AUC

- **PR AUC = 1:** идеальный классификатор
- **PR AUC = положительная доля:** случайный классификатор
- **PR AUC близко к 0:** классификатор хуже случайного

Важно: Базовый уровень (случайный классификатор) для PR кривой — горизонтальная линия на уровне положительной доли:

$$\text{Precision}_{\text{random}} = \frac{n_+}{n} = P$$

Средние метрики для многоклассовой классификации

1. **Macro-average PR AUC:**

$$\text{Macro PR AUC} = \frac{1}{C}\sum_{c=1}^{C}\text{PR AUC}_c$$

Здесь PR AUC_c вычисляется для класса c против всех остальных

2. **Micro-average PR AUC:** Объединение предсказаний для всех классов и вычисление общей PR кривой

3. **Weighted-average PR AUC:**

$$\text{Weighted PR AUC} = \sum_{c=1}^{C} w_c \cdot \text{PR AUC}_c, \quad w_c = \frac{N_c}{N}$$

Преимущества PR AUC для несбалансированных данных

1. **Чувствительность к изменениям в миноритарном классе:** небольшие изменения в предсказаниях миноритарного класса сильно влияют на PR кривую

2. **Наглядность trade-off:** ясно показывает компромисс между точностью и полнотой

3. **Информативность при high-recall:** важна в задачах, где нужно найти как можно больше положительных примеров (медицина, безопасность)

4. **Устойчивость к изменению распределения:** PR кривая зависит только от ранжирования, а не от абсолютных значений вероятностей

19.5 Многоклассовые расширения

Если классов становится больше двух, расчёт метрик усложняется. Если задача классификации на K классов ставится как K задач об отделении класса i от остальных ($i = 1, \ldots, K$), то для каждой из них можно посчитать свою матрицу ошибок. Затем есть два варианта получения итогового значения метрики из K матриц ошибок:

19.5.1 Микро- и макро-усреднение

1. Усредняем элементы матрицы ошибок (TP, FP, TN, FN) между бинарными классификаторами, например $TP = \frac{1}{K}\sum_{i=1}^{K} TP_i$. Затем по одной усреднённой матрице ошибок считаем Precision, Recall, F-меру. Это называют микроусреднением.

2. Считаем Precision, Recall для каждого классификатора отдельно, а потом усредняем. Это называют макроусреднением.

Порядок усреднения влияет на результат в случае дисбаланса классов. Показатели TP, FP, FN – это счётчики объектов. Пусть некоторый класс обладает маленькой мощностью (обозначим её MM). Тогда значения TP и FN при классификации этого класса против остальных будут не больше MM, то есть тоже маленькие. Про FP мы ничего уверенно сказать не можем, но скорее всего при дисбалансе классов классификатор не будет предсказывать редкий класс слишком часто, потому что есть большая вероятность ошибиться. Так что FP тоже мало. Поэтому усреднение первым способом сделает вклад маленького класса в общую метрику незаметным. А при усреднении вторым способом среднее считается уже для нормированных величин, так что вклад каждого класса будет одинаковым.

Рассмотрим пример. Пусть есть датасет из объектов трёх цветов: жёлтого, зелёного и синего. Жёлтого и зелёного цветов поровну – 21 и 20 объектов соответственно, а синих объектов всего 4.

Модель по очереди для каждого цвета пытается отделить объекты этого цвета от объектов оставшихся двух цветов. Результаты классификации проиллюстрированы матрицей ошибок. Модель «покрасила» в жёлтый 25 объектов, 20 из которых были действительно жёлтыми (левый столбец матрицы). В синий был «покрашен» только один объект, который на самом деле жёлтый (средний столбец матрицы). В зелёный – 19 объектов, все на самом деле зелёные (правый столбец матрицы). Макроусреднение лучше отражает тот факт, что синий цвет, которого в датасете было совсем мало, модель практически игнорирует.

20 Матричные нормы

For many cases in math we need to measure matrices, so we introduce the norm term for a matrix

1. Nonnegativity: $\|A\| \geq 0$
2. Definiteness: $\|A\| = 0 \iff A = 0_{m,n}$
3. Absolute homogeneity: $\|\alpha A\| = |\alpha| \|A\|$
4. Triangle inequality: $\|A + B\| \leq \|A\| + \|B\|$
5. Submultiplicativity: $\|AB\| \leq \|A\| \|B\|$

20.1 p-нормы, самые важные частные случаи

Especially useful are vector-induced norms, they are defined as

$$\|A\|_p = \sup_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p} = \sup_{\|x\|_p = 1} \|Ax\|_p$$

Some corner cases:

- $p = 2$: spectral norm
- $p = 1$: maximum absolute column sum norm
- $p = \infty$: maximum absolute row sum norm
- $p = 0$: maximum number of nonzero entries per column

$$\|A\|_2 = \sup_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \sigma_{\max}(A)$$

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^{m} |a_{ij}|$$

$$\|A\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^{n} |a_{ij}|$$

$$\|A\|_0 = \max_{1 \leq j \leq n} \sum_{i=1}^{m} \mathbf{1}_{\{a_{ij} \neq 0\}}$$

21 Метод опорных векторов (SVM)

Иногда возникает логичное желание не только найти разделяющую прямую, но и постараться провести её на одинаковом удалении от обоих классов, то есть максимизировать минимальный отступ

21.1 Геометрическая интерпретация

Это можно сделать, слегка поменяв функцию ошибки, а именно положив её равной:

$$F(M) = \max(0, 1 - M)$$

$$L(w, x, y) = \lambda \|w\|_2^2 + \sum_i \max(0, 1 - y_i \langle w, x_i \rangle)$$

$$\nabla_w L(w, x, y) = 2\lambda w + \sum_i \begin{cases} 0, & 1 - y_i \langle w, x_i \rangle \leq 0 \\ -y_i x_i, & 1 - y_i \langle w, x_i \rangle > 0 \end{cases}$$

Почему же добавленная единичка приводит к желаемому результату? Интуитивно это можно объяснить так: объекты, которые проклассифицированы правильно, но не очень "уверенно" (то есть $0 \leq y_i(w,x_i) < 1$), продолжают вносить свой вклад в градиент и пытаются "отодвинуть" от себя разделяющую плоскость как можно дальше.

21.2 Оптимизационная задача для линейно разделимого случая

К данному выводу можно прийти и чуть более строго; для этого надо совершенно по-другому взглянуть на выражение, которое мы минимизируем. Если мы максимизируем минимальный отступ, то надо максимизировать $\frac{2}{|w|_2}$, то есть ширину полосы при условии, что большинство объектов лежат с правильной стороны, что эквивалентно решению исходной задачи:

$$\lambda|w|_2^2 + \sum_i \max(0, 1 - y_i\langle w, x_i\rangle) \to min_w$$

Отметим, что первое слагаемое у нас обратно пропорционально ширине полосы, но мы и максимизацию заменили на минимизацию, так что тут всё в порядке. Второе слагаемое – это штраф за то, что объекты неправильно расположены относительно разделительной полосы. В конце концов, никто нам не обещал, что классы наши линейно разделимы и можно провести оптимальную плоскость вообще без ошибок.

Итоговое положение плоскости задаётся всего несколькими обучающими примерами. Это ближайшие к плоскости правильно классифицированные объекты, которые называют опорными векторами. Весь метод, соответственно, зовётся методом опорных векторов, или support vector machine, или сокращённо SVM. Начиная с шестидесятых годов это был сильнейший из известных методов машинного обучения. В девяностые его сменили методы, основанные на деревьях решений, которые, в свою очередь, недавно передали «пальму первенства» нейросетям.

Почему же SVM был столь популярен? Из-за небольшого количества параметров и доказуемой оптимальности. Сейчас для нас нормально выбирать специальный алгоритм под задачу и подбирать оптимальные гиперпараметры для этого алгоритма перебором, а когда-то трава была зеленее, а компьютеры медленнее, и такой роскоши у людей не было. Поэтому им нужны были модели, которые гарантированно неплохо работали бы в любой ситуации. Такой моделью и был SVM.

Другие замечательные свойства SVM: существование уникального решения и доказуемо минимальная склонность к переобучению среди всех популярных классов линейных классификаторов. Кроме того, несложная модификация алгоритма, ядровый SVM, позволяет проводить нелинейные разделяющие поверхности.

21.2.1 Двойственная задача

21.3 Неразделимый случай, slack-переменные

В реальных задачах данные часто не являются линейно разделимыми из-за:

1. Наложения классов (перекрывающиеся распределения)
2. Выбросов (outliers) в данных
3. Шума в измерениях
4. Нелинейной границы между классами

Подробнее на Рисунке 4

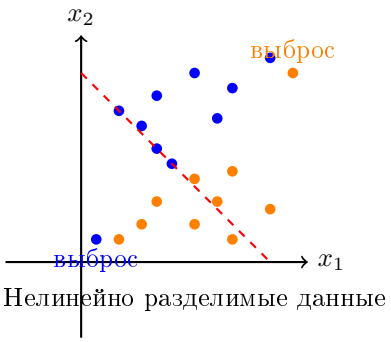


Рис. 4: Пример неразделимых данных с выбросами

Slack-переменные (ослабляющие переменные)

Для обработки неразделимого случая вводятся **slack-переменные** $\xi_i \geq 0$, которые позволяют объектам нарушать запас на Рисунке 5

$$y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1 - \xi_i, \quad i = 1, \ldots, n$$
$$\xi_i \geq 0, \quad i = 1, \ldots, n$$

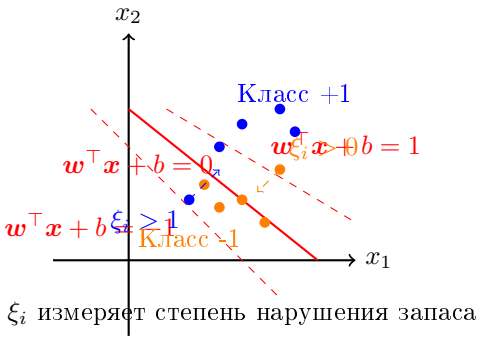


Рис. 5: Slack-переменные ξ_i позволяют точкам нарушать запас

Интерпретация slack-переменных

- $\xi_i = 0$: Точка $\boldsymbol{x}_i$ корректно классифицирована с запасом ≥ 1.
- $0 < \xi_i \leq 1$: Точка $\boldsymbol{x}_i$ находится внутри запаса, но классифицирована правильно.
- $\xi_i > 1$: Точка $\boldsymbol{x}_i$ классифицирована неправильно

Величина ξ_i измеряет, насколько точка $\boldsymbol{x}_i$ нарушает условие правильной классификации с запасом.
Задача оптимизации с slack-переменными
Исходная задача SVM модифицируется

$$\begin{aligned} \min_{\boldsymbol{w},b,\boldsymbol{\xi}} \quad & \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{n} \xi_i \\ \text{при условиях} \quad & y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1 - \xi_i, \quad i = 1, \ldots, n \\ & \xi_i \geq 0, \quad i = 1, \ldots, n \end{aligned}$$

где $C > 0$ — гиперпараметр, задающий компромисс между:

- Максимизацией запаса (минимизацией $\|\boldsymbol{w}\|^2$)
- Минимизацией ошибок (минимизацией $\sum \xi_i$)

Влияние параметра C

- **Большое** C: Большой штраф за ошибки → узкий запас, возможно переобучение
- **Малое** C: Маленький штраф за ошибки → широкий запас, возможна недотяжка

Проиллюстрированно на Рисунке 6

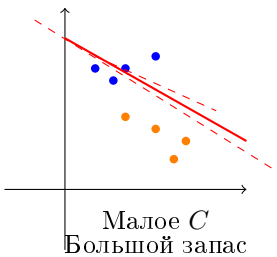


Рис. 6: Влияние параметра C на ширину запаса

21.4 Hinge loss

[Hinge loss] Функция потерь hinge (hinge loss) определяется как:

$$\mathcal{L}_{\text{hinge}}(y, z) = \max(0, 1 - yz)$$

где $y \in \{-1, +1\}$ — истинная метка, $z = \boldsymbol{w}^\top \boldsymbol{x} + b$ — выход классификатора.

Связь с задачей SVM

Задачу SVM с slack-переменными можно переписать в терминах hinge loss:

$$\begin{aligned} \min_{\boldsymbol{w},b} \quad & \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{n} \max(0, 1 - y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b)) \\ = \min_{\boldsymbol{w},b} \quad & \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{n} \mathcal{L}_{\text{hinge}}(y_i, \boldsymbol{w}^\top \boldsymbol{x}_i + b) \end{aligned}$$

Таким образом, $\xi_i = \max(0, 1 - y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b))$.

Свойства hinge loss изображены на Рисунке 7

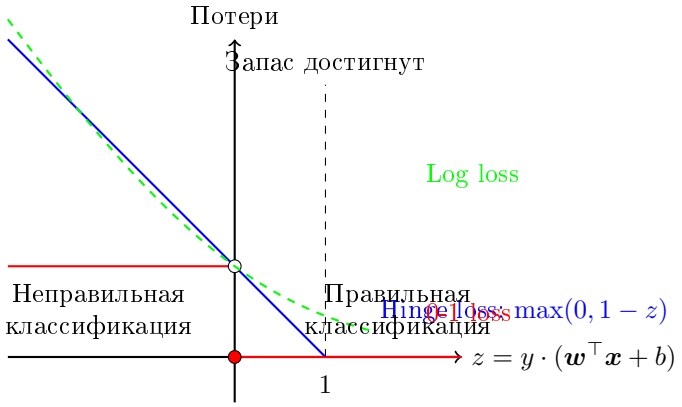


Рис. 7: Сравнение hinge loss с другими функциями потерь

Важные свойства hinge loss

1. **Выпуклость:** hinge loss выпукла, но не везде дифференцируема
2. **Субградиент:** в точке $z = 1$ градиент плавает:

$$\partial\mathcal{L}_{\text{hinge}}(z) = \begin{cases} -1, & \text{если } z < 1 \\ [-1, 0], & \text{если } z = 1 \\ 0, & \text{если } z > 1 \end{cases}$$

3. **Разреженность:** при $z > 1$ градиент равен нулю $\rightarrow$ только точки на границе или внутри запаса влияют на решение (опорные векторы)
4. **Робастность:** менее чувствительна к выбросам, чем log loss

Оптимизация с hinge loss

Задача SVM является задачей выпуклой оптимизации с ограничениями.

Прямая задача:

$$\begin{aligned} & \min_{\boldsymbol{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{n} \xi_i \\ & \text{s.t. } y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

Решение выражается через **опорные векторы** (объекты с $\alpha_i > 0$):

$$\boldsymbol{w} = \sum_{i=1}^{n} \alpha_i y_i \boldsymbol{x}_i$$

21.5 Ядровой трюк, свойства ядер, примеры ядер

Трюк изображён на Рисунке 8

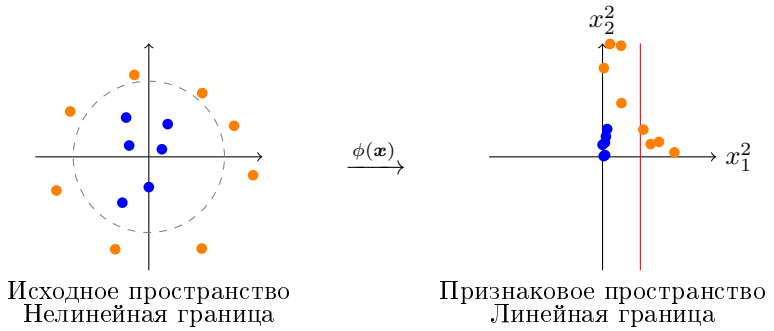


Рис. 8: Идея ядрового метода: нелинейное преобразование в пространство большей размерности

Основная идея

Вместо явного вычисления преобразования $\phi : \mathbb{R}^d \to \mathbb{R}^D$ (где $D \gg d$ или $D = \infty$), мы работаем только со **скалярными произведениями**:

$$K(\boldsymbol{x}, \boldsymbol{z}) = \langle \phi(\boldsymbol{x}), \phi(\boldsymbol{z}) \rangle$$

Определение ядра

[Положительно определённое ядро] Функция $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ называется **положительно определённым ядром**, если:

1. K симметрична: $K(\boldsymbol{x}, \boldsymbol{z}) = K(\boldsymbol{z}, \boldsymbol{x})$
2. Для любого набора точек $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \in \mathcal{X}$ матрица Грама $[K(\boldsymbol{x}_i, \boldsymbol{x}_j)]_{i,j=1}^n$ положительно полуопределена

Примеры ядер изображены на Рисунке 11

Название ядра	**Формула**	**Параметры**
Линейное	$K(\boldsymbol{x}, \boldsymbol{z}) = \boldsymbol{x}^\top \boldsymbol{z}$	Нет
Полиномиальное	$K(\boldsymbol{x}, \boldsymbol{z}) = (\gamma \boldsymbol{x}^\top \boldsymbol{z} + r)^d$	$d \in \mathbb{N}, \gamma > 0, r \geq 0$
RBF (Гауссово)	$K(\boldsymbol{x}, \boldsymbol{z}) = \exp\left(-\gamma \lVert \boldsymbol{x} - \boldsymbol{z} \rVert^2\right)$	$\gamma > 0$
Сигмоидное	$K(\boldsymbol{x}, \boldsymbol{z}) = \tanh\left(\gamma \boldsymbol{x}^\top \boldsymbol{z} + r\right)$	$\gamma > 0, r$

Таблица 11: Часто используемые ядерные функции

Математические свойства ядер

[Теорема Мерсера] Если K — непрерывное симметричное положительно определённое ядро на компактном множестве $\mathcal{X} \subset \mathbb{R}^d$, то существует последовательность $\{\lambda_i\}_{i=1}^\infty$ ($\lambda_i \geq 0$) и ортонормированная система $\{\phi_i\}_{i=1}^\infty$ в $L^2(\mathcal{X})$ такая, что:

$$K(\boldsymbol{x}, \boldsymbol{z}) = \sum_{i=1}^{\infty} \lambda_i \phi_i(\boldsymbol{x}) \phi_i(\boldsymbol{z})$$

Такой ряд сходится абсолютно и равномерно.

Замкнутость относительно операций

1. **Сумма ядер:** Если K_1 и K_2 — ядра, то $K(\boldsymbol{x}, \boldsymbol{z}) = K_1(\boldsymbol{x}, \boldsymbol{z}) + K_2(\boldsymbol{x}, \boldsymbol{z})$ — ядро
2. **Произведение ядер:** Если K_1 и K_2 — ядра, то $K(\boldsymbol{x}, \boldsymbol{z}) = K_1(\boldsymbol{x}, \boldsymbol{z}) \cdot K_2(\boldsymbol{x}, \boldsymbol{z})$ — ядро
3. **Масштабирование:** Если K — ядро и $c > 0$, то cK — ядро
4. **Композиция с функцией:** Если K — ядро и $f : \mathbb{R} \to \mathbb{R}$ — многочлен с неотрицательными коэффициентами, то $f(K)$ — ядро

Настройка гиперпараметров:

Для SVM с RBF ядром нужно настраивать:

- C: компромисс между запасом и ошибками.
- γ: сложность границы решения

Часть V
Методы без учителя и методы ближайших соседей

22 Метод главных компонент

Метод главных компонент во всех библиотечных реализациях называется PCA, существует для понижения размерности. Вспомним зачем вообще интересно понижать размерность

22.1 Связь с SVD

Причины понижения размерности

1. В задачах ML размерность данных может быть очень большой. Например, миллионы признаков, особенно когда это разреженные признаки – становится сложно работать.
2. Такие данные сложно визуализировать, сложно обучать модели.
3. Чтобы снизить размерность, нам требуется использовать какой-то механизм, который по-хорошему воспроизводим.

PCA – линейный способ понижения размерности, который интуитивен и используется широко на практике. Зачем нам может понадобиться матричное разложение на практике? Матрица объект-признак X может быть приближена произведением двух матриц меньшего ранга

$$X_{l,d} = U_{l,k} V^T_{k,d}$$

Здесь размерность $k < d, l$. В таком случае мы можем сказать, что у нас есть матрица перехода и матрица, которая кодирует всю информацию. И теперь использовать приближение меньшего ранга для всех наших данных, потеряв часть информации, но используя меньше признаков. Такое разложение можно построить с помощью минимизации $\left|X - UV^T\right| \to \min$. Тут норма Фробениуса или какая-нибудь ещё. PCA нам говорит, что мы можем матрицу признаков разложить на произведение трёх матриц: $X = U\Sigma V^T$ В этом выражении матрицы U, V^T являются ортогональными матрицами, а Σ – диагональная матрица. Можем сказать, что k ненулевых признаков остаются ненулевыми и значимыми в ходе такого SVD разложения. Получаем это с помощью взятия k диагональных элементов X и соответствующих им столбцов в матрицах U, V. Таким образом при перемножении получается итоговая матрица меньшей размерности, итоговый ранг равен k

22.2 Теорема Эккарта–Юнга, доказательство

Постановка задачи

Рассмотрим матрицу $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ с центрированными столбцами (нулевым средним). Мы хотим найти матрицу $\hat{\boldsymbol{X}} \in \mathbb{R}^{n \times d}$ ранга $k < \operatorname{rank}(\boldsymbol{X})$ такую, что:

$$\hat{\boldsymbol{X}} = \arg \min_{\operatorname{rank}(\boldsymbol{B}) \le k} \|\boldsymbol{X} - \boldsymbol{B}\|_F^2$$

Здесь $\|\cdot\|_F$ — норма Фробениуса: $\|\boldsymbol{A}\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$.

Формулировка теоремы

[Эккарта-Юнга, 1936] Пусть $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ имеет сингулярное разложение:

$$\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^\top$$

где $r = \operatorname{rank}(\boldsymbol{X})$, $\sigma_1 \ge \sigma_2 \ge \cdots \ge \sigma_r > 0$ — сингулярные числа, $\boldsymbol{U} = [\boldsymbol{u}_1, \ldots, \boldsymbol{u}_n]$, $\boldsymbol{V} = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_d]$ — ортогональные матрицы.

Тогда для любого $k < r$ оптимальная аппроксимация ранга k в смысле нормы Фробениуса задаётся как:

$$\hat{\boldsymbol{X}}_k = \sum_{i=1}^{k} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^\top = \boldsymbol{U}_k \boldsymbol{\Sigma}_k \boldsymbol{V}_k^\top$$

Здесь $\boldsymbol{U}_k$, $\boldsymbol{V}_k$ — первые k столбцов $\boldsymbol{U}$ и $\boldsymbol{V}$, $\boldsymbol{\Sigma}_k = \operatorname{diag}(\sigma_1, \ldots, \sigma_k)$.

При этом минимальная ошибка равна:

$$\|\boldsymbol{X} - \hat{\boldsymbol{X}}_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2$$

22.2.1 способ через собственное разложение или через подстановку

Доказательство способом через собственное разложение

Доказательство. Рассмотрим два подхода к доказательству.

Подход 1: Через спектральное разложение $\boldsymbol{X}^\top \boldsymbol{X}$

Пусть $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$ — SVD. Тогда:

$$\boldsymbol{X}^\top \boldsymbol{X} = \boldsymbol{V}\boldsymbol{\Sigma}^2\boldsymbol{V}^\top$$

где $\boldsymbol{\Sigma}^2 = \operatorname{diag}(\sigma_1^2, \ldots, \sigma_r^2, 0, \ldots, 0)$.

Для любой матрицы $\boldsymbol{B}$ ранга не более k:

$$\|\boldsymbol{X} - \boldsymbol{B}\|_F^2 = \operatorname{tr}((\boldsymbol{X} - \boldsymbol{B})^\top (\boldsymbol{X} - \boldsymbol{B}))$$

Зафиксируем $\boldsymbol{B}$ и рассмотрим её SVD: $\boldsymbol{B} = \boldsymbol{P}\boldsymbol{\Lambda}\boldsymbol{Q}^\top$, где $\boldsymbol{P} \in \mathbb{R}^{n \times k}$, $\boldsymbol{Q} \in \mathbb{R}^{d \times k}$ имеют ортонормированные столбцы, $\boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_k)$.

Тогда

$$\begin{aligned}\|\boldsymbol{X} - \boldsymbol{B}\|_F^2 &= \|\boldsymbol{X}\|_F^2 + \|\boldsymbol{B}\|_F^2 - 2\operatorname{tr}(\boldsymbol{X}^\top \boldsymbol{B}) \\ &= \sum_{i=1}^{r} \sigma_i^2 + \sum_{i=1}^{k} \lambda_i^2 - 2\operatorname{tr}(\boldsymbol{X}^\top \boldsymbol{P}\boldsymbol{\Lambda}\boldsymbol{Q}^\top)\end{aligned}$$

По неравенству Коши-Шварца для матриц

$$\operatorname{tr}(\boldsymbol{X}^\top \boldsymbol{P}\boldsymbol{\Lambda}\boldsymbol{Q}^\top) \leq \sum_{i=1}^{k} \sigma_i \lambda_i$$

Здесь равенство достигается при $\boldsymbol{P} = \boldsymbol{U}_k$, $\boldsymbol{Q} = \boldsymbol{V}_k$, $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}_k$.

Таким образом:

$$\min_{\operatorname{rank}(\boldsymbol{B}) \leq k} \|\boldsymbol{X} - \boldsymbol{B}\|_F^2 \geq \sum_{i=1}^{r} \sigma_i^2 - \sum_{i=1}^{k} \sigma_i^2 = \sum_{i=k+1}^{r} \sigma_i^2$$

И эта нижняя граница достигается при $\boldsymbol{B} = \boldsymbol{U}_k \boldsymbol{\Sigma}_k \boldsymbol{V}_k^\top$.

Подход 2: Через подпространства.

Рассмотрим проекцию $\boldsymbol{X}$ на подпространство, натянутое на первые k правых сингулярных векторов:

$$\hat{\boldsymbol{X}}_k = \boldsymbol{X}\boldsymbol{V}_k\boldsymbol{V}_k^\top$$

Покажем, что это минимизирует ошибку. Для любой матрицы $\boldsymbol{B}$ ранга k пусть $\mathcal{V}$ — подпространство, натянутое на её правые сингулярные векторы. Тогда:

$$\|\boldsymbol{X} - \boldsymbol{B}\|_F^2 \geq \|\boldsymbol{X} - \boldsymbol{X}\boldsymbol{P}_{\mathcal{V}}\|_F^2$$

Здесь $\boldsymbol{P}_{\mathcal{V}}$ — проектор на $\mathcal{V}$.

Но $\|\boldsymbol{X} - \boldsymbol{X}\boldsymbol{P}_{\mathcal{V}}\|_F^2 = \|\boldsymbol{X}\|_F^2 - \|\boldsymbol{X}\boldsymbol{P}_{\mathcal{V}}\|_F^2$, и:

$$\|\boldsymbol{X}\boldsymbol{P}_{\mathcal{V}}\|_F^2 = \operatorname{tr}(\boldsymbol{P}_{\mathcal{V}}\boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{P}_{\mathcal{V}}) = \operatorname{tr}(\boldsymbol{P}_{\mathcal{V}}\boldsymbol{V}\boldsymbol{\Sigma}^2\boldsymbol{V}^\top \boldsymbol{P}_{\mathcal{V}})$$

Максимум этого выражения (и, соответсвенно, минимум нормы исходной разницы) достигается, когда $\mathcal{V}$ — подпространство, натянутое на первые k правых сингулярных векторов $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$.

Таким образом, $\hat{\boldsymbol{X}}_k = \boldsymbol{X}\boldsymbol{V}_k\boldsymbol{V}_k^\top$ является оптимальной аппроксимацией. □

Следствия теоремы

1. **Оптимальность PCA:** PCA находит оптимальное (в смысле нормы Фробениуса) k-мерное подпространство для аппроксимации данных.
2. **Доля объяснённой дисперсии:** доля дисперсии, сохранённая при использовании k главных компонент:
$$R_k^2 = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2}$$
3. **Вложенная оптимальность:** для любого $k_1 < k_2$, оптимальная аппроксимация ранга k_2 содержит в себе оптимальную аппроксимацию ранга k_1.

22.3 Две альтернативные итеративные формулировки

Формулировка 1: Последовательная максимизация дисперсии
Алгоритм может быть сформулирован как последовательное нахождение главных компонент:

1. Первая главная компонента:
$$\boldsymbol{v}_1 = \arg\max_{\|\boldsymbol{v}\|=1} \|\boldsymbol{X}\boldsymbol{v}\|^2 = \arg\max_{\|\boldsymbol{v}\|=1} \boldsymbol{v}^\top \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{v}$$
2. Для k-й компоненты ($k > 1$):
$$\boldsymbol{v}_k = \arg\max_{\|\boldsymbol{v}\|=1, \boldsymbol{v}\perp\{\boldsymbol{v}_1,\ldots,\boldsymbol{v}_{k-1}\}} \|\boldsymbol{X}\boldsymbol{v}\|^2$$

Формулировка 2: Минимизация ошибки реконструкции
Альтернативно, PCA можно сформулировать как задачу минимизации при условии $\boldsymbol{W}^\top\boldsymbol{W} = \boldsymbol{I}_k$:

$$\min_{\boldsymbol{W}\in\mathbb{R}^{d\times k}, \boldsymbol{Z}\in\mathbb{R}^{n\times k}} \|\boldsymbol{X} - \boldsymbol{Z}\boldsymbol{W}^\top\|_F^2$$

Здесь:

- $\boldsymbol{W}$ — матрица нагрузок (loadings)
- $\boldsymbol{Z}$ — матрица счётных факторных значений (scores)

Связь между формулировками
Две формулировки эквивалентны. Из решения задачи минимизации ошибки реконструкции:

$$\boldsymbol{Z} = \boldsymbol{X}\boldsymbol{W}, \quad \boldsymbol{W} = \boldsymbol{V}_k$$

Итеративный алгоритм NIPALS
Алгоритм NIPALS (Nonlinear Iterative Partial Least Squares) вычисляет главные компоненты итеративно:

1. Инициализировать $\boldsymbol{t}$ (например, первый столбец $\boldsymbol{X}$)
2. Повторять до сходимости:
$$\begin{aligned} \boldsymbol{p} &\leftarrow \frac{\boldsymbol{X}^\top\boldsymbol{t}}{\boldsymbol{t}^\top\boldsymbol{t}} \quad \text{(нагрузки)} \\ \boldsymbol{p} &\leftarrow \frac{\boldsymbol{p}}{\|\boldsymbol{p}\|} \quad \text{(нормализация)} \\ \boldsymbol{t} &\leftarrow \boldsymbol{X}\boldsymbol{p} \quad \text{(счётные значения).} \end{aligned}$$
3. Вычесть компоненту: $\boldsymbol{X} \leftarrow \boldsymbol{X} - \boldsymbol{t}\boldsymbol{p}^\top$
4. Перейти к следующей компоненте

Метод	Формула	Когда использовать
Центрирование	$\tilde{x}_{ij} = x_{ij} - \bar{x}_j$	Всегда для PCA
Масштабирование (StandardScaler)	$\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$	Когда признаки в разных единицах измерения
Масштабирование к [0,1] (MinMaxScaler)	$\tilde{x}_{ij} = \frac{x_{ij} - \min_j}{\max_j - \min_j}$	Когда важны относительные различия
Нормализация (L2)	$\tilde{\boldsymbol{x}}_i = \frac{\boldsymbol{x}_i}{\|\boldsymbol{x}_i\|}$	Для данных с разной длиной векторов

Таблица 12: Методы предобработки для PCA

22.4 Предобработка для PCA

Важность предобработки

PCA чувствителен к масштабированию признаков. Без правильной предобработки компоненты могут быть определены преимущественно признаками с наибольшей дисперсией, что не всегда соответствует их информативности.

Основные методы предобработки

Центрирование: необходимое условие

Для PCA **обязательно** требуется центрирование данных. Без этого первая главная компонента будет направлена к центру масс данных, а не в направлении максимальной дисперсии.

$$\bar{\boldsymbol{x}} = \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{x}_i, \quad \tilde{\boldsymbol{x}}_i = \boldsymbol{x}_i - \bar{\boldsymbol{x}}$$

Когда масштабировать

1. Признаки измерены в разных единицах (метры vs килограммы)
2. Дисперсии признаков сильно различаются
3. Нет априорных причин считать один признак более важным

Когда не масштабировать:

1. Все признаки измерены в одних единицах
2. Различия в дисперсии отражают реальную важность признаков.
3. Хотим сохранить оригинальную шкалу

Пример: влияние масштабирования

Рассмотрим два признака:

- x_1: рост в сантиметрах (дисперсия ≈ 100)
- x_2: вес в килограммах (дисперсия ≈ 100)

Без масштабирования PCA будет определяться обоими признаками. Если изменить единицы измерения:

- x_1: рост в метрах (дисперсия ≈ 0.01)
- x_2: вес в килограммах (дисперсия ≈ 100)

Без масштабирования первая компонента будет почти полностью определяться весом.

Обработка пропущенных значений

Перед применением PCA необходимо обработать пропущенные значения

1. Удаление объектов с пропусками (при небольшом их количестве)
2. Заполнение средним/медианой значением
3. PCA с пропусками (опционально, сложные методы)

Обработка выбросов. PCA чувствителен к выбросам. Методы обработки

1. Использование RobustScaler
2. Методы robust PCA
3. Удаление выбросов перед анализом

Перед применением PCA стоит проверить линейные зависимости
Практический конвейер предобработки

1. Обработка пропущенных значений
2. Обнаружение и обработка выбросов
3. Проверка и преобразование типов данных
4. Кодирование категориальных переменных (если необходимо)
5. Центрирование данных
6. Масштабирование (при необходимости)
7. Проверка корреляций между признаками

Визуализация необходимости предобработки

1. **Boxplot:** Для обнаружения различий в масштабах признаков
2. **Scatterplot matrix:** Для визуализации отношений между признаками
3. **Гистограммы:** Для проверки распределений
4. **Корреляционная матрица:** Для выявления мультиколлинеарности

23 k ближайших соседей (kNN) для классификации и регрессии

Представим, что мы проводим классификацию объектов на два класса – красный или жёлтый. Нам дана некоторая обучающая выборка и целевой объект (серый) Мы хотим определить, к какому классу относится серый объект. Интуитивно очевидно, что он должен быть жёлтым, потому что все его соседи жёлтые. Эта интуиция и отражает суть метода KNN – классифицировать целевой объект, исходя из того, какие классы у объектов, которые максимально похожи на него.

Перейдём теперь к более формальному описанию алгоритма. Рассмотрим сначала задачу многоклассовой классификации, а регрессией займёмся позже.

Пусть дана обучающая выборка $X = \{(x_i, y_i)\}_{i=1}^N$, где $x_i \in \mathcal{X}$, $y_i \in \mathcal{Y} = \{1, \ldots, C\}$. Пусть также задана некоторая симметричная по своим аргументам функция расстояния $\rho : \mathcal{X} \times \mathcal{X} \to [0, +\infty)$. Предположим, что требуется классифицировать новый объект u. Для этого найдём k наиболее близких к u в смысле расстояния ρ объектов обучающей выборки $X_k(u) = \{x_{u(1)}, \ldots, x_{u(k)}\}$:

$$\forall x_{\text{in}} \in X_k(u) \quad \forall x_{\text{out}} \in X \setminus X_k(u) \quad \rho(u, x_{\text{in}}) \leqslant \rho(u, x_{\text{out}})$$

Метку класса объекта $x_{u(i)}$ будем обозначать $y_{u(i)}$. Класс нового объекта естественным образом определим как наиболее часто встречающийся класс среди объектов $X_k(u)$:

$$a(u) = \arg\max_{y \in \mathcal{Y}} \sum_{i=1}^{k} \mathbb{I}[y_{u(i)} = y]$$

Формула может показаться страшной, но на самом деле всё довольно просто: для каждой метки класса $y \in \mathcal{Y}$ количество соседей u с такой меткой можно посчитать, просто просуммировав по всем соседям индикаторы событий, соответствующих тому, что метка соседа равна y.

Легко заметить, что этот алгоритм позволяет также оценивать вероятности классов. Для этого достаточно просто посчитать частоты классов соседей:

$$P(u \sim y) = \frac{\sum_{i=1}^{k} \mathbb{I}[y_{u(i)} = y]}{k}$$

Стоит, однако, понимать, что, хоть такая функция и удовлетворяет свойствам вероятности (она неотрицательна, аддитивна и ограничена единицей), это не более чем эвристика.

Несмотря на то что формально фаза обучения отсутствует, алгоритм может легко переобучиться. Вы можете убедиться в этом сами, использовав маленькое количество соседей (например, одного или двух), – границы классов оказываются довольно сложными. Происходит это из-за того, что параметрами алгоритма можно считать всю обучающую выборку, довольно большую по размеру. Из-за этого алгоритму легко подстроиться под конкретные данные

23.1 Параметры и гиперпараметры

Гиперпараметры есть k и метрика. Параметров нет

24 Индексы для kNN

Main object to apply kNN nowadays are embeddings (vectors) produced by neural networks. Such embeddings represent object properties in a compact format. Nearest neighbors are used to roughly select possible candidates for further processing.

24.1 Зачем нужны индексы

KNN index is a data structure that allows you to quickly find the N nearest vectors for a given vector. We perform some precompute beforehand to increase speed of final search.

24.2 Идеи индексов для kNN от простых к сложным

1. Clustering
2. k-d tree
   - default for solid team
   - ball tree is extension for high dimensional cases
3. HNSW (most common nowadays)

24.3 k-dimensional tree (kd-tree)

Представим на секунду, что у нас есть всего лишь один признак, то есть объекты выражаются вещественными числами, а не векторами. В этом случае для поиска ближайшего соседа напрашивается всем вам известное бинарное дерево поиска, которое позволяет находить элементы за логарифмическое время. Оказывается, существует аналог данной структуры в многомерном пространстве, который называется k-d-дерево (k-d tree, сокращение от k-dimensional tree).

Как и в обычном дереве поиска, в k-d-дереве каждый узел является объектом обучающей выборки, который особым образом делит пространство на два полупространства. Таким образом, всё пространство оказывается поделено на множество малых областей, и такое деление оказывается очень полезным при поиске ближайших соседей.

Рассмотрим подробнее, как строится такое дерево. Трудность в применении обычного дерева поиска состоит в том, что мы не можем напрямую сравнить два вектора так же, как два вещественных числа. Чтобы эту проблему преодолеть, узлы дерева будут делить пространство лишь по одной оси. При движении вниз по дереву оси, по которым точки делят пространство, циклически сменяют друг друга. Например, в двумерном пространстве корень будет отвечать за деление по x-координате, его сыновья – за деление по y-координате, а внуки – снова за x-координату, и так далее. Посмотрим, как это работает на примере:

На картинке выше корень $(30, 40)$ делит все точки по оси x: слева оказываются точки, у которых $x < 30$, а справа – те, у которых $x \geq 30$. Аналогично левый сын корня (5,25) делит своё поддерево по оси y: слева оказываются точки, у которых $y < 25$, а справа – те, у которых $y \geq 25$.

Остаётся вопрос – как выбирать точки, которые будут делить пространство пополам? Чтобы дерево было сбалансированным, нужно находить точку с медианой, соответствующей уровню поддерева координаты. На практике часто ограничиваются выбором случайной точки или любой эвристикой по приближённому поиску медианы (например, медиана некоторого подмножества точек). Это позволяет ускорить построение дерева, но убирает все гарантии на его сбалансированность.

Добавлять новые точки можно так же, как и в одномерном дереве поиска. Спускаясь по дереву, можно однозначно определить лист, к которому нужно подвесить новую точку, чтобы не нарушить все свойства

дерева. При добавлении большого количества точек, однако, дерево может перестать быть сбалансированным, и нужно проводить ребалансировку. Также существуют варианты k-d-деревьев, которые сохраняют сбалансированность при добавлении / удалении точек.

Поговорим теперь про то, как же находить ближайших соседей с помощью такого дерева. Будем производить обход дерева в глубину с двумя модификациями.

- Во-первых, будем запоминать наиболее близкую точку. Это позволит не заходить в поддеревья, задающие области, которые заведомо дальше, чем текущая наиболее близкая точка, поэтому не имеет смысла искать в них ближайших соседей.
- Во-вторых, будем прежде всего обходить те поддеревья, которые задают наиболее близкие области, а значит, с большей вероятностью содержат ближайшего соседа.

Сложность метода по размеру обучающей выборки в среднем равна $O(\log N)$ при равномерном распределении точек. При большой размерности пространства, однако, алгоритму приходится посещать больше ветвей дерева, чтобы найти ближайших соседей. Например, если $N \approx D$, то сложность становится примерно такой же, как и в случае полного перебора. В общем случае для того чтобы асимптотика действительно была логарифмической, нужно, чтобы $N \geq 2^D$. Поэтому уже при количестве признаков порядка сотни алгоритм не даёт существенных преимуществ перед полным перебором

24.4 Annoy

Алгоритмы, основанные на деревьях, очень часто применяются в задачах поиска соседей. Идея всех таких методов заключается в итеративном разделении пространства случайными гиперплоскостями и построении на базе этого разделения дерева, в листах которого содержится малое число объектов.

Одним из наиболее ярких представителей этого семейства является Annoy – алгоритм, который используется Spotify для рекомендаций музыки. Задача подобных рекомендательных систем довольно простая – нужно посоветовать пользователю композиции, которые он ещё не слушал, но которые при этом с высокой вероятностью ему понравятся. Простая и рабочая идея – предлагать композиции, похожие на те, которые он уже слушает. Здесь на помощь как раз и приходят методы поиска ближайших соседей.

Annoy в какой-то степени похож на k-d-деревья. Сначала выбираются два случайных объекта обучающей выборки и проводится гиперплоскость, симметрично их разделяющая. Затем для каждого полученного полупространства итеративно запускается такая же процедура, которая продолжается до тех пор, пока в каждой области будет не более M объектов (M — гиперпараметр).

Таким образом задаётся бинарное дерево с глубиной порядка $O(\log N)$ в среднем. Спускаясь по этому дереву, можно найти область, в которой лежит целевой объект и некоторое количество близких к нему элементов обучающей выборки. Проблема в том, что это не обязательно будут самые близкие объекты, поэтому для увеличения точности составляется лес из таких деревьев и берётся объединение соответствующих целевому объекту областей.

Чем больше таких деревьев берётся, тем более точным будет результат, но придётся тратить большее время на его поиск.

Преимущество алгоритма – простота нахождения компромисса между скоростью работы и точностью с помощью тюнинга гиперпараметров. К минусам можно отнести то, что алгоритм плохо параллелится и переносится на GPU, не работает эффективно с батчами, а также то, что для добавления новой точки в обучающую выборку придётся перезапускать процедуру с самого начала

24.5 HNSW

Следующий класс алгоритмов основан на построении специального графа близости (proximity graph) на объектах выборки и дальнейшем жадном поиске по этому графу. Алгоритмы этого семейства сейчас считаются state-of-the-art (SotA) для многих задач.

Рассмотрим подробнее этот класс алгоритмов на примере одного из наиболее популярных из них под названием Navigable small world (NSW). Идея его в следующем: на данных строится граф (он также называется NSW), который удовлетворяет двум следующим свойствам:

1. Между любыми двумя точками существует короткий путь, или, более формально, матожидание числа кратчайшего пути между двумя случайно выбранными вершинами растёт как $O(\log N)$.
2. Средняя степень вершины мала.

На первый взгляд может показаться, что тяжело выполнить одновременно оба свойства, но на самом деле большая часть графов в реальном мире являются NSW-графами. Самый простой пример –

это известное правило шести рукопожатий: любые два случайных человека соединены короткой последовательностью личных контактов длиной не более шести, несмотря на то, что количество знакомых у среднего человека (100-1000) мало по сравнению с населением Земли.

В таких графах существует очень простой метод поиска соседей. Нужно выбрать случайную точку, среди её соседей выбрать того, который ближе всего к целевому объекту, и повторить процедуру уже для него. Показано, что такой жадный поиск имеет полилогарифмическую асимптотику.

24.5.1 Построение графа

Проблема такого подхода в том, что можно попасть в плотный кластер и очень долго оттуда выбираться. Для решения этой проблемы используется иерархия NSW, или *Hierarchical navigable small world (HNSW)*. Исходный граф является нулевым слоем. Каждый следующий слой строится в два шага

1. Каждая вершина текущего слоя попадает в следующий с некоторой вероятностью p.
2. На всех вершинах, попавших в новый слой, строится NSW.

По построению количество слоёв будет $O(\log N)$.

Поиск начинается в самом верхнем слое. После нахождения ближайшей к целевому объекту вершины спускаемся на слой ниже и начинаем поиск из этой вершины. Повторяем процедуру, пока не спустимся до нулевого слоя. Таким образом, на каждом слое мы всё больше уточняем наш ответ. Стоит отметить, что для ускорения работы иногда поиск останавливают не при нахождении ближайшей вершины, а раньше, используя критерии остановки.

Интуитивно легко понять, почему такая иерархическая структура решает проблему плотных кластеров: в верхних слоях вершин мало, а расстояния между ними в среднем большие, а значит, таких кластеров там почти нет. Поэтому, попадая в нижний слой, мы чаще всего оказываемся уже в нужном кластере и просто уточняем результат работы алгоритма.

24.5.2 Использование графа

HNSW, так же как и рассмотренные ранее приближённые методы, позволяет искать баланс между точностью и скоростью. Плюс ко всему на реальных данных он часто работает лучше других методов и сейчас считается предметом искусства. Однако этот способ поиска не лишён и недостатков. Главный заключается в том, что нельзя добавлять точки в обучающую выборку без перестройки структуры. Помимо этого, он довольно требователен по памяти из-за того, что для каждого слоя приходится хранить как вершины, которые в него входят, так и связи между этими вершинами

24.6 Примеры векторных баз данных

- OpenAI
- Qdrant
- Try chroma
- Feast
- NVIDIA AI store

Все, кроме первой, ready for production

25 Горький урок

Будущее ИИ – за общими, масштабируемыми методами, которые используют всё больше вычислений и данных, а не за попытками встроить в системы готовые человеческие знания и модели мышления. Об этом и была статья весёлого деда

25.1 Основные выводы статьи

1. Вычислительные методы в долгосрочной перспективе эффективнее человеческих знаний. Самый важный урок за 70 лет исследований в области ИИ – общие методы, использующие вычислительные мощности, в итоге оказываются намного эффективнее, чем подходы, основанные на знаниях человека.

2. Закон Мура меняет правила игры. Благодаря экспоненциальному росту доступных вычислений методы, которые масштабируются с ростом вычислительных ресурсов, со временем становятся доминирующими.

3. Поиск и обучение – ключевые методы. Два основных способа использования больших объёмов вычислений – это поиск (search) и обучение (learning), особенно с применением самообучения (self-play) и больших данных.

4. Примеры из истории ИИ подтверждают этот урок

   Шахматы: глубокий перебор оказался эффективнее экспертных систем.

   Го: методы, основанные на поиске и обучении (AlphaGo), превзошли подходы, использующие человеческие знания.

   Распознавание речи: статистические методы (HMM, глубокое обучение) победили экспертные системы.

   Компьютерное зрение: нейросетевые подходы заменили ручное выделение признаков.

5. Встроенные знания человека тормозят прогресс. Попытки встроить в ИИ человеческое понимание мира дают краткосрочный выигрыш, но в долгосрочной перспективе ограничивают развитие, усложняют систему и мешают использовать возможности масштабируемых вычислений.

6. Сложность мира нельзя «вписать» в систему. Содержание человеческого мышления невероятно сложно, и не стоит пытаться формализовать его в простых моделях. Вместо этого нужно создавать мета-методы, которые сами смогут находить и изучать эту сложность.

7. Горький урок – это урок смирения. Успех приходит тогда, когда мы отказываемся от антропоцентричного подхода и позволяем алгоритмам учиться и масштабироваться, даже если это противоречит нашим интуитивным представлениям.

Часть VI
Деревья решений, ансамбли, интерпретируемость

26 Деревья решений

26.1 Определение жадных алгоритмов

Это алгоритмы, заключающиеся в принятии локально оптимальных решений на каждом этапе, допуская, что конечное решение также окажется оптимальным

26.2 Процедура построения

В ходе построения дерева мы разбиваем вашу выборку на две части. Для этого представим, что у нас есть L и R — левое и правое поддерево объектов, тогда вспомним, что разбиение в узле строится по критерию вида $\mathbb{I}[x_i^j < b]$, b – порог, x_i^j – j-й признак у i-го объекта. Осталось понять, как выбрать j и b. Для этого будем перебирать все возможные признаки j, а для каждого признака перебирать все его возможные значения x^j в выборке, тогда мы можем построить разбиение для каждого из вариантов, осталось выбрать оптимальный. Для этого используют функционал следующего вида: $f(X) = \frac{|L|}{|X|}H(L) + \frac{|R|}{|X|}H(R)$

26.3 Информационные критерии, энтропия, индекс Джини

Функция H называется критерием информативности (information criteria). Есть разные способы задать H для задачи классификации. Посмотрим сначала на примере бинарной классификации. Далее p_i – нормированная частота встречаемости i-ого класса в множестве R

Наиболее простым является Misclassification criteria: количество ошибок классификации (предсказываем доминирующий класс и смотрим сколько объектов классифицировали неправильно)

$$H(R) = 1 - \max(p_0, p_1)$$

Не очень хороший способ: например для многоклассовой классификации дерево может хорошо отделять несколько каких-то классов от остальных, но этот критерий нам этого не покажет

Энтропия: чем больше энтропия, тем больше хаоса

$$H(R) = -p_0 \log p_0 - p_1 \log p_1 = -p_0 \log p_0 - (1 - p_0) \log(1 - p_0)$$

Индекс Джини:

$$H(X_m) = \sum_{k=1}^{K} p_k(1 - p_k)$$

Критерий Джини допускает и следующую интерпретацию: $H(X_m)$ равно математическому ожиданию числа неправильно классифицированных объектов в случае, если мы будем приписывать им случайные метки из дискретного распределения, заданного вероятностями $(p_1, \ldots, p_k)$.

26.4 Ответ в листе

Если в лист дерева попали объекты разных классов и нет задачи получить вектор вероятностей, то просто наиболее часто встречаемый в листе класс. Если надо еще получить вектор вероятностей для каждого класса, то вернем вектор из частот встречаемости в листе. Если подробнее, то надо расписать лагранжиан

26.5 Обработка пропусков

Одна из приятных особенностей деревьев – это способность обрабатывать пропуски в данных. Разберёмся, что при этом происходит на этапе обучения и на этапе применения дерева.

Пусть у нас есть некоторый признак x_i, значение которого пропущено у некоторых объектов. Как обычно, обозначим через X_m множество объектов, пришедших в рассматриваемую вершину, а через V_m – подмножество X_m, состоящее из объектов с пропущенным значением x_i. В момент выбора сплитов по этому признаку мы будем просто игнорировать объекты из V_m, а когда сплит выбран, мы отправим их в оба поддерева. При этом логично присвоить им веса: $\frac{|X_l|}{|X_m|}$ для левого поддерева и $\frac{|X_r|}{|X_m|}$ для правого. Веса будут учитываться как коэффициенты при $L(y_i,c)$ в формуле информативности.

Теперь рассмотрим этап применения дерева. Допустим, в вершину, где сплит идёт по i-му признаку, пришёл объект x_0 с пропущенным значением этого признака. Предлагается отправить его в каждую из дальнейших веток и получить по ним предсказания y^l и y^r. Эти предсказания мы усредним с весами $\frac{|X_l|}{|X_m|}$ и $\frac{|X_r|}{|X_m|}$ (которые мы запомнили в ходе обучения):

$$y = \frac{|X_l|}{|X_m|} y^l + \frac{|X_r|}{|X_m|} y^r$$

Для задачи регрессии это сразу даст нам таргет, а в задаче бинарной классификации – оценку вероятности класса 1. Заметим, что если речь идёт о категориальном признаке, может оказаться хорошей идеей ввести дополнительное значение «пропущено» для категориального признака и дальше работать с пропусками, как с обычным значением. Особенно это актуально в ситуациях, когда пропуски имеют системный характер и их наличие несёт в себе определённую информацию

26.6 Бинаризация

Идея состоит в выборе нескольких порогов для одного признака, вместо одного Это настраивается, например, by Border Count Hyperparameter in Catboost (defaults to 254 = 256 - inf) То есть если подпространство исходного пространства признаков, то в соответствии с листом дерева предсказание будет иметь вид $y = \sum_j w_j[x \in J_j]$

26.7 Обрезка (pruning), варианты C4.5 и CART

Мы уже упоминали выше, что деревья легко переобучаются и процесс ветвления надо в какой-то момент останавливать. Для этого есть разные критерии, обычно используются все сразу

- Ограничение по максимальной глубине дерева;
- ограничение на минимальное количество объектов в листе;
- ограничение на максимальное количество листьев в дереве;

- требование, чтобы функционал качества BranchBranch при делении текущей подвыборки на две улучшался не менее чем на s процентов.

Делать это можно на разных этапах работы алгоритма, что не меняет сути, но имеет разные устоявшиеся названия

- Можно проверять критерии прямо во время построения дерева, такой способ называется pre-pruning или early stopping;
- а можно построить дерево жадно без ограничений, а затем провести стрижку (pruning), то есть удалить некоторые вершины из дерева так, чтобы итоговое качество упало не сильно, но дерево начало подходить под условия регуляризации. При этом качество стоит измерять на отдельной, отложенной выборке.

Стандарт C4.5

- Критерий нормализованной энтропии
- Остановка зависит от размера листа.
- Включает обрезку

Стандарт CART

- Критерии Джини
- Сокращение затрат и сложности
- Замещающие предикаты для отсутствующих данных

26.8 Гиперпараметры (минимум 4 основных)

- Максимальная глубина
- Минимальное число элементов в листьях (и выбор условия в листе)
- Минимальное разделение образцов
- Минимальное количество примесей

27 Разложение bias–variance–noise

Есть проблема поиска баланса между недообучением и переобучением.

Компромисс между смещением и дисперсией – это свойство набора моделей предсказания, когда модели с меньшим отклонением (low bias) от имеющихся данных имеют более высокую дисперсию (high variance) на новых данных (то есть подвержены переобучению), и наоборот модели с меньшей дисперсией (low variance) от имеющихся данных имеют более высокое смещение (high bias) на новых данных (это приводит к недообучению). Притом всегда остаётся неустранимая случайность. Ниже более подробное объяснение этого процесса. Возникает конфликт при попытке одновременно минимизировать эти два источника ошибок, которые мешают алгоритмам обучения с учителем делать обобщение за пределами тренировочного набора.

27.1 Случайность в ошибке

Случайность возникает

- Из данных
- Из-за точности
- В процессе разбиения на тренировочный и тестовый наборы
- Вследствие человеческого фактора
- Пересборки тренировочного набора

27.1.1 Интуиция каждой компоненты

Итого,

1. Random sampling in training procedure
   for Linear model – initial weights
   for Tress – feature and threshold sampling
   for SGD – sampling of batches
2. Computational errors due to limited precision arithmetics
3. Target noise
   ε in regression model
4. Objects sampling – crucial source of variance
   Train set sampling
   Validation set sampling

Смещение – это разница между средним прогнозом нашей модели и правильным значением, которое мы пытаемся предсказать. Модель с высоким смещением уделяет очень мало внимания данным обучения и чрезмерно упрощает модель. Это всегда приводит к высокой погрешности в обучающих и тестовых данных (недообучение).

Дисперсия – это изменчивость прогноза модели для данной точки данных или значения, которое говорит нам о разбросе наших данных. Модель с высокой дисперсией уделяет большое внимание обучающим данным и не обобщается на данные, которые она раньше не видела. В результате такие модели очень хорошо работают с обучающими данными, но имеют высокую частоту ошибок в тестовых данных (переобучение)

27.2 Доказательство для MSE

Рассмотрим задачу регрессии с квадратичной функцией потерь. Представим также для простоты, что целевая переменная y — одномерная и выражается через переменную x как:

$$y = f(x) + \varepsilon,$$

где f — некоторая детерминированная функция, а ε — случайный шум со следующими свойствами:

$$\mathbb{E}\,\varepsilon = 0, \quad \operatorname{Var}\varepsilon = \mathbb{E}\,\varepsilon^2 = \sigma^2$$

В зависимости от природы данных, которые описывает эта зависимость, её представление в виде точной $f(x)$ и случайной ε может быть продиктовано по-разному

- Данные на самом деле имеют случайный характер;
- измерительный прибор не может зафиксировать целевую переменную абсолютно точно;
- имеющихся признаков недостаточно, чтобы исчерпывающим образом описать объект, пользователя или событие.

Функция потерь на одном объекте x равна

$$\text{MSE} = \big(y(x) - a(x)\big)^2$$

Однако знание значения MSE только на одном объекте не может дать нам общего понимания того, насколько хорошо работает наш алгоритм. Какие факторы мы бы хотели учесть при оценке качества алгоритма? Например, то, что выход алгоритма на объекте x зависит не только от самого этого объекта, но и от выборки X, на которой алгоритм обучался:

$$X = \big((x_1,y_1),\ldots,(x_\ell,y_\ell)\big), \qquad a(x) = a(x,X)$$

Кроме того, значение y на объекте x зависит не только от x, но и от реализации шума в этой точке:

$$y(x) = y(x,\varepsilon)$$

Наконец, измерять качество мы бы хотели на тестовых объектах x — тех, которые не встречались в обучающей выборке, а тестовых объектов у нас в большинстве случаев более одного. При включении всех вышеперечисленных источников случайности в рассмотрение логичной оценкой качества алгоритма a кажется следующая величина:

$$Q(a) = \mathbb{E}_x \, \mathbb{E}_{X,\varepsilon} \big[y(x,\varepsilon) - a(x,X) \big]^2$$

Внутреннее матожидание позволяет оценить качество работы алгоритма в одной тестовой точке x в зависимости от всевозможных реализаций X и ε, а внешнее матожидание усредняет это качество по всем тестовым точкам.

Замечание. Запись $\mathbb{E}_{X,\varepsilon}$ в общем случае обозначает взятие матожидания по совместному распределению X и ε. Однако поскольку X и ε независимы, она равносильна последовательному взятию матожиданий по каждой из переменных: $\mathbb{E}_{X,\varepsilon} = \mathbb{E}_X \, \mathbb{E}_\varepsilon$, но последний вариант выглядит несколько более громоздко.

Попробуем представить выражение для $Q(a)$ в более удобном для анализа виде. Начнём с внутреннего матожидания:

$$\begin{aligned}
\mathbb{E}_{X,\varepsilon} \big[y(x,\varepsilon) - a(x,X) \big]^2 &= \mathbb{E}_{X,\varepsilon} \big[f(x) + \varepsilon - a(x,X) \big]^2 \\
&= \mathbb{E}_{X,\varepsilon} \Big[\underbrace{\big(f(x) - a(x,X) \big)^2}_{\text{не зависит от } \varepsilon} + 2\varepsilon \cdot \underbrace{\big(f(x) - a(x,X) \big)}_{\text{множители независимы}} + \varepsilon^2 \Big] \\
&= \mathbb{E}_X \big[(f(x) - a(x,X))^2 \big] + 2 \underbrace{\mathbb{E}_\varepsilon [\varepsilon]}_{=0} \cdot \mathbb{E}_X \big(f(x) - a(x,X) \big) + \mathbb{E}_\varepsilon \, \varepsilon^2 \\
&= \mathbb{E}_X \big[(f(x) - a(x,X))^2 \big] + \sigma^2.
\end{aligned}$$

Из общего выражения для $Q(a)$ выделилась шумовая компонента σ^2. Продолжим преобразования:

$$\begin{aligned}
\mathbb{E}_X \big[(f(x) - a(x,X))^2 \big] &= \mathbb{E}_X \big[(f(x) - \mathbb{E}_X[a(x,X)] + \mathbb{E}_X[a(x,X)] - a(x,X))^2 \big] \\
&= \underbrace{\big(f(x) - \mathbb{E}_X[a(x,X)] \big)^2}_{\operatorname{bias}^2_X \, a(x,X)} + \underbrace{\mathbb{E}_X \big[(a(x,X) - \mathbb{E}_X[a(x,X)])^2 \big]}_{\operatorname{Var}_X [a(x,X)]} \\
&\quad + 2 \big(f(x) - \mathbb{E}_X[a(x,X)] \big) \cdot \underbrace{\mathbb{E}_X \big[\mathbb{E}_X[a(x,X)] - a(x,X) \big]}_{=0} \\
&= \operatorname{bias}^2_X \, a(x,X) + \operatorname{Var}_X [a(x,X)].
\end{aligned}$$

Таким образом, итоговое выражение для $Q(a)$ примет вид

$$Q(a) = \mathbb{E}_x \, \mathbb{E}_{X,\varepsilon} \big[y(x,\varepsilon) - a(x,X) \big]^2 = \mathbb{E}_x \operatorname{bias}^2_X \, a(x,X) + \mathbb{E}_x \operatorname{Var}_X [a(x,X)] + \sigma^2,$$

Здесь

$$\operatorname{bias}_X \, a(x,X) = f(x) - \mathbb{E}_X [a(x,X)]$$

— смещение предсказания алгоритма в точке x, усреднённого по всем возможным обучающим выборкам, относительно истинной зависимости f;

$$\operatorname{Var}_X [a(x,X)] = \mathbb{E}_X \big[a(x,X) - \mathbb{E}_X [a(x,X)] \big]^2$$

— дисперсия (разброс) предсказаний алгоритма в зависимости от обучающей выборки X;

$$\sigma^2 = \mathbb{E}_x \, \mathbb{E}_\varepsilon \big[y(x,\varepsilon) - f(x) \big]^2$$

— неустранимый шум в данных.

Смещение показывает, насколько хорошо с помощью данного алгоритма можно приблизить истинную зависимость f, а разброс характеризует чувствительность алгоритма к изменениям в обучающей выборке. Например, деревья маленькой глубины будут в большинстве случаев иметь высокое смещение и низкий разброс предсказаний, так как они не могут слишком хорошо запомнить обучающую выборку. А глубокие деревья, наоборот, могут безошибочно выучить обучающую выборку и потому будут иметь высокий разброс в зависимости от выборки, однако их предсказания в среднем будут точнее. На рисунке ниже приведены возможные случаи сочетания смещения и разброса для разных моделей

Синяя точка соответствует модели, обученной на некоторой обучающей выборке, а всего синих точек столько, сколько было обучающих выборок. Красный круг в центре области представляет ближайшую

окрестность целевого значения. Большое смещение соответствует тому, что модели в среднем не попадают в цель, а при большом разбросе модели могут как делать точные предсказания, так и довольно сильно ошибаться.

27.3 Компромисс bias–variance

При построении моделей стремятся получить и низкий bias и адекватный variance

27.4 Анализ основных моделей

- kNN: low variance, average bias
- Linear models: very low variance, very high bias
- Trees: high variance, low bias for big trees. Depends on hyperparameters

27.5 За пределами MSE

Полученное нами разложение ошибки на три компоненты верно только для квадратичной функции потерь. Для других функций потерь существуют более общие формы этого разложения с похожими по смыслу компонентами. Это позволяет предполагать, что для большинства основных функций потерь имеется некоторое представление в виде смещения, разброса и шума (хоть и, возможно, не в столь простой аддитивной форме)

27.6 Double descent

Double Descent – феномен в машинном обучении, при котором ошибка теста модели сначала уменьшается, затем увеличивается, а затем снова уменьшается по мере роста сложности модели. Это контрастирует с традиционной U-образной кривой компромисса между предвзятостью и дисперсией, которая предполагает, что ошибка теста должна монотонно увеличиваться с ростом сложности модели после определённого момента.

Явление Double Descent наблюдается в сложных моделях машинного обучения, например в глубоких нейронных сетях.

Механизм

Процесс Double Descent включает три фазы

1. Фаза недопереобучения – по мере увеличения сложности модели ошибка теста уменьшается. Это характерно для моделей, которые не достаточно сложны, чтобы захватывать основные закономерности в данных.
2. Фаза переобучения – дальнейшее увеличение сложности модели приводит к увеличению ошибки теста. На этой фазе модели слишком сложны по сравнению с объёмом обучающих данных, захватывают шум как сигнал, что приводит к высокой дисперсии.
3. Второй спуск – по мере дальнейшего роста сложности модели (в режиме переобучения) ошибка теста снова начинает уменьшаться.

Причины

Явление Double Descent возникает из-за взаимодействия сложности модели, размера данных и шума. По мере увеличения сложности модели она сначала переобучается шуму в данных, что увеличивает ошибку теста. Однако по мере того, как модель становится ещё более сложной, она начинает интерполировать данные, меньше подгоняя шум и больше – сигнал, что снова уменьшает ошибку теста.

28 Bagging

Идея bagging (bootstrap aggregation) заключается в следующем. Пусть обучающая выборка состояла из n объектов. Выберем из неё n примеров равновероятно, с возвращением. Получим новую выборку X_1, в которой некоторых элементов исходной выборки не будет, а какие-то могут войти несколько раз. С помощью некоторого алгоритма b обучим на этой выборке модель $b_1(x) = b(x, X_1)$. Повторим процедуру: сформируем вторую выборку X_2 из n элементов с возвращением и с помощью того же алгоритма обучим на ней модель $b_2(x) = b(x, X_2)$. Повторив процедуру k раз, получим k моделей, обученных на k выборках. Чтобы получить одно предсказание, усредним предсказания всех моделей:

$$a(x) = \frac{1}{k}\big(b_1(x) + \cdots + b_k(x)\big)$$

28.1 Мотивация через bias–variance

Если предположить, что базовые алгоритмы некоррелированы, то:

$$\begin{aligned} V_X[a(x,X)] &= \frac{1}{k^2}\sum_{i=1}^{k} V_X b(x,X_i) \\ &= \frac{1}{k^2}\sum_{i=1}^{k} V_X b(x,X) \\ &= \frac{1}{k}\, V_X b(x,X). \end{aligned}$$

Получилось, что в этом случае дисперсия композиции в k раз меньше дисперсии отдельного алгоритма.

28.2 Bootstrap

Процесс генерации подвыборок с помощью семплирования с возвращением называется (bootstrap), а модели $b_1(x),\ldots,b_k(x)$ часто называют базовыми алгоритмами (хотя, наверное, лучше было бы назвать их базовыми моделями). Модель $a(x)$ называется ансамблем этих моделей.

Посмотрим, что происходит с качеством предсказания при переходе от одной модели к ансамблю. Сначала убедимся, что смещение ансамбля не изменилось по сравнению со средним смещением отдельных моделей. Будем считать, что когда мы берём матожидание по всем обучающим выборкам X, то в эти выборки включены также все подвыборки, полученные by bootstrap.

$$\begin{aligned} \operatorname{bias}_X a(x,X) &= f(x) - \mathbb{E}_X[a(x,X)] \\ &= f(x) - \mathbb{E}_X\Big[\frac{1}{k}\sum_{i=1}^{k} b(x,X_i)\Big] \\ &= f(x) - \frac{1}{k}\sum_{i=1}^{k} \mathbb{E}_X[b(x,X_i)] \\ &= f(x) - \frac{1}{k}\sum_{i=1}^{k} \mathbb{E}_X[b(x,X)] \\ &= f(x) - \mathbb{E}_X b(x,X) \\ &= \operatorname{bias}_X b(x,X). \end{aligned}$$

Получили, что смещение композиции равно смещению одного алгоритма. Теперь посмотрим, что происходит с разбросом.

$$\begin{aligned} V_X[a(x,X)] &= \mathbb{E}_X\big[a(x,X) - \mathbb{E}_X[a(x,X)]\big]^2 \\ &= \mathbb{E}_X\Big[\frac{1}{k}\sum_{i=1}^{k} b(x,X_i) - \mathbb{E}_X\Big[\frac{1}{k}\sum_{i=1}^{k} b(x,X_i)\Big]\Big]^2 \\ &= \frac{1}{k^2}\,\mathbb{E}_X\Big[\sum_{i=1}^{k}\big(b(x,X_i) - \mathbb{E}_X b(x,X_i)\big)\Big]^2 \\ &= \frac{1}{k^2}\sum_{i=1}^{k}\mathbb{E}_X\big(b(x,X_i) - \mathbb{E}_X b(x,X_i)\big)^2 \\ &\quad + \frac{1}{k^2}\sum_{k_1\neq k_2}\mathbb{E}_X\Big[\big(b(x,X_{k_1}) - \mathbb{E}_X b(x,X_{k_1})\big)\big(b(x,X_{k_2}) - \mathbb{E}_X b(x,X_{k_2})\big)\Big] \\ &= \frac{1}{k^2}\sum_{i=1}^{k} V_X b(x,X_i) + \frac{1}{k^2}\sum_{k_1\neq k_2}\operatorname{cov}\big(b(x,X_{k_1}), b(x,X_{k_2})\big). \end{aligned}$$

28.3 Метод случайных подпространств

Суть в том, что в каждой вершине случайно выбираются $n < N$ признаков, где N – полное число признаков (метод случайных подпространств), и среди них ищется оптимальный сплит. Такой приём как

раз позволяет управлять степенью скоррелированности базовых алгоритмов.

28.4 Случайный лес

В предыдущем разделе мы сделали предположение, что базовые алгоритмы некоррелированы, и за счёт этого получили очень сильное уменьшение дисперсии у ансамбля относительно входящих в него базовых алгоритмов. Однако в реальной жизни добиться этого сложно: ведь базовые алгоритмы учили одну и ту же зависимость на пересекающихся выборках. Поэтому будет странно, если корреляция на самом деле нулевая. Но на практике оказывается, что строгое выполнение этого предположения не обязательно. Достаточно, чтобы алгоритмы были в некоторой степени непохожи друг на друга. На этом строится развитие идеи bagging для решающих деревьев – случайный лес.

Построим ансамбль алгоритмов, где базовый алгоритм – это решающее дерево. Будем строить по следующей схеме:

1. Для построения i-го дерева
   - Сначала, как в обычном bagging, из обучающей выборки X выбирается с возвращением случайная подвыборка X_i того же размера, что и X.
   - В процессе обучения каждого дерева используется метод случайных подпространств
2. Чтобы получить предсказание ансамбля на тестовом объекте, усредняем отдельные ответы деревьев (для регрессии) или берём самый популярный класс (для классификации).
3. Мы построили Random Forest (случайный лес) – комбинацию bagging и метода случайных подпространств над решающими деревьями.

При построении случайного леса есть несколько степеней свободы

29 Задача обнаружения аномалий

29.1 Isolation Forest

Модель Isolation Forest: алгоритм поиска аномалий, эксплуатирующий простую идею. Есть дерево, которое разделяет выборку на две подвыборки на каждом шаге. Если поставить другую оптимизационную задачу и попросить отделять объект от всех остальных объектов, то можно отделить аномальные объекты. Смотрим на какой глубине каждый объект был отделен от всех остальных и выбросы как правило будут отделены ближе к корню. Одна из двух главных моделей для поиска аномалий в данный момент

29.2 One-Class SVM

Подходит, когда в обучающем наборе данные подчиняются нормальному распределению, а в тестовом содержат аномалии. Одноклассовый метод опорных векторов строит нелинейную поверхность вокруг начала координат. Возможно задать границу отсечения, какие данные считать аномальными.

30 GB

В прошлых разделах мы научились соединять базовые алгоритмы в ансамбль с помощью bagging (и, в частности, строить из решающих деревьев случайные леса). Теперь мы рассмотрим другой способ объединять базовые алгоритмы в композицию – градиентный бустинг.

В ходе обучения случайного леса каждый базовый алгоритм строится независимо от остальных. Бустинг, в свою очередь, воплощает идею последовательного построения линейной комбинации алгоритмов. Каждый следующий алгоритм старается уменьшить ошибку текущего ансамбля.

Бустинг, использующий деревья решений в качестве базовых алгоритмов, называется градиентным бустингом над решающими деревьями, (Gradient Boosting on Decision Trees, GBDT).

Он отлично работает на выборках с «табличными», неоднородными данными. Пример таких данных – описание пользователя Яндекса через его возраст, пол, среднее число поисковых запросов в день, число заказов такси и так далее. Такой бустинг способен эффективно находить нелинейные зависимости в данных различной природы.

Этим свойством обладают все алгоритмы, которые используют деревья решений, однако именно GBDT обычно выигрывает в подавляющем большинстве задач. Благодаря этому он широко применяется во многих конкурсах по машинному обучению и задачах из индустрии

- В поисковом ранжировании;
- рекомендательных системах;
- таргетировании рекламы;
- предсказании погоды;
- выбора пункта назначения такси и многих других.

Не так хорошо бустинг проявляет себя на однородных данных: текстах, изображениях, звуке, видео. В таких задачах нейросетевые подходы почти всегда демонстрируют лучшее качество.

И хотя деревья решений – традиционный выбор для объединения в ансамбли, никто не запрещает использовать и другие алгоритмы (например, линейные модели) в качестве базовых. Эта возможность реализована в пакете XGBoost.

Стоит только понимать, что построенная композиция окажется линейной комбинацией линейных моделей, то есть опять-таки линейной моделью - или нейросетью с одним полносвязным слоем. Это уменьшает возможности ансамбля эффективно определять нелинейные зависимости в данных. Поэтому в этом параграфе мы рассмотрим только бустинг над решающими деревьями.

30.1 Процедура построения для MSE

Рассмотрим задачу регрессии с квадратичной функцией потерь:

$$L(y,x) = \frac{1}{2}\sum_{i=1}^{N}\big(y_i - a(x_i)\big)^2 \;\to\; \min$$

Для решения будем строить композицию из K базовых алгоритмов:

$$a(x) = a_K(x) = b_1(x) + b_2(x) + \cdots + b_K(x)$$

Если мы обучим единственное решающее дерево, то качество такой модели, скорее всего, будет низким. Однако мы знаем, на каких объектах построенное дерево давало точные предсказания, а на каких ошибалось.

Попробуем использовать эту информацию и обучим ещё одну модель. Допустим, что предсказание первой модели на x_l на 10 больше, чем необходимо (то есть $b_1(x_l) = y_l + 10$). Если бы мы могли обучить новую модель, которая на x_l будет выдавать ответ -10, то сумма ответов этих двух моделей на объекте x_l в точности совпала бы с истинным значением:

$$b_1(x_l) + b_2(x_l) = (y_l + 10) + (-10) = y_l$$

Другими словами, если вторая модель научится предсказывать разницу между реальным значением и ответом первой, то это позволит уменьшить ошибку композиции.

В реальности вторая модель тоже не сможет обучиться идеально, поэтому обучим третью, которая будет «компенсировать» неточности первых двух. Будем продолжать так, пока не построим композицию из K алгоритмов.

Для объяснения метода градиентного бустинга полезно воспользоваться следующей аналогией. Бустинг можно представить как гольфиста, цель которого – загнать мяч в лунку с координатой y_{ball}. Положение мяча здесь – ответ композиции $a(x_{\text{ball}})$. Гольфист мог бы один раз ударить по мячу, не попасть в лунку и пойти домой, но настырность заставляет его продолжить.

По счастью, ему не нужно начинать каждый раз с начальной позиции. Следующий удар переводит мяч из текущего положения $a_k(x_{\text{ball}})$ в положение $a_{k+1}(x_{\text{ball}})$. Каждый следующий удар – это та поправка, которую вносит очередной базовый алгоритм в композицию. Если гольфист всё делает правильно, то функция потерь будет уменьшаться:

$$L\big(y, a_{k+1}(x)\big) < L\big(y, a_k(x)\big),$$

То есть мяч постепенно будет приближаться к лунке.

Удары при этом делаются не хаотично. Гольфист оценивает текущее положение мяча относительно лунки и следующим ударом старается нивелировать те проблемы, которые он создал себе всеми предыдущими. Подбираясь к лунке, он будет бить всё аккуратнее и, возможно, даже возьмёт другую клюшку, но точно не будет лупить так же, как из первоначальной позиции. В итоге комбинация всех ударов рано или поздно перенесет мяч в лунку.

Подобно тому, как гольфист постепенно подводит мяч к цели, бустинг с каждым новым базовым алгоритмом всё больше приближает предсказание к истинному значению метки объекта.

Рассмотрим теперь другую аналогию — разложение функции в ряд Тейлора. Из курса математического анализа известно, что (достаточно хорошую) бесконечно дифференцируемую функцию $f(x)$ на интервале $x \in (a-R, a+R)$ можно представить в виде бесконечной суммы степенных функций:

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!}(x-a)^n.$$

Одна, самая первая степенная функция в разложении, очень грубо приближает $f(x)$. Прибавляя к ней следующую, мы получим более точное приближение. Каждая следующая элементарная функция увеличивает точность приближения, но менее заметна в общей сумме. Если нам не требуется абсолютно точное разложение, вместо бесконечного ряда Тейлора мы можем ограничиться суммой его первых k элементов. Таким образом, интересующую нас функцию мы с некоторой точностью представили в виде суммы «простых» функций.

Перенесём эту идею на задачи машинного обучения. В машинном обучении мы пытаемся по выборке (x_i,y_i) восстановить неизвестную истинную зависимость. Прежде всего, мы выбираем подходящий алгоритм. Мы можем выбрать «сложный» алгоритм, который сразу хорошо выучит истинную зависимость.

А можем обучить «простой», который выучит истинную зависимость посредственно. Затем мы добавим к нему ещё один такой простой алгоритм, чтобы уточнить предсказание первого алгоритма. Продолжая этот процесс, мы получим сумму простых алгоритмов, где первый алгоритм грубо приближает истинную зависимость, а каждый следующий делает приближение всё точнее.

30.2 Бустинг как градиентный спуск

Градиентный бустинг (Gradient Boosting) можно рассматривать как градиентный спуск в пространстве функций. Вместо оптимизации параметров $\boldsymbol{\theta}$ модели $f(\boldsymbol{x};\boldsymbol{\theta})$, мы оптимизируем саму функцию $f : \mathbb{R}^d \to \mathbb{R}$.

Формальная постановка

Рассмотрим задачу минимизации эмпирического риска:

$$\min_{f \in \mathcal{F}} \mathcal{L}(f) = \sum_{i=1}^{n} L(y_i, f(\boldsymbol{x}_i))$$

где:

- $L : \mathbb{R} \times \mathbb{R} \to \mathbb{R}_+$ — функция потерь
- $\mathcal{F}$ — пространство функций (например, деревья решений)
- $f(\boldsymbol{x})$ — наша модель

Градиентный спуск в пространстве функций

На каждой итерации t обновляем функцию:

$$f_t(\boldsymbol{x}) = f_{t-1}(\boldsymbol{x}) - \eta_t \cdot g_t(\boldsymbol{x})$$

Здесь $g_t(\boldsymbol{x})$ — "градиент" функции потерь по f в точке f_{t-1}.

Градиент функции потерь

Для фиксированной точки $(\boldsymbol{x}_i, y_i)$ рассмотрим производную:

$$g_i = \left.\frac{\partial L(y_i, z)}{\partial z}\right|_{z=f_{t-1}(\boldsymbol{x}_i)}$$

Нам нужно найти функцию $h_t \in \mathcal{F}$, которая лучше всего аппроксимирует отрицательный градиент:

$$h_t \approx -g_t$$

Здесь $g_t(\boldsymbol{x}_i) = g_i$

30.2.1 вывод для кросс-энтропии

Функция потерь и вероятностная модель
Для бинарной классификации с $y_i \in \{0, 1\}$ используем кросс-энтропийную потерь:

$$L(y, f(\boldsymbol{x})) = -y \log(p(\boldsymbol{x})) - (1 - y) \log(1 - p(\boldsymbol{x}))$$

Здесь $p(\boldsymbol{x}) = P(y = 1 \mid \boldsymbol{x}) = \sigma(f(\boldsymbol{x})) = \frac{1}{1+e^{-f(\boldsymbol{x})}}$.
Вычисление градиента
Выразим потерю через $f(\boldsymbol{x})$:

$$\begin{aligned} L(y, f) &= -y \log\left(\frac{1}{1 + e^{-f}}\right) - (1 - y) \log\left(1 - \frac{1}{1 + e^{-f}}\right) \\ &= -y \log\left(\frac{1}{1 + e^{-f}}\right) - (1 - y) \log\left(\frac{e^{-f}}{1 + e^{-f}}\right) \\ &= y \log\big(1 + e^{-f}\big) + (1 - y) \left[\log\big(1 + e^{-f}\big) - \log\big(e^{-f}\big)\right] \\ &= y \log\big(1 + e^{-f}\big) + (1 - y) \left[\log\big(1 + e^{-f}\big) + f\right] \\ &= \log\big(1 + e^{-f}\big) + (1 - y) f \end{aligned}$$

Теперь вычислим градиент:

$$\begin{aligned} \frac{\partial L(y, f)}{\partial f} &= \frac{\partial}{\partial f} \left[\log\big(1 + e^{-f}\big) + (1 - y) f\right] \\ &= \frac{-e^{-f}}{1 + e^{-f}} + (1 - y) \\ &= -\frac{1}{1 + e^{f}} + (1 - y) \\ &= (1 - y) - \sigma(-f) \\ &= (1 - y) - \frac{1}{1 + e^{f}} \end{aligned}$$

Упростим дальше. Заметим, что $\sigma(-f) = 1 - \sigma(f)$, поэтому:

$$\begin{aligned} \frac{\partial L(y, f)}{\partial f} &= (1 - y) - [1 - \sigma(f)] \\ &= \sigma(f) - y \end{aligned}$$

Таким образом, получаем простую формулу:

$$\boxed{\frac{\partial L(y, f)}{\partial f} = p(\boldsymbol{x}) - y}$$

Здесь $p(\boldsymbol{x}) = \sigma(f(\boldsymbol{x}))$.
Интерпретация градиента
Для каждого объекта $(\boldsymbol{x}_i, y_i)$:

$$g_i = \left.\frac{\partial L(y_i, f)}{\partial f}\right|_{f=f_{t-1}(\boldsymbol{x}_i)} = p_{t-1}(\boldsymbol{x}_i) - y_i$$

Здесь $p_{t-1}(\boldsymbol{x}_i) = \sigma(f_{t-1}(\boldsymbol{x}_i))$ — предсказанная вероятность класса 1 на предыдущей итерации.

- Если $y_i = 1$ и $p_{t-1}(\boldsymbol{x}_i) = 0.9$, то $g_i = -0.1$ (немного уменьшаем f)
- Если $y_i = 0$ и $p_{t-1}(\boldsymbol{x}_i) = 0.9$, то $g_i = 0.9$ (сильно увеличиваем f)
- Если $y_i = 1$ и $p_{t-1}(\boldsymbol{x}_i) = 0.1$, то $g_i = -0.9$ (сильно уменьшаем f)

Практические аспекты
Регуляризация

1. **Скорость обучения (learning rate)** η: значения ($\eta < 0.1$) улучшают обобщающую способность

2. **Глубина деревьев**: ограничивает сложность базовых моделей
3. **Subsampling**: использование подвыборки объектов на каждой итерации
4. **Column subsampling**: использование подвыборки признаков

Преимущества градиентной интерпретации

1. **Гибкость**: можно использовать любую дифференцируемую функцию потерь
2. **Регуляризация**: легко добавлять регуляризаторы
3. **Интерпретируемость**: можем анализировать вклады каждой итерации

Таким образом

- Градиентный бустинг можно рассматривать как градиентный спуск в пространстве функций
- Для кросс-энтропийной потери градиент имеет простую форму: $p(\boldsymbol{x}) - y$
- Это объясняет, почему на каждой итерации мы обучаем модель на остатках $y - p(\boldsymbol{x})$
- Градиентная интерпретация позволяет обобщать бустинг на различные функции потерь и задачи

Градиентный бустинг остаётся одним из самых мощных методов машинного обучения, и понимание его как градиентного спуска помогает как в его применении, так и в разработке новых модификаций.

30.3 Выбор базовой модели

Why not linear? Why not kNN or others? Why trees as basic algorithms?

1. GB doesn't need differentiable base model

   target is given independently between iterations

2. Trees have high variance

   gradient varies a lot between iterations

   это означает высокую дисперсию целевых показателей для базовой модели

3. Error on any step doesn't ruin whole algorithm

   we recompute true gradients on each step

   meta-algorithm (GB) fixes such errors

So for now trees are the only option for GB

30.4 Параллелизм

Сам градиентный бустинг не parallelized, но так как обычно его делают на деревьях, то можно распараллелить процесс построения каждого дерева

31 Библиотеки градиентного бустинга

Для общего развития имеет смысл посмотреть реализацию в sklearn, но на практике она весьма медленная и не умная

31.1 XGBoost, LightGBM, CatBoost

Хороших реализаций GBDT есть, как минимум, три: LightGBM, XGBoost и CatBoost. Исторически они отличались довольно сильно, но за последние годы успели скопировать друг у друга все хорошие идеи

31.2 Плюсы и минусы каждой

Одно из основных отличий LightGBM, XGBoost и CatBoost – форма решающих деревьев. LightGBM строит деревья по принципу: «На каждом шаге делим вершину с наилучшим скором», а основным критерием остановки выступает максимально допустимое количество вершин в дереве. Это приводит к тому, что деревья получаются несимметричными, то есть поддеревья могут иметь разную глубину. Например, левое поддерево может иметь глубину 2, а правое может разрастись до глубины 15.

С одной стороны, это позволяет быстро подогнаться под обучающие данные. С другой – бесконтрольный рост дерева в глубину неизбежно ведет к переобучению, поэтому LightGBM позволяет помимо количества вершин ограничивать и максимальную глубину. Впрочем, это ограничение обычно все равно выше, чем для XGBoost и CatBoost.

XGBoost строит деревья по принципу: «Строим дерево последовательно по уровням до достижения максимальной глубины». Отдельного ограничения на количество вершин нет, так как оно естественным образом получается из ограничения на глубину дерева. В XGBoost деревья «стремятся» быть симметричными по глубине, и в идеале получается полное бинарное дерево, если это не противоречит другим ограничениям (например, ограничению на минимальное количество объектов в листе). Такие деревья обычно являются более устойчивыми к переобучению.

CatBoost строит деревья по принципу: «Все вершины одного уровня имеют одинаковый предикат». Одинаковые сплиты во всех вершинах одного уровня позволяют избавиться от ветвлений (конструкций if-else) в коде инференса модели с помощью битовых операций и получить более эффективный код, который в разы ускоряет применение модели, в особенности в случае применения на батчах.

Кроме этого, такое ограничение на форму дерева выступает в качестве сильной регуляризации, что делает модель более устойчивой к переобучению. Основной критерий остановки, как и в случае XGBoost, – ограничение на глубину дерева. Однако, в отличие от XGBoost, в CatBoost всегда создаются полные бинарные деревья, несмотря на то, что в некоторые поддеревья может не попасть ни одного объекта из обучающей выборки.

32 Интерпретируемое ML

32.1 Таксономия по свойствам

32.2 Линейные модели, важности в деревьях

32.3 LIME

32.4 Подход SHAP (Shapley additive explanations)

32.4.1 Аксиомы, формула

32.4.2 SHAP-значения для ML

32.4.3 KernelSHAP, TreeSHAP

32.5 GradCAM

33 Оптимизация гиперпараметров (HP)

In optimization theory methods are associated with the order of derivatives they use.

33.1 Классы задач оптимизации

Main ones are

- **First order optimization** (gradient based optimization)
  - use gradient of optimized function
  - например стохастический градиентный спуск
- **Second order optimization** (Newton's method)
  - use Hessian matrix
  - they are quite slow
- **Zero order** (black box optimization)
  - don't need gradient, only values of optimized function

33.2 Основные подходы к оптимизации HP

0-order optimization approaches

1. Manual trials
2. Grid search
3. Random search

1. Manual trials
2. Grid search
3. Random search
4. Bayesian methods
5. Evolutionary methods

33.3 Библиотеки для оптимизации HP

- HYPEROPT. В библиотеке реализованы три метода оптимизации гиперпараметров: Random Search, TPE, Adaptive TPE Кроме того, у них есть обёртка над sklearn, позволяющая работать с моделями оттуда
- OPTUNA. В библиотеке реализованы те же методы оптимизации, что и в Hyperopt, но по многим параметрам она удобнее.
- sklearn. В библиотек есть реализации Grid Search и Random Search, что очень удобно, если вы используете её модели
- В библиотеке Scikit-Optimize реализованы алгоритмы байесовской оптимизации и Random Search. Кроме самих методов оптимизации библиотека предоставляет отличный инструментарий для различных.
- Библиотека Keras Tuner позволяет подбирать гиперпараметры для нейросеток, написанных на TensorFlow 2.0, и для обычных моделей из Scikit-learn. Доступные методы оптимизации – Random Search и Hyperband

Часть VII
Нейросети, матричные производные и базовые слои

34 Матричное исчисление и производные матричных операций

Базовые операции

$$
\begin{aligned}
&A(B+C) = AB + AC,\\
&(A+B)^T = A^T + B^T,\\
&(AB)^T = B^T A^T,\\
&(AB)^{-1} = B^{-1}A^{-1},\\
&(A^{-1})^T = (A^T)^{-1}.\\
&\det(AB) = \det A \det B,\\
&\det\left(A^{-1}\right) = 1/\det A,\\
&\det A = \prod_j \lambda_j,\\
&\operatorname{tr} A = \sum_j A_{jj} = \sum_j \lambda_j,\\
&\operatorname{tr}(ABC) = \operatorname{tr}(BCA) = \operatorname{tr}(CAB).
\end{aligned}
$$

Производные следа и определителя

$$\frac{\partial}{\partial A}\operatorname{tr}(AB) = B^T,$$
$$\frac{\partial}{\partial A}\det A = (\det A)\,(A^{-1})^T,$$
$$\frac{\partial}{\partial x}\log\det A(x) = \operatorname{tr}(A^{-1}\frac{\partial A}{\partial x}).$$

Производные

$$\frac{\partial}{\partial x}x^T a = a,$$
$$\frac{\partial}{\partial x}x^T Ax = (A + A^T)x,$$
$$\frac{\partial}{\partial A}x^T Ay = xy^T,$$
$$\frac{\partial}{\partial x}A^{-1} = -A^{-1}\frac{\partial A}{\partial x}A^{-1}.$$

Примеры вычисления градиентов

1. $y = \operatorname{tr}(AB)$

$$\operatorname{tr}(AB) = \sum_{i=1}^{n} a_{1i}b_{i1} + \cdots + \sum_{i=1}^{n} a_{ni}b_{in} \quad \Longrightarrow \quad \frac{dy}{da_{ij}} = b_{ji}, \qquad \frac{dy}{dA} = B^T.$$

34.1 Производная скалярного произведения

34.1.1 Для двух векторов

$$y = x^T x, \quad x \in \mathbb{R}^N$$
$$\frac{d}{dx_i}x^T x = \frac{d}{dx_i}\sum x_i^2 = \frac{d}{dx_i}x_i^2 = 2x_i, \qquad \frac{dy}{dx} = 2x.$$
$$y = a^T x$$
$$\frac{d}{dx_i}a^T x = \frac{d}{dx_i}\sum a_i x_i = a_i, \qquad \frac{d}{dx}a^T x = a.$$

34.1.2 Для матрицы и вектора

$$y = Ax$$
$$\frac{d}{dx_i}[Ax]_k = \frac{d}{dx_i}\sum_j a_{kj}x_j = a_{ki}, \qquad \frac{d}{dx}Ax = A$$